DOCUMENTATION

MedEnroll834

Documentation — A Modern Distributed Data Platform for Growing Data

PySpark · Delta Lake · Databricks · Apache Airflow · PostgreSQL

SIVABALA SELVAM

Table of Contents

00. Healthcare Enrollment Primer **18**

What is an "834"? 18

Why "HIPAA X12"? 18

Who is enrolling in what? The core entities 19

Medicaid, Medicare, and the marketplace — who pays, and who's eligible 20

Why enrollment data is time-sensitive 20

Relationship codes used in this project 21

Terms you'll see throughout this documentation 21

01. Project Overview **23**

Scope 23

Objectives 23

Conceptual model 23

Target workflow 24

Design principles 24

Success measures 25

02. Business Problem **26**

Business drivers 26

Current constraint (observed) 26

The three-V challenge 26

Volume — an industry-wide growth trend, not a one-time spike 26

Velocity 27

Variety 27

Business objective 27

03. Legacy Architecture 28

Existing platform 28

Processing flow 28

Operational limitations 29

Modernization boundary 29

04. Modern Architecture 30

Idea: Modern Distributed Data Platform for Growing Data 30

Three capabilities 30

Capability 1 — Distributed Processing 30

Capability 2 — Transactional Platform 30

Capability 3 — Analytical Platform 31

Full architecture diagram 31

Architecture layers 31

4. Analytical consumption — two modes 32

5. Orchestration — Airflow 32

6. POC vs. enterprise storage 32

05. Technology Decisions 33

Processing engine: PySpark 4.0+ / Delta Lake 4.0+ 33

Orchestration: Apache Airflow 3.x (TaskFlow API, CeleryExecutor) 33

Compute platform: Databricks (Free/Serverless tier compatible) 34

Operational store: PostgreSQL (OLTP) 34

Object storage: S3-compatible blob storage (POC) / AWS S3 (enterprise target) 34

Containerization: Docker / Docker Compose 34

CI/CD: GitHub Actions 35

Summary table 36

06. Project Repository Structure 37

The src/medenroll834/ package 38

Key configuration files 38

07. AWS / S3 Setup 39

Terraform-managed S3 bucket 39

Folder layout provisioned as code 40

Least-privilege IAM policy 40

Applying the Terraform 40

The S3 client interface (enterprise target) 41

AWS Cloud Shell backout 41

08. PostgreSQL Setup 42

Connection configuration 42

Deploying the schema 42

Rolling back 43

Verifying a deployment 43

Schemas provisioned 44

09. Database Schema 45

OLTP schema (PostgreSQL) 45

CORE schema 46

ENRL schema 46

Example DDL — CORE_MEME_MEMBER 46

Foreign keys 47

Indexes 47

OLAP star schema (Delta Lake) 48

Dimensions (all SCD Type 2 via CURRENT_FLAG, except DIM_DATE) 49

Schema deployment order 49

10. Synthetic 834 Data Generation 50

Reference data pools 50

X12 envelope construction 50

Member, subscriber, and group identifiers 51

Generating a batch 52

Direct-to-Databricks generation 52

messy mode 53

Used by both tests and the Airflow DAG 53

11. PySpark Development Environment 54

Delta package resolution 54

Shared by tests and scripts 55

Setting up locally 55

12. 834 Parser 56

Segment splitting 56

Parsing a directory of files 57

Schema 57

The relationship code map 58

Also supports in-memory DataFrames 58

13. Bronze Layer **59**

BronzeLayer.ingest() 61

Write semantics 62

Output schema 62

Databricks notebook equivalent 63

Pipeline summary contribution 63

14. Silver Layer **64**

Why HL resolution is hard (and how it's solved) 64

Deriving member row ranges from HL boundaries 65

Subscriber resolution without a fan-out join 66

Date formatting 66

Output schema 67

15. Business Rules / Plan Assignment **68**

Where the plan code comes from 68

Valid plan codes (the business rule) 68

Relationship-driven subscriber assignment 69

Gold finalizes the subscriber assignment 69

Enrollment maintenance type (OLTP load) 70

16. Data Quality & Reject Handling 71

Validation rules 71

The six validation rules 72

is_valid flag 72

Quarantine routing 73

Where quarantine fits in the pipeline 73

OLTP-side rejects (ENRL_RJRW_REJECT) 73

Pipeline run summary 74

17. PostgreSQL Bulk Load / Upsert 75

Reading Gold from Databricks 75

The load: one transaction, natural-key upserts 76

Upsert pattern (ON CONFLICT ... DO UPDATE) 76

Member and enrollment inserts (not upserted) 77

One transaction row created per file 77

Helper conversions 77

Invoked from the Airflow DAG 78

Verified load evidence 78

18. Gold Layer 79

Full transform 80

What Gold does, step by step 81

Write semantics 81

Gold's row count in the pipeline summary 82

19. OLAP Star Schema 83

Dimension builds (all SCD Type 2-shaped via CURRENT_FLAG) 84

DIM_DATE — generated date spine 84

DIM_TRANSACTION_SOURCE — hashed file identity 85

FACT_ENROLLMENT — resolving surrogate keys 86

Verifying the star schema 87

Verified fact table evidence 87

20. Airflow Architecture 88

Stack topology 88

Project mount and dependency injection 89

Configuration precedence 90

Why CeleryExecutor (not LocalExecutor) 90

21. Airflow DAG Implementation 91

DAG declaration 91

Task graph 91

Why files are staged into a Unity Catalog Volume 92

Running and waiting on the Databricks job 93

Archiving is scoped and success-gated; failures are quarantined, not archived 94

Genie is skippable 95

Validated DAG import 95

Verified run evidence 95

22. Idempotency & Incremental Processing 96

Delta layer idempotency: overwrite + natural grain 96

Gold-level deduplication 96

OLTP idempotency: natural-key upserts 96

OLAP idempotency: deterministic hashed keys 97

Transactional atomicity 97

Incremental processing: Change Data Feed 97

File-level idempotency boundary 98

23. Error Handling & Retry 99

Airflow-level retries 99

Explicit failure propagation from Databricks 99

Transactional rollback (OLTP) 99

Per-query error isolation in reporting 100

Operational failure handling: archive vs. quarantine 100

Validation errors are not failures 101

Known error classes and their handling 102

Verified archive evidence 103

24. Observability 104

Standardized, configurable logging 104

Pipeline run-context correlation 105

Error logging 106

Pipeline-level run summaries 106

Airflow-native observability 107

OLTP and OLAP lineage columns 107

What's deliberately out of scope for this POC 107

25. Spark Performance Optimization **108**

Adaptive Query Execution (AQE) 108

Broadcast map instead of a self-join (biggest algorithmic win) 108

Window-based "carry forward latest value" instead of a loop 109

Delta write-time optimization 109

Delta retention tuning 109

Change Data Feed for future incremental reads 110

Databricks-side: read_files() + explode() over a Python loop 110

26. Benchmark Framework **111**

What already exists to build a benchmark on 111

1. Structured per-run summaries 111

2. Timestamped layer-boundary logs 111

3. A known, reproducible smoke-test baseline 112

Proposed framework design (not yet implemented) 112

What a benchmark should measure 113

27. Databricks SQL Dashboard **114**

The seven report queries 114

Executing against a SQL warehouse 115

Rendering self-contained HTML 115

Invoked from the Airflow DAG 116

Verified report evidence 116

28. Genie / GenAI 117

Implementation 117

Invoked from the Airflow DAG (optional) 118

Enabling Genie 118

Why Genie is optional, not required 118

Positioning note (why this matters architecturally) 118

Verified Genie evidence 119

29. Security 120

Credential handling 120

Blob storage credentials 120

AWS enterprise-target credentials 120

Secrets live only in .env (git-ignored) 120

IAM least privilege (S3) 121

Database connection security 121

Transactional integrity as a security property 121

PHI/PII posture 121

CI security scanning 122

30. Testing 123

Test coverage by layer 123

Running the suite 123

DAG validation (outside pytest) 123

What is and isn't covered 124

31. CI/CD 125

CI pipeline (ci.yml) 125

CD pipeline (cd.yml) 126

Local equivalents (Makefile) 127

32. Docker / Local Development 128

Starting the stack 128

Verifying the stack is healthy 128

Dockerfile (standalone package image) 128

Backing out the local stack 129

Full environment teardown (all deployed components) 129

Suggested teardown order (if backing out everything) 129

Verifying a backout 130

Local Python development (without Docker) 130

33. End-to-End Execution 131

Flow 131

Already-provisioned prerequisites 131

Step 1 — Fill in secrets in .env 131

Step 2 — Recreate the Airflow stack 132

Step 3 — Open Airflow and run 132

Step 4 — Verify outputs 132

Re-deploying the Databricks notebook after a code change 132

Troubleshooting 133

Enterprise migration note 133

Local (non-Airflow) smoke test 133

34. Legacy vs Modern Benchmark 134

Legacy system (documented constraint) 134

Modern pipeline (measured, smoke-test scale) 134

Qualitative comparison (architecturally grounded) 135

What a rigorous head-to-head benchmark would require 136

35. Business Impact 137
Velocity 137
Volume 137
Variety 137
Operational cost reduction 137
New analytical capability (not present in the legacy system at all) 138
Compliance and audit posture 138
36. Portfolio README 139
MedEnroll834 — Modern Distributed Data Platform for Growing Data 139
Highlights 139
Tech stack 140
What this demonstrates 140
Links 140
37. Future Improvements 141
Testing 141
Performance & benchmarking 141
Data model 141
Observability 142

Architecture 142

Documentation 142

00. Healthcare Enrollment Primer

This chapter is for readers new to U.S. healthcare administration. It defines the terms, programs, and regulatory concepts used throughout this documentation — read it first if phrases like "834," "Medicaid managed care," or "HIPAA X12" aren't already familiar.

What is an "834"?

834 is the name of a specific electronic document format — formally, the **ANSI ASC X12 834 Benefit Enrollment and Maintenance transaction**. Health insurers, employers, and government agencies use it to tell each other "this person is enrolling in (or changing, or ending) this health plan." It is one of dozens of numbered X12 transaction types used in U.S. healthcare administration; others include **837** (claims), **835** (payment/remittance), and **270/271** (eligibility inquiry/response). This project processes only 834 files.

An 834 file is not human-readable prose — it is a structured text format with fixed segment codes (`ISA`, `GS`, `ST`, `HL`, `NM1`, `DTP`, `HD`, ...), each carrying specific pieces of information (a name, a date, a plan code) in a fixed position. Chapter 12 covers exactly how this project parses that structure.

Why "HIPAA X12"?

HIPAA (the Health Insurance Portability and Accountability Act of 1996) is the U.S. federal law best known for patient-privacy rules, but it also *mandates* that health plans and providers use standardized electronic transaction formats — specifically, the **X12** standard published by ASC X12 — for exchanges like enrollment, claims, and eligibility checks. "HIPAA X12 834" is shorthand for "the federally mandated standard enrollment transaction format." Using a non-standard, proprietary format for these exchanges is not legally permitted for covered entities, which is part of why every health plan ingests the same 834 structure regardless of which vendor built its systems.

Who is enrolling in what? The core entities

Term	Plain-language meaning
Member	A specific person covered by a health plan — could be the policyholder or one of their dependents.
Subscriber	The member who is the primary policyholder — the person whose employment, Medicaid eligibility, or marketplace application the coverage is based on.
Dependent	A member covered *through* a subscriber's policy — a spouse, child, or domestic partner, per the 834's relationship code.
Employer group	The organization (an employer, or a state Medicaid program) whose contract with the health plan covers a population of subscribers.
Benefit plan	The specific product a member is enrolled in (e.g. an HMO, a PPO, a specific metal tier) — determines what's covered and how.
Enrollment	The event of a member being added, changed, or termed on a benefit plan — the "transaction" an 834 file communicates.

These six concepts map directly onto this project's data model — see Chapter 1's conceptual model and Chapter 9's schema.

Medicaid, Medicare, and the marketplace — who pays, and who's eligible

Program	Who it's for	Who runs it
Medicaid	Low-income individuals and families; eligibility rules vary by state	Jointly funded by federal and state governments; administered by each state, often through contracted **managed care organizations**
CHIP (Children's Health Insurance Program)	Children in families who earn too much for Medicaid but can't afford private coverage	Same federal/state structure as Medicaid, often administered alongside it
Medicare	People 65+, or younger people with certain disabilities	Fully federal (via CMS — the Centers for Medicare & Medicaid Services)
Medicare dual-eligible	People who qualify for *both* Medicare and Medicaid	Coordinated between federal Medicare and state Medicaid benefits
ACA Marketplace (also "Affordable Care Act Marketplace" or "Exchange")	Individuals/families buying private coverage directly, often with income-based federal subsidies	Federal framework (HealthCare.gov or a state-run exchange); plans sold by private insurers
Commercial/employer-sponsored	Employees (and dependents) covered through an employer's group health plan	Fully private, negotiated between employer and insurer

A **managed care organization (MCO)** is a private health plan that a state *contracts with* to administer Medicaid (and sometimes Medicare dual-eligible and Marketplace) benefits on the state's behalf — this is the kind of organization whose enrollment processing this project models (see Chapter 2).

Why enrollment data is time-sensitive

A member's coverage record has to be accurate and *current* before a pharmacy, doctor's office, or hospital can rely on it — a claim submitted for a service the plan doesn't yet show the member as covered for will be rejected or delayed, even if the member is, per the state or employer, actually enrolled. This is the real-world reason enrollment file processing speed ("Velocity" in the Three-V framing used throughout this documentation) matters operationally, not just as an abstract performance metric.

Relationship codes used in this project

The 834's `INS` segment carries a numeric relationship code identifying how a dependent relates to their subscriber. This prototype's generator and validator use four of the most common codes:

Code	Relationship
`18`	Self (the subscriber)
`01`	Spouse
`03`	Child
`19`	Domestic partner

Terms you'll see throughout this documentation

Term	Meaning
EDI (Electronic Data Interchange)	The general practice of exchanging structured business documents (like 834s) electronically between organizations, instead of paper/fax.
Plan code / plan tier	A short code identifying a specific benefit plan product (this project's synthetic data uses tiers like `GOLD`, `SILVER`, `BRONZE`, `PLATINUM`, `HDHP`, `EPO`, `PPO`, `HMO`).
Coverage start/end date	The date range a member's enrollment in a plan is effective — an open-ended (still-active) enrollment has no end date yet.
Termination ("term")	Ending a member's coverage on a plan, as opposed to adding or changing it.
Group number	An identifier for the employer group or state contract a subscriber's coverage is issued under.
PHI / PII	Protected Health Information / Personally Identifiable Information — the categories of sensitive data HIPAA and general privacy law protect. This project uses **zero** real PHI/PII; all data is synthetically generated (see Chapter 10).

With these fundamentals in place, Chapter 1 picks up with how this project's own pipeline is scoped and structured.

01. Project Overview

Scope

MedEnroll834 is a modernization prototype for healthcare enrollment processing. It models a production-style pipeline for inbound HIPAA X12 834 transactions — the standard EDI format carriers and employers use to enroll, change, and terminate member benefit coverage.

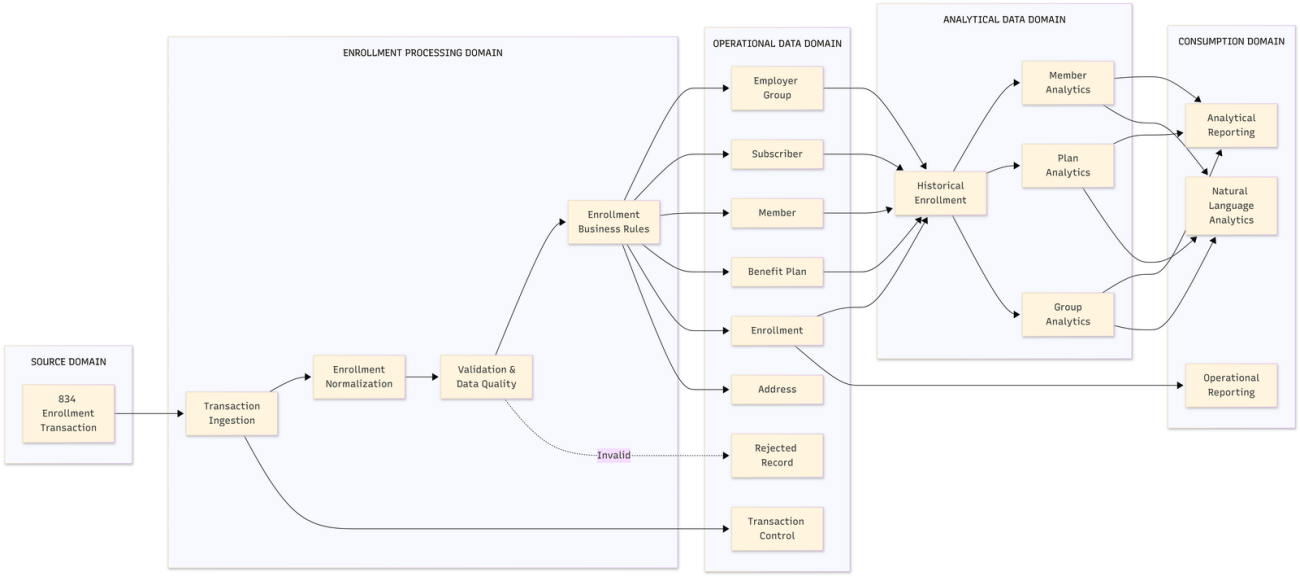
The prototype demonstrates a faster, more observable, and more reliable operating model than the legacy file-handoff chain it replaces. **It does not use production member data** — all 834 files are synthetically generated (see Chapter 10).

Objectives

- Handle high-volume enrollment files through distributed processing (PySpark on Databricks) instead of a single-machine batch job.
- Preserve raw source data and track every transformation (Bronze → Silver → Gold medallion architecture with Delta Lake).
- Serve both **operational** enrollment processing (PostgreSQL OLTP) and **analytical** reporting (Delta Lake star schema + Databricks SQL), explicitly kept as two separate consumption paths.
- Demonstrate modern data-engineering practices — idempotent pipelines, data-quality quarantine, Airflow orchestration, CI/CD, and Infrastructure as Code — in a this project.

Conceptual model

At a conceptual level, five business entities drive the whole system: a **member** belongs to a **subscriber**'s policy, the subscriber is covered under an **employer group**'s contract, that group offers one or more **benefit plans**, and every addition, change, or termination to a member's coverage is captured as an **enrollment** event. The pipeline's entire job is to take an inbound 834 file, resolve these five entities and the relationships between them, and make the result available two ways at once: as live, queryable records for **operational** use (a member's current plan, a subscriber's dependents) and as historical, aggregated records for **analytical** use (enrollment trends by plan, by group, by time period).



Rejected/invalid records enter a controlled remediation path (quarantine) rather than silently failing or corrupting downstream tables — see Chapter 16.

Target workflow

1. Receive an 834 file from a (synthetic, in this POC) external source.
2. Land the original file in object storage (S3-compatible blob storage in the POC, AWS S3 in the enterprise target).
3. Parse, validate, and normalize member enrollment transactions with PySpark (`EDI834Parser`, `SilverLayer`).
4. Apply plan-assignment business rules (`GoldLayer`).
5. Load operational enrollment data into PostgreSQL (`load_gold_to_oltp`).
6. Publish curated Delta tables (OLAP star schema) for analytics and dashboards (Databricks SQL + Genie).

Apache Airflow coordinates the end-to-end workflow (`med834_e2e_pipeline` DAG). Databricks provides scalable processing, Delta Lake storage, and SQL analytics.

Design principles

- **Idempotent and incremental** — natural-key upserts (`ON CONFLICT ... DO UPDATE`) in the OLTP loader, `overwrite` / `append` semantics with `mergeSchema` in the Delta layers.
- **Invalid records are quarantined** with actionable rejection reasons (`validation_errors` array column), not silently dropped.
- **Every run emits metrics, logs, and lineage signals** — `setup_logging`, `log_pipeline_start` / `log_pipeline_end`, and run summaries returned from each layer (`{"bronze_segments": ..., "silver_valid": ..., "gold_records": ...}`).

Success measures

The prototype targets the legacy system's velocity, volume, and variety constraints described in Chapter 2. A validated smoke-test run (1 synthetic file, 3 subscribers + 3 dependents) produced:

```
BRONZE     : segments_loaded = 83
SILVER     : total = 6, valid = 6, invalid = 0
GOLD       : records = 6
QUARANTINE : records = 0
```

See Chapter 34 for the full legacy vs. modern comparison and what is measured vs. what is a design target.

02. Business Problem

Business drivers

Healthcare enrollment processing relies on timely, accurate HIPAA 834 transactions. The current (legacy) workflow uses a chain of specialized tools and scheduled jobs rather than a unified, distributed pipeline.

An inbound 834 file arrives in a designated folder. An **EDI** job converts the transaction into a proprietary format. An **ECA** job evaluates each member and assigns a plan. A **Legacy Programs** process generates a keyword file for the downstream **CORE System**. **CORE Programs** then loads the enrollment result.

The legacy implementation chains together **Tidal** scheduling, **SSIS**, **DataStage**, **Legacy Programs**, **SCP** file movement, **Bash** validations, and the **CORE Programs** load. Each handoff adds an operational dependency and a potential failure point — see Chapter 3 for the full flow diagram.

Current constraint (observed)

Enrollment processing jobs in the legacy system are observed to take approximately **one hour** end to end for a full daily batch file. Database locks and single-machine processing limit throughput. Investigating a failure across six-plus disparate tools can delay recovery significantly, since each tool keeps its own logs and none share a common run/lineage identifier.

The three-V challenge

Volume — an industry-wide growth trend, not a one-time spike

Total U.S. health insurance enrollment grew significantly through the COVID-19 pandemic years, reaching roughly **297 million covered individuals** by late 2025, before entering a period of policy-driven contraction heading into 2026 — largely due to the "Great Unwinding" of continuous Medicaid eligibility and the expiration of pandemic-era ACA premium tax credits. Even accounting for that contraction, the underlying structural trend for managed care organizations has been sustained, multi-decade membership growth punctuated by sharp step-changes (pandemic enrollment surges, Medicaid expansion, new ACA Marketplace lines of business). A fixed-capacity, single-machine legacy pipeline has no way to absorb that kind of growth or a sudden policy-driven enrollment swing without a hard redesign — only horizontally scalable, distributed processing can expand capacity on demand rather than hitting a hard ceiling.

Velocity

Enrollment changes require prompt processing — a member's coverage often needs to be active before a claim is submitted. Long batch windows delay downstream availability and operational response.

Variety

834 transactions contain varying member, employer, subscriber, group, and plan information — different relationship codes (self/spouse/child/domestic partner), different plan products (`GOLD`, `SILVER`, `BRONZE`, `PLATINUM`, `HDHP`, `EPO`, `PPO`, `HMO` in this prototype's validation rules — see `DataFrameValidator.VALID_PLANS` in `src/medenroll834/quality/validator.py`), and segments that may be mixed `~`-terminated or newline-terminated within the same file (`split_segments()` in `src/medenroll834/ingestion/bronze.py` normalizes both). The pipeline must validate these differences without losing source traceability back to the raw segment.

Business objective

The prototype shows how a modern, distributed data platform can address all three constraints simultaneously: it improves throughput (parallel Spark execution vs. serial single-machine jobs), observability (structured logs + run summaries + Delta table lineage instead of six separate tool logs), recoverability (idempotent re-runs, quarantine instead of hard failure), and analytical access (an OLAP star schema and Databricks SQL/Genie that the legacy CORE System-only flow does not provide). It is also a practical modernization blueprint for presenting an architecture decision to stakeholders.

03. Legacy Architecture

> **Conceptual overview**
>
> This is an illustrative view of a traditional X12 834 enrollment processing workflow — file ingestion, format conversion, business-rule application, batch orchestration, validation, and integration with a downstream core administration system. It represents a generalized, educational architecture pattern commonly seen in healthcare enrollment processing and is not intended to reproduce any specific organization's production environment, internal job names, or proprietary workflow details.

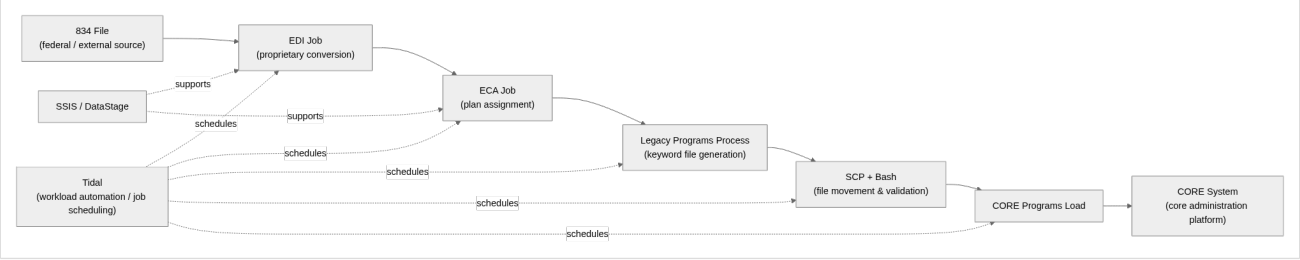
Existing platform

The legacy process is a scheduled, file-based enrollment workflow. It moves data across specialized tools before the **CORE System** receives the final keyword file — there is no single distributed processing engine or shared data lake underneath it.

Processing flow

1. A federal or external source places an 834 file in a monitored folder.
2. An **EDI** job converts the 834 transaction into a proprietary format.
3. An **ECA** job processes each member and assigns a benefit plan.
4. A **Legacy Programs** process generates the keyword file.
5. **SCP** and Linux jobs move files and run **Bash** validations.
6. **CORE Programs** loads the keyword file into the **CORE System** core platform.

Tidal schedules and coordinates the jobs. **SSIS** and **DataStage** support loading and transformation activities elsewhere in the estate.



Operational limitations

- **Sequential file handoffs.** Each stage must complete and hand a file to the next tool; there is no parallelism within a stage.
- **Fragmented logging.** Each tool maintains separate logs and operational controls — there is no common run ID connecting an EDI failure to its downstream ECA or CORE System load.
- **Database contention.** Single-machine, serial processing against a shared database can create lock-related delays or failures under load.
- **Limited parallelism.** All transformation work happens on one machine, so the observed ~1 hour processing-time constraint (see Chapter 2) cannot be improved without adding distributed compute.
- **Proprietary intermediate format.** The EDI → proprietary conversion complicates reuse: the data cannot easily be queried, re-processed, or fed into an analytics stack without a custom reader.
- **Operational-only data shape.** The data produced is optimized purely for the CORE System keyword-file load, not for broad analytics, historical reporting, or ad hoc BI/AI querying.

Modernization boundary

CORE System remains the downstream system of record in this prototype's conceptual framing. The modernization effort documented in this documentation focuses on **ingestion, transformation, validation, and data delivery** — replacing the EDI → ECA → Legacy Programs → SCP → CORE Programs Load chain with a distributed PySpark/Delta/Airflow pipeline (see Chapter 4) while keeping the downstream operational contract (an enrollment record reaching a core administration system) conceptually the same.

04. Modern Architecture

Idea: Modern Distributed Data Platform for Growing Data

This proof of concept demonstrates how distributed processing can prepare a healthcare data platform for increasing data **volume, velocity, and variety**. X12 834 enrollment processing is the representative business module; the architecture is designed to be reusable across other business domains.

> **Note**
> "Enrollment" names the representative POC module, not the platform itself. The platform capability being demonstrated is distributed processing plus operational/analytical separation, applicable to any high-volume healthcare (or other) data domain.

Three capabilities

The architecture delivers three distinct, composable capabilities, in the order data actually flows through them. Keeping them conceptually separate is the core of the modernization story.

Capability 1 — Distributed Processing

```
X12 834 → Landing (object storage) → Databricks / PySpark → Bronze → Silver → Gold
```

Purpose: Handle increasing Volume, Velocity, and Variety by parsing and validating EDI data as a distributed, replayable medallion pipeline rather than a monolithic nightly batch job.

Capability 2 — Transactional Platform

```
Gold → PostgreSQL OLTP database → Operational applications
```

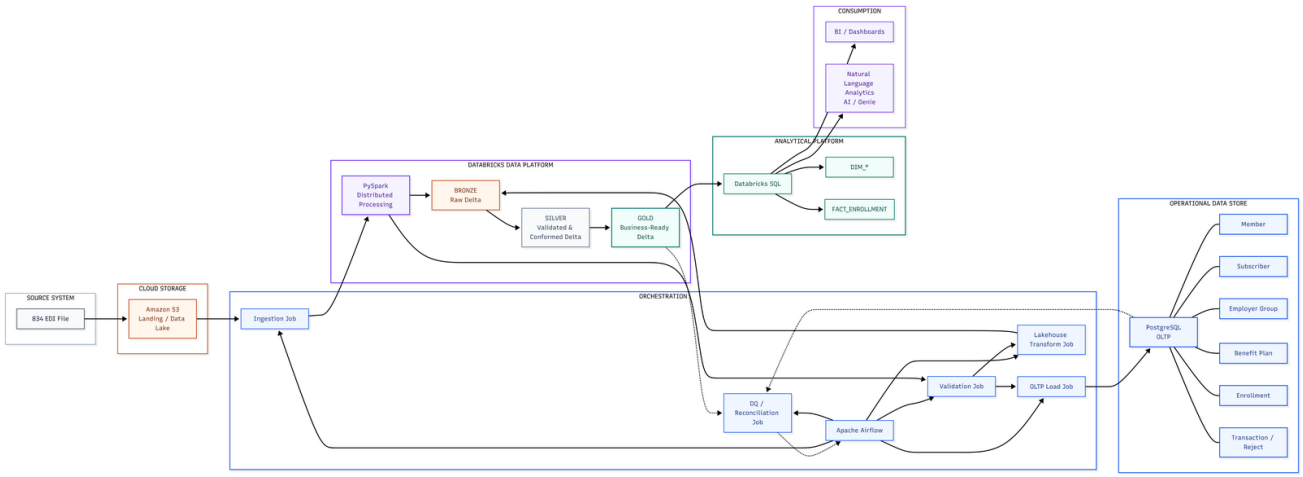
Purpose: Provide an operational relational store for application-facing reads/writes, decoupled from analytical query load.

Capability 3 — Analytical Platform

```
Gold → Star Schema (Delta Lake) → Databricks SQL → BI/HTML Reports + Genie AI
```

Purpose: Separate analytical workloads from operational systems and provide governed data for BI and AI use cases, independent of the transactional (OLTP) store.

Full architecture diagram



Architecture layers

1. Ingestion — Synthetic / real X12 834 files land in object storage, then get staged into a Databricks Unity Catalog Volume (`/Volumes/med834enrl/edi/landing/834`) so serverless Databricks compute can read them.

2. Distributed processing (Databricks / PySpark) — Bronze: raw X12 segments, one row per segment, schema fixed but content untouched. Silver: validated, normalized member records with a `validation_errors` array and `is_valid` flag; invalid rows go to DQ / Quarantine. Gold: analytics-ready deduplicated enrollment records with derived fields (`full_name` , `age_years`).

3. Two downstream consumers of Gold — OLTP load: `load_gold_to_oltp()` upserts Gold rows into the PostgreSQL CORE/ENRL schemas for operational applications. OLAP star schema: the same Gold data is modeled into `DIM_MEMBER` , `DIM_PLAN` , `DIM_GROUP` , `DIM_DATE` , `DIM_TRANSACTION_SOURCE` , and `FACT_ENROLLMENT` for BI/dashboarding and AI/Genie natural-language analytics.

4. Analytical consumption — two modes

Both the HTML report and Genie are consumers of the governed analytical layer, not steps inside the pipeline's transformation logic.

Mode	Flow	Use case
Traditional analytics	Predefined SQL → HTML report	Fixed KPIs, scheduled reporting
AI analytics	Natural language → SQL → result	Ad hoc exploration via Genie

5. Orchestration — Airflow

Airflow orchestrates the end-to-end workflow, including ingestion, Databricks processing, OLTP loading, reporting, and archival, with task dependencies, retries, and failure handling. Airflow sits **above** the pipeline as orchestration — it is not a data layer itself. The archive step runs with the default `all_success` trigger rule, wired directly to the core processing chain's own outputs, so files are only archived once Databricks and the OLTP load have actually succeeded; a separate `quarantine_failed_files` task (`trigger_rule="one_failed"`) moves this run's files to a `failed/` prefix instead if that chain fails. See Chapter 20.

6. POC vs. enterprise storage

For the proof of concept, S3-compatible blob storage and Databricks Unity Catalog Volumes provide the development storage path. In the target enterprise architecture, the landing/object-storage layer can be implemented using AWS S3 while retaining the same downstream Bronze-Silver-Gold, OLTP, OLAP, BI, and AI architecture — the interface is already implemented in `src/medenroll834/s3/s3_utils.py` alongside the POC's `src/medenroll834/storage/blob_storage.py` .

See also: End-to-End Execution and Docker / Local Development (includes the backout/rollback procedures).

05. Technology Decisions

Every technology choice below is pinned in `requirements.txt` / `pyproject.toml` and actually exercised by the pipeline — this is not an aspirational stack list.

Processing engine: PySpark 4.0+ / Delta Lake 4.0+

Why: Distributed DataFrame processing scales horizontally with data volume (the core "Volume" constraint from Chapter 2), and Delta Lake adds ACID transactions, schema evolution (`mergeSchema`), time travel, and Change Data Feed (`delta.enableChangeDataFeed`) on top of plain Parquet. The medallion layers (`src/medenroll834/ingestion/{bronze,silver,gold}.py`) write exclusively in Delta format.

Why Delta over plain Parquet: Parquet alone is already a columnar, compressed format — splitting data into column chunks means a query that only needs a few fields (e.g. `plan_id` , `coverage_start_date`) reads far less from disk than a row-oriented format would, and per-column min/max statistics let Spark skip whole files that can't match a filter. Delta Lake keeps that storage format but adds a transaction log on top of it: concurrent/retried writes (an Airflow retry re-running Bronze) can't corrupt or partially overwrite a table, `mergeSchema` lets a layer tolerate an added column without a manual migration, and `delta.enableChangeDataFeed` lets a future consumer read only the rows that changed since a given version instead of re-scanning the whole table (see Chapter 37 for where this is headed next).

Orchestration: Apache Airflow 3.x (TaskFlow API, CeleryExecutor)

Why: The core requirement here is production-grade orchestration — a task-dependency graph, automatic retries with backoff, and visibility into every run — delivered by a widely-adopted, actively maintained open-source project with a large operator/provider ecosystem (official Databricks and AWS provider packages, in this case) rather than a proprietary scheduler. The `@dag` / `@task` TaskFlow decorators (`05_orchestration/airflow/01_dags/med834_e2e_pipeline.py`) keep each stage as a typed Python function with explicit inputs/outputs, and the same DAG definition runs identically in this project's own Docker Compose stack or in a managed service (MWAA, Cloud Composer, Astronomer) with no code change — a proprietary enterprise scheduler would require a dedicated license and operator training, while Airflow's skill set and DAG code are directly portable.

Compute platform: Databricks (Free/Serverless tier compatible)

Why: Managed Spark clusters, Unity Catalog governance, and Databricks SQL warehouses in one workspace, with a SQL-callable `WorkspaceClient` SDK (`databricks.sdk`) used directly from both the Airflow DAG and the reporting/OLTP-loader modules — no separate JDBC driver management.

Operational store: PostgreSQL (OLTP)

Why: A relational, foreign-key-enforced schema (`CORE` / `ENRL` / `UTIL` — see Chapter 9) is the natural fit for application-facing enrollment reads/writes, and is intentionally **kept separate** from the analytical Delta star schema so analytical query load never contends with operational traffic. `psycopg2` is used directly (`src/medenroll834/oltp/oltp_loader.py`) rather than an ORM, keeping the upsert SQL explicit and auditable.

Object storage: S3-compatible blob storage (POC) / AWS S3 (enterprise target)

Why: Databricks' free/serverless tier does not support a live connection to an external AWS S3 bucket, so AWS S3 was not a feasible landing layer for this proof of concept. A third-party S3-compatible blob provider was used instead for the POC's landing/archive/report storage (`src/medenroll834/storage/blob_storage.py`). For an enterprise deployment with a paid Databricks tier, AWS S3 is the recommended target — the interface is already implemented at `src/medenroll834/s3/s3_utils.py` with the equivalent `upload_834` / `list_landing_files` / `archive_834` functions, so swapping storage providers requires no change to Bronze/Silver/Gold, OLTP, OLAP, BI, or AI code.

Containerization: Docker / Docker Compose

Why: The full Airflow stack (webserver, scheduler, worker, Postgres metadata DB, Redis broker) is reproducible with one `docker compose up -d` (`docker-compose.yaml`), with the project bind-mounted at `/opt/airflow/project` so `src/` and `generator/` import directly into DAG tasks without a separate package build step.

CI/CD: GitHub Actions

Why: Free for public repos, with first-class native GitHub integration (no separate CI account/billing to provision) and a large marketplace of pre-built actions for exactly the steps this project needs (Python setup, Docker buildx/push, Codecov upload). The existing `.github/workflows/ci.yml` / `cd.yml` cover lint, test, security scan, build, and Docker image publish — see Chapter 31 for the full pipeline and a known path-configuration bug that currently needs fixing in `ci.yml`.

Summary table

Concern	Technology	Why (grounded in this repo)
Distributed processing	PySpark 4.0+, Delta Lake 4.0+	Scales with volume; ACID + schema evolution on Bronze/Silver/Gold
Orchestration	Apache Airflow 3.x (TaskFlow, Celery)	Retries, task graph, Variable overrides, UI, portable open-source skill set
Compute platform	Databricks (Serverless-compatible)	Managed Spark + Unity Catalog + SQL warehouses + `databricks-sdk`
Operational store	PostgreSQL	FK-enforced OLTP schema, kept apart from OLAP
OLTP driver	`psycopg2` (raw SQL)	Explicit, auditable upsert statements
Object storage (POC)	S3-compatible blob storage	Works on Databricks' free tier, where native AWS S3 access isn't available
Object storage (enterprise)	AWS S3	Same interface (`s3_utils.py`), swap-in only, once on a paid Databricks tier
Containerization	Docker / Docker Compose	One-command reproducible Airflow stack
CI/CD	GitHub Actions	Free, native GitHub integration, large action marketplace
IaC	Terraform	S3 bucket + policy as code (`04_infrastructure/01_terraform`)

06. Project Repository Structure

The repository follows a numbered, enterprise-style top-level layout so directory order communicates pipeline/delivery order at a glance.

```
MedEnroll834/
├── 02_tests/                  # pytest + PySpark test suite (Ch. 30)
├── 03_config/                 # pipeline_config.yaml (Delta paths, validation rules)
├── 04_infrastructure/         # Terraform (S3) + SQL DDL (OLTP + OLAP)
├── 05_orchestration/          # Airflow DAGs + Databricks notebook/job
├── 06_deploy/                 # Deployment configs
├── 07_scripts/                # Deployment, backout, and utility scripts
├── 08_docs/                   # Architecture docs, diagrams, runbooks (and this documentation's source)
├── 10_data/                   # Local Delta table storage (bronze/silver/gold/quarantine)
├── 11_build/                  # Build artifacts
├── 12_tools/                  # Vendored dev tools (local Delta dependencies)

├── docs-site/                 # This GitBook-style documentation site (MkDocs)
├── generator/                 # synthetic_834_generator.py
├── src/
│   └── medenroll834/           # the importable Python package (see below)
├── docker-compose.yaml        # Airflow 3.x stack (Celery executor)
├── Dockerfile
├── Makefile                   # install / test / lint / format / docker / setup-db targets
├── pyproject.toml             # package metadata, console scripts, tool configs
└── requirements.txt
```

> **Why `src/` isn't numbered**
>
> Python does not allow import paths that start with a digit, so the importable package could not live under a `01_src/` folder. Every *other* top-level directory keeps the numbered convention; `src/` is the one documented exception.

The `src/medenroll834/` package

```
src/medenroll834/
├── common/
│   ├── config.py           # load_config(), resolve_delta_path(), get_landing_path()
│   ├── logging.py          # setup_logging(), log_pipeline_start/end()
│   └── spark_session.py    # get_local_spark() — Delta session bootstrap
├── parsers/
│   └── edi834_parser.py    # EDI834Parser — X12 834 → Bronze DataFrame
├── ingestion/
│   ├── bronze.py           # BronzeLayer — split_segments(), raw Delta write
│   ├── silver.py           # SilverLayer — HL hierarchy resolution, validation
│   ├── gold.py             # GoldLayer — dedup, full_name, age_years
│   └── pipeline.py         # DeltaMedallionPipeline — Bronze→Silver→Gold orchestrator
├── quality/
│   ├── validator.py        # DataFrameValidator — Spark-SQL validation rules
│   └── quarantine.py       # DeltaQuarantineManager — invalid-record Delta sink
├── oltp/
│   └── oltp_loader.py      # load_gold_to_oltp() — Databricks SQL → psycopg2 upsert
├── reporting/
│   └── report_generator.py # generate_report(), ask_genie()
├── storage/
│   └── blob_storage.py     # S3-compatible blob storage client (current landing layer)
└── s3/
    └── s3_utils.py          # AWS S3 client, same interface as blob_storage.py
```

Key configuration files

File	Purpose
`03_config/pipeline_config.yaml`	Delta table paths/options per layer, Unity Catalog settings, validation plan codes
`pyproject.toml`	Package metadata, `medenroll834-generate` console script, ruff/black/mypy/pytest config
`docker-compose.yaml`	Full local Airflow 3.x stack definition
`.env.example`	Template for all pipeline secrets (copy to `.env`, git-ignored)
`Makefile`	`install`, `test`, `lint`, `format`, `docker-up/down`, `setup-db`, `setup-databricks`

07. AWS / S3 Setup

This chapter covers the **enterprise-target** AWS S3 setup (the current POC runs on S3-compatible blob storage instead — see Chapter 4, section 6). The Terraform and IAM policy below are real, committed infrastructure code already in the repo (`04_infrastructure/01_terraform/`), even though the runtime pipeline currently points at blob storage.

Terraform-managed S3 bucket

> Source: `04_infrastructure/01_terraform/main.tf`

```
resource "aws_s3_bucket" "med834" {
  bucket = var.bucket_name
  tags = {
    Project            = "MED834ENRL"
    Environment        = "dev"
    DataClassification = "synthetic"
    ManagedBy          = "Terraform"
  }
}

resource "aws_s3_bucket_versioning" "med834" {
  bucket = aws_s3_bucket.med834.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "med834" {
  bucket = aws_s3_bucket.med834.id
  rule {
    apply_server_side_encryption_by_default { sse_algorithm = "AES256" }
  }
}

resource "aws_s3_bucket_public_access_block" "med834" {
  bucket                  = aws_s3_bucket.med834.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

A bucket policy denies any non-TLS request, and ownership controls disable ACL-based access entirely — standard hardening that applies automatically once a deployment is on a

standard AWS account; nothing further to configure beyond attaching the Terraform-managed policy.

Folder layout provisioned as code

Terraform pre-creates every prefix the pipeline expects, as zero-byte placeholder objects (`aws_s3_object.folders`, `for_each` over a list):

```
landing/834/
archive/834/
lakehouse/bronze/{edi_834_raw,edi_834_segments,ingestion_metadata}/
lakehouse/silver/{member,subscriber,employer_group,benefit_plan,enrollment,address,transaction,rejected_record}/
lakehouse/gold/{dim_member,dim_plan,dim_group,dim_date,dim_transaction_source,fact_enrollment}/
quarantine/834/{invalid_format,validation_failed,processing_failed}/
audit/{ingestion,processing,reconciliation,errors}/
```

Least-privilege IAM policy

> Source: `04_infrastructure/01_terraform/s3_pipeline_least_privilege_policy.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "ListTheBucket", "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::med834enrl-s3" },
    { "Sid": "ReadWriteObjects", "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::med834enrl-s3/*" }
  ]
}
```

Only `ListBucket` on the bucket ARN itself, and `Get` / `Put` / `Delete` scoped to objects under it — no `s3:*`, no cross-bucket access.

Applying the Terraform

```
cd 04_infrastructure/01_terraform
terraform init
terraform plan -var="aws_region=us-east-1" -var="bucket_name=med834enrl-s3"
terraform apply
```

The S3 client interface (enterprise target)

> Source: `src/medenroll834/s3/s3_utils.py`

Credentials resolve through boto3's default chain (env vars, shared config, or IAM role) — in Airflow these come from the AWS connection exported as environment variables (`AWS_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` in `docker-compose.yaml`).

```
def upload_834(local_path: str, bucket: str, s3_prefix: str = "landing/834/") -> str:
    """Upload one .834 file under the given prefix, preserving its filename."""
    name = Path(local_path).name
    key = f"{s3_prefix.rstrip('/')}/{name}"
    return upload_file(local_path, bucket, key)

def archive_834(bucket: str, key: str, archive_prefix: str = "archive/834/") -> str:
    """Move a processed object from landing to archive (copy + delete)."""
    ...
```

This is the exact function signature shape mirrored by the POC's `src/medenroll834/storage/blob_storage.py` (`upload_834`, `archive_834`, `list_landing_files`, `put_text`), which is why migrating the Airflow DAG's `upload_to_blob` / `archive_blob` tasks to S3 is a drop-in swap — see the "Enterprise migration note" in Chapter 33.

AWS Cloud Shell backout

A standalone backout script exists for environments where S3 was provisioned outside Terraform (e.g. via AWS CloudShell):

```
bash 04_infrastructure/03_aws_cloud_shell/aws_s3_backout.sh
```

Removes all folder placeholders and objects from the `med834enrl-s3` bucket.

08. PostgreSQL Setup

MedEnroll834 uses PostgreSQL exclusively as the **OLTP operational store** (see Chapter 4 for why it is kept separate from the Delta/OLAP analytical layer). In this POC the database is a managed Neon Postgres instance, but any standard PostgreSQL 13+ works.

Connection configuration

The connection string lives in a single environment variable, `OLTP_DATABASE_URL`, consumed by the loader and passed through `docker-compose.yaml` to the Airflow scheduler/worker:

```
# .env
OLTP_DATABASE_URL=postgresql://<user>:<password>@<host>/<db>?sslmode=require
```

`sslmode=require` is mandatory for managed providers like Neon — see the troubleshooting note in the E2E runbook.

Deploying the schema

> Source: `07_scripts/02_deployment/deploy_oltp_database.py`

The deploy script runs every SQL file under `04_infrastructure/02_sql/02_oltp/` in dependency order (schemas → parent tables → child tables → indexes → constraints → comments):

```
DEPLOY_ORDER = [
    "01_schemas/01_create_core_schema.sql",
    "01_schemas/02_create_enrl_schema.sql",
    "01_schemas/03_create_util_schema.sql",
    "02_tables/CORE_GRGR_GROUP.sql",
    "02_tables/CORE_PLPL_PLAN.sql",
    "02_tables/CORE_MESB_SUBSCRIBER.sql",
    "02_tables/CORE_MEME_MEMBER.sql",
    "02_tables/CORE_SBAD_ADDRESS.sql",
    "02_tables/ENRL_EDXN_TRANSACTION.sql",
    "02_tables/CORE_MEEN_ENROLLMENT.sql",
    "02_tables/ENRL_RJRW_REJECT.sql",
    "03_indexes/INDEXES.sql",
    "04_constraints/CONSTRAINTS.sql",
    "05_views/COMMENTS.sql",
]
```

Run it with:

```
python 07_scripts/02_deployment/deploy_oltp_database.py "<OLTP_DATABASE_URL>"
```

The whole deployment runs in a **single transaction** — any statement failure rolls back everything, so the database never ends up in a half-migrated state.

Equivalently, via the Makefile (runs against a local `psql` target):

```
make setup-db
```

Rolling back

```
python 07_scripts/02_deployment/backout_oltp_database.py "<OLTP_DATABASE_URL>"
```

Runs `04_infrastructure/02_sql/02_oltp/SCHEMA_OLTP_BACKOUT.sql`, which drops all foreign keys, the 8 CORE/ENRL tables, and the CORE/ENRL/UTIL schemas with `IF EXISTS` / `CASCADE`, so re-running a backout that already ran is a safe no-op.

Verifying a deployment

> Source: `07_scripts/03_utils/verify_oltp_deploy.py`

```
URL = (
    "postgresql://neondb_owner:***@"
    "ep-late-leaf-b3moj6iv-pooler.c-4.ap-southeast-1.aws.neon.tech/"
    "med834enrl?sslmode=require"
)
```

The verify script connects and confirms the expected tables/row counts exist after a deploy or an end-to-end pipeline run.

Schemas provisioned

Schema	Purpose
`CORE`	Group, plan, subscriber, member, address, enrollment — the operational entity model
`ENRL`	Transaction control and rejected-record tracking
`UTIL`	Shared utility objects (sequences, lookups)

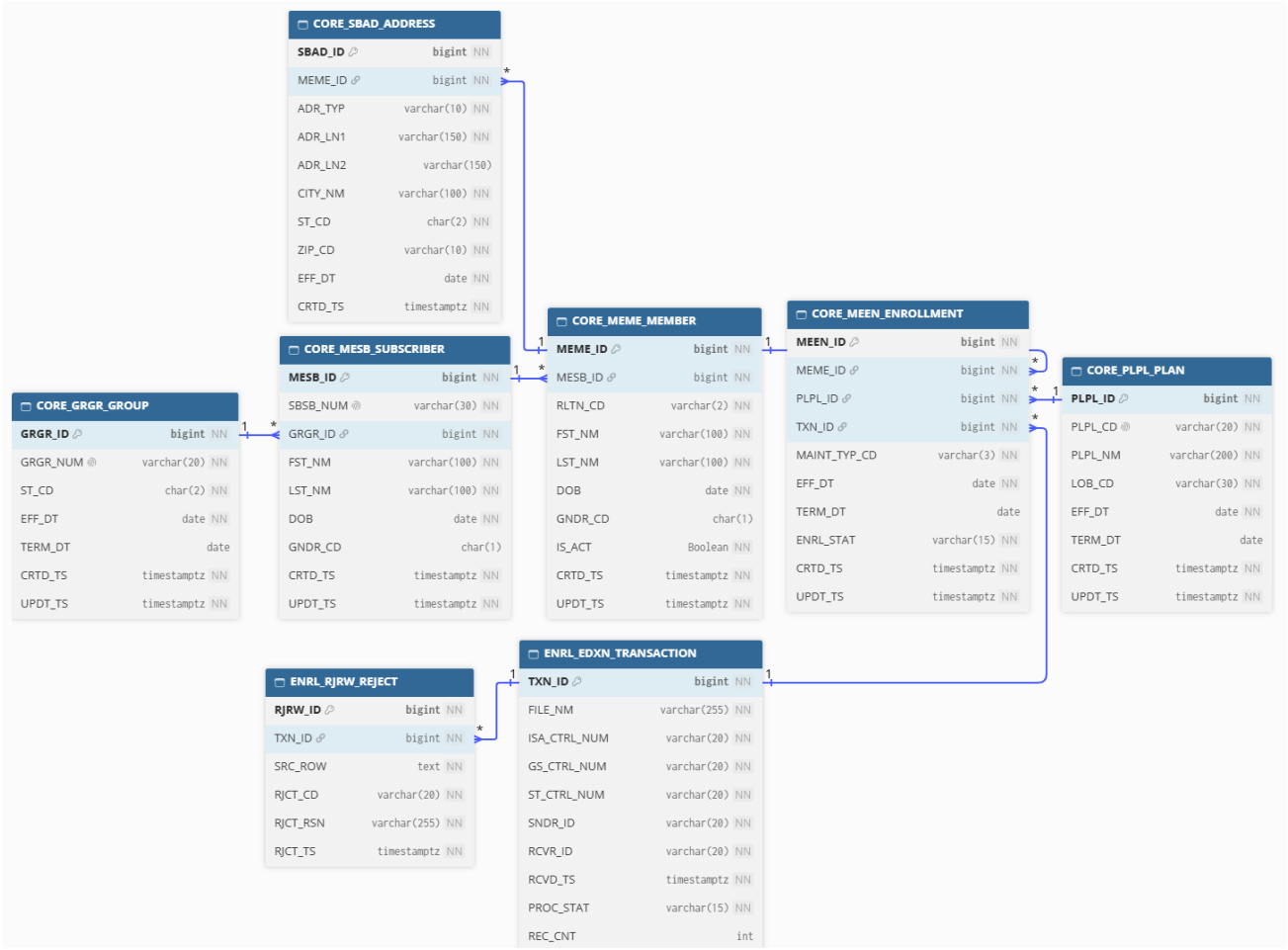
See Chapter 9 for the full table-by-table DDL.

09. Database Schema

MedEnroll834 has **two physically separate schemas**: an OLTP relational model in PostgreSQL and an OLAP star schema in Delta Lake. This chapter documents both with their actual DDL.

OLTP schema (PostgreSQL)

> Source: `04_infrastructure/02_sql/02_oltp/`



CORE schema

Table	Key columns	Purpose
`CORE_GRGR_GROUP`	`GRGR_ID` PK, `GRGR_NUM` unique	Employer group
`CORE_PLPL_PLAN`	`PLPL_ID` PK, `PLPL_CD` unique	Benefit plan
`CORE_MESB_SUBSCRIBER`	`MESB_ID` PK, `SBSB_NUM` unique, FK → `GRGR_ID`	Policy subscriber
`CORE_MEME_MEMBER`	`MEME_ID` PK, FK → `MESB_ID`	Member (subscriber or dependent), `RLTN_CD` relationship code
`CORE_SBAD_ADDRESS`	`SBAD_ID` PK, FK → `MEME_ID`	Member address
`CORE_MEEN_ENROLLMENT`	`MEEN_ID` PK, FK → `MEME_ID`, `PLPL_ID`, `TXN_ID`	Enrollment event

ENRL schema

Table	Key columns	Purpose
`ENRL_EDXN_TRANSACTION`	`TXN_ID` PK	One row per ingested 834 file (ISA/GS/ST control numbers, `PROC_STAT`)
`ENRL_RJRW_REJECT`	`RJRW_ID` PK, FK → `TXN_ID`	Rejected source rows with reason code

Example DDL — `CORE_MEME_MEMBER`

```
CREATE TABLE IF NOT EXISTS CORE.CORE_MEME_MEMBER (
  MEME_ID BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  MESB_ID BIGINT NOT NULL,
  RLTN_CD VARCHAR(2) NOT NULL,
  FST_NM VARCHAR(100) NOT NULL,
  LST_NM VARCHAR(100) NOT NULL,
  DOB DATE NOT NULL,
  GNDR_CD CHAR(1),
  IS_ACT BOOLEAN NOT NULL,
  CRTD_TS TIMESTAMPTZ NOT NULL,
  UPDT_TS TIMESTAMPTZ NOT NULL
);
```

Foreign keys

All seven FKs are `DEFERRABLE INITIALLY IMMEDIATE`, declared in `04_constraints/CONSTRAINTS.sql`:

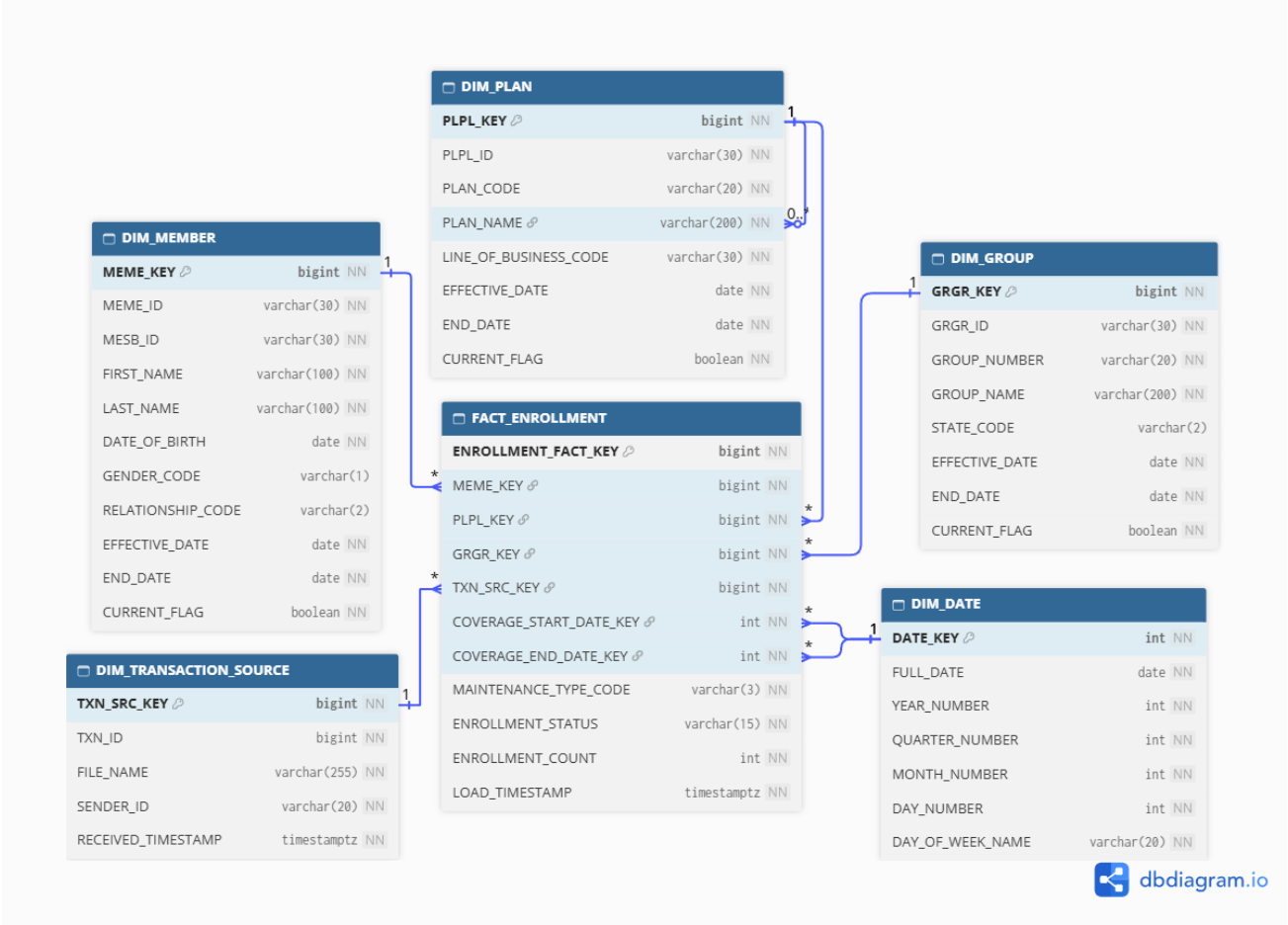
```
ALTER TABLE CORE.CORE_MESB_SUBSCRIBER ADD CONSTRAINT FK_CORE_MESB_SUBSCRIBER_GRGR_ID
  FOREIGN KEY (GRGR_ID) REFERENCES CORE.CORE_GRGR_GROUP (GRGR_ID) DEFERRABLE INITIALLY
 IMMEDIATE;
ALTER TABLE CORE.CORE_MEME_MEMBER ADD CONSTRAINT FK_CORE_MEME_MEMBER_MESB_ID
  FOREIGN KEY (MESB_ID) REFERENCES CORE.CORE_MESB_SUBSCRIBER (MESB_ID) DEFERRABLE INIT
IALLY IMMEDIATE;
-- ... and 5 more (address→member, enrollment→member/plan/transaction, reject→transact
ion)
```

Indexes

16 indexes support the common operational access patterns — foreign-key columns, status/date filters, and one composite index (`IX_CORE_MEEN_ENROLLMENT_MEME_PLPL` on `(MEME_ID, PLPL_ID)`) for the member-plan lookup path. See `03_indexes/INDEXES.sql`.

OLAP star schema (Delta Lake)

> Source: `04_infrastructure/02_sql/03_olap/04_gold/`



`FACT_ENROLLMENT` sits at the center, surrounded by five dimensions — all Delta tables in the `GOLD` schema:

```
CREATE TABLE IF NOT EXISTS GOLD.FACT_ENROLLMENT (
  ENROLLMENT_FACT_KEY BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  MEME_KEY BIGINT NOT NULL,
  PLPL_KEY BIGINT NOT NULL,
  GRGR_KEY BIGINT NOT NULL,
  TXN_SRC_KEY BIGINT NOT NULL,
  COVERAGE_START_DATE_KEY INT NOT NULL,
  COVERAGE_END_DATE_KEY INT NOT NULL DEFAULT 99991231,
  MAINTENANCE_TYPE_CODE STRING NOT NULL,
  ENROLLMENT_STATUS STRING NOT NULL,
  ENROLLMENT_COUNT INT NOT NULL,
  LOAD_TIMESTAMP TIMESTAMP NOT NULL
)
USING DELTA
COMMENT 'Gold enrollment fact. Grain: one row per enrollment event.';
```

`COVERAGE_END_DATE_KEY` (and `TERM_DT` / `END_DATE` throughout OLTP and OLAP) is never `NULL` — open-ended coverage stores the fixed `9999-12-31` / `99991231` sentinel instead, joining against a dedicated `DIM_DATE` "Open Ended" row (see Chapter 19).

Dimensions (all SCD Type 2 via `CURRENT_FLAG`, except `DIM_DATE`)

Dimension	Surrogate key	Natural key	Notes
`DIM_MEMBER`	`MEME_KEY`	`MEME_ID`	First/last name, DOB, relationship code
`DIM_PLAN`	`PLPL_KEY`	`PLPL_ID`	`PLAN_CODE`, `PLAN_NAME`, line of business
`DIM_GROUP`	`GRGR_KEY`	`GRGR_ID`	Group number, name, state
`DIM_DATE`	`DATE_KEY` (PK = `YYYYMMDD` int)	—	Year/quarter/month/day/day-name, no SCD needed
`DIM_TRANSACTION_SOURCE`	`TXN_SRC_KEY`	`TXN_ID`	File name, sender id, received timestamp

See Chapter 19 for how the Databricks notebook actually resolves these surrogate keys when building `FACT_ENROLLMENT`.

Schema deployment order

Store	Script	Order
OLTP	`07_scripts/02_deployment/deploy_oltp_database.py`	Schemas → tables → indexes → constraints → comments
OLAP	`07_scripts/02_deployment/deploy_databricks_olap.py`	Schemas → dimension tables → fact table → constraints

Both are idempotent (`CREATE TABLE IF NOT EXISTS`, `DROP ... IF EXISTS` before recreate) and have matching backout scripts documented in the Backout Runbook.

10. Synthetic 834 Data Generation

No production member data is used anywhere in this project. All 834 files are generated by `generator/synthetic_834_generator.py`, which produces structurally valid X12 834 (5010) EDI files from randomized but realistic name/address/plan pools.

Reference data pools

The generator draws from hard-coded, realistic-looking pools so the output resembles production data in shape without being production data:

```
FIRST_NAMES = ["JOHN", "JANE", "ROBERT", "MARY", ...]       # 38 names
LAST_NAMES  = ["SMITH", "JOHNSON", "WILLIAMS", ...]          # 38 names
STREET_NAMES = ["MAIN ST", "OAK AVE", "PINE RD", ...]
CITIES = ["NEW YORK", "LOS ANGELES", "CHICAGO", ...]         # 20 cities
STATES = ["AL", "AK", "AZ", ..., "WY"]                       # all 50
PLAN_CODES = ["100", "101", "102", "103", "104", "105", "106", "107"]  # plain numeric
 plan codes
RELATIONSHIP = ["18", "01", "03", "19"]  # self, spouse, child, domestic_partner
ADR_TYP = ["HOME", "MAILING", "BILLING", "WORK"]
GROUP_NUMBERS = [str(3300 + i) for i in range(50)]  # pure-numeric employer group pool
```

`PLAN_TIER_NAME` maps each numeric `PLAN_CODE` to a display tier (`"100" -> "GOLD"`, `"101" -> "SILVER"`, ...) for `PLAN_NAME` synthesis in Silver/Gold/OLAP, since the plan code itself is now opaque. `PLAN_CODES` is the exact set `DataFrameValidator.VALID_PLANS` validates against in Silver — generated files are guaranteed valid by construction unless `--messy` is passed (see below).

X12 envelope construction

Each file is a properly-terminated X12 interchange: `ISA` → `GS` → `ST` → (member segments) → `SE` → `GE` → `IEA`, with a computed ISA checksum:

```
def _checksum_isa(isa_segment: str) -> str:
    total = sum(ord(c) for c in isa_segment)
    return f"{total % 256:02d}"

def build_isa(ctrl: str, file_date: str) -> str:
    body = (
        f"*00*          *00*          "
        f"*ZZ*SENDERID     *ZZ*RECEIVERID   "
        f"*{file_date}*1200*^*00501*{ctrl}*0*T*:"
    )
    checksum = _checksum_isa(body)
```

```
    return f"ISA{body}{checksum}"
```

Segments are `~`-terminated, with a trailing newline for readability — the `~` is the real X12 record delimiter; newlines are cosmetic (and the Bronze parser's `split_segments()` normalizes both, see Chapter 13):

```
return "\n".join(s + "~" for s in [isa, gs] + segments + [se, ge, iea]) + "\n"
```

Member, subscriber, and group identifiers

All business identifiers are now plain-digit numeric strings instead of prefixed codes (e.g. `"2043434"` for a member, `"3341"` for an employer group), matching the `dbdiagram` physical models under `docs-site/docs/assets/architecture/physical-oltp` and `physical-olap`:

```
def _numeric_id(used: set, low: int = 1000000, high: int = 9999999) -> str:
    """Unique 7-digit member/subscriber identifier, e.g. "2043434"."""
    while True:
        candidate = random.randint(low, high)
        if candidate not in used:
            used.add(candidate)
            return str(candidate)
```

Each subscriber is assigned a `GRGR_NUM` drawn from the fixed `GROUP_NUMBERS` pool (`"3300"` – `"3349"`) and carried through the file via the X12-standard `REF*1L*<group number>` (Group or Policy Number) segment — the same segment Silver's `_extract_ref_group_data()` reads back out, so `group_id` is populated from real file content end-to-end instead of a hardcoded default:

```
def build_ref_group(grgr_num: str) -> str:
    return f"REF*1L*{grgr_num}"
```

Generating a batch

```
def generate_batch(
    output_dir: str,
    num_files: int = 10,
    subscribers_per_file: int = 100,
    dependents_per_sub: int = 2,
    start_date: Optional[date] = None,
    messy: bool = False,
) -> list:
    ...
```

CLI usage:

```
python -m generator.synthetic_834_generator \
  --output 10_data/raw/ \
  --files 10 \
  --subs 100 \
  --deps 2
```

Or via the installed console script (`pyproject.toml` → `[project.scripts]`):

```
medenroll834-generate --output 10_data/raw/ --files 3 --subs 50 --deps 2
```

Generated filenames follow `enrollment_<YYYYMMDD>_<NNN>.834`, one file per day going backward from `start_date` (defaults to today).

Direct-to-Databricks generation

For generating straight into a Unity Catalog Volume from a Databricks notebook/job context:

```
def generate_and_load_to_dbfs(
    spark: "SparkSession",
    output_dir: str,
    num_files: int = 10,
    subscribers_per_file: int = 100,
    dependents_per_sub: int = 2,
    volume_path: str = "/Volumes/med834enrl/edi/landing/834",
) -> None:
    ...
```

Invoked with `--dbfs --volume-path /Volumes/...` on the CLI when PySpark is importable in the environment.

`messy` mode

Passing `--messy` intentionally introduces malformed records (missing required fields, out-of-range dates, invalid plan codes) so the Silver layer's validation rules and the quarantine path have something real to catch during testing — without this flag, synthetic data is constructed to always pass validation.

Used by both tests and the Airflow DAG

The same `generate_batch()` function backs:

- `02_tests/test_pipeline.py`'s `sample_834_files` fixture (2 files, 5 subscribers each)
- the `generate_834` task in `med834_e2e_pipeline.py` (1 file, 50 subscribers, 2 dependents per run)

so the exact same generator code is exercised by unit tests and by the production-shaped DAG.

11. PySpark Development Environment

> Source: `src/medenroll834/common/spark_session.py`

Production processing for this project runs on **Databricks** (see Chapter 25 for the cluster-side tuning), not on a local machine. `get_local_spark()` exists solely to give the test suite (and fast local iteration on parser/Silver/Gold logic) a real, Delta-enabled Spark session without needing a live Databricks workspace for every test run.

```
def get_local_spark(app_name: str = "med834enrl") -> SparkSession:
    builder = (
        SparkSession.builder.appName(app_name)
        .master("local[*]")
        .config("spark.sql.adaptive.enabled", "true")
        .config("spark.sql.adaptive.coalescePartitions.enabled", "true")
        .config("spark.sql.shuffle.partitions", "8")
        .config("spark.sql.session.timeZone", "UTC")
        # Static configs -- must be set before the session is created.
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",
                "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    )
    ...
```

Delta package resolution

Delta Maven coordinates are **derived**, not hardcoded, to avoid a mismatch after a Spark version upgrade:

```
def _delta_packages() -> str:
    import delta
    delta_version = getattr(delta, "__version__", "")
    from pyspark import __version__ as pyspark_version
    spark_series = ".".join(pyspark_version.split(".")[:2])
    scala = os.environ.get("DELTA_SCALA_VERSION", "2.13")
    return f"io.delta:delta-spark_{spark_series}_{scala}:{delta_version}"
```

This only applies when `delta-spark` is pip-installed locally (`_delta_packages_available()` checks `import delta` first) — on Databricks, Delta is already provided by the runtime and this branch is skipped.

Shared by tests and scripts

`get_local_spark()` is the **single source of truth** — reused by `02_tests/conftest.py`'s `spark` fixture, `07_scripts/03_utils/run_e2e_smoke.py`, and any future entry point:

```
# 02_tests/conftest.py
@pytest.fixture(scope="session")
def spark():
    return get_local_spark("med834enrl-test")
```

Setting up locally

```
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
pytest 02_tests/ -v         # exercises get_local_spark() via conftest.py
```

See Chapter 30 for the full test suite and Chapter 25 for the production (Databricks) side of Spark tuning.

12. 834 Parser

Source: `src/medenroll834/parsers/edi834_parser.py`

`EDI834Parser` is the PySpark-native entry point that turns raw 834 file content into a Bronze-shaped DataFrame. It is a thin wrapper around the same segment-splitting logic used by `BronzeLayer` (see Chapter 13) — the parser exists as a standalone, directly-testable class so tests can parse files without going through the full `BronzeLayer.ingest()` Delta-write path.

Segment splitting

```
def split_segments(content: str) -> list:
    """Split an X12 834 payload into individual segments.

    Handles both segment conventions found in real 834 files:
    * terminator-delimited ("ISA*...~GS*...~") where "~" is the only
      record delimiter and physical newlines are cosmetic;
    * line-delimited ("ISA*...\\nGS*...\\n") which some upstream systems
      emit, where the newline is the de-facto terminator and "~" may be
      absent.

    The two are frequently mixed within a single file, so neither
    delimiter can be assumed on its own. Normalise every newline to "~"
    then split on "~" to get exactly one record per segment whatever the
    producer used.
    """
    normalised = content.replace("\r\n", "~").replace("\r", "~").replace("\n", "~")
    return [seg.strip() for seg in normalised.split("~") if seg.strip()]
```

This is a deliberate real-world robustness decision: production 834 feeds are not guaranteed to be consistently delimited, and this normalization means the parser never silently drops or merges segments because of a delimiter mismatch.

Parsing a directory of files

```
def parse_files(self, landing_path: str) -> DataFrame:
    path = Path(landing_path)
    files = sorted(path.glob("*.834"))
    rows = []
    for file in files:
        content = file.read_text(encoding="utf-8")
        file_name = file.name
        for seg_order, line in enumerate(split_segments(content)):
            elements = line.split("*")
            rows.append({
                "file_name": file_name,
                "segment_id": elements[0],
                "elements": elements,
                "raw_segment": line,
                "seg_order": seg_order,
            })
    return self.spark.createDataFrame(rows, schema=BRONZE_SCHEMA)
```

Each X12 segment is split on `*` into its data elements; `elements[0]` is always the segment identifier (`ISA`, `GS`, `ST`, `HL`, `NM1`, `INS`, `DTP`, `HD`, `N3`, `N4`, `REF`, `SE`, `GE`, `IEA`). `seg_order` preserves the segment's original position within the file — critical because downstream Silver logic resolves hierarchy and "most recent value" using window functions ordered by this column.

Schema

```
BRONZE_SCHEMA = StructType([
    StructField("file_name", StringType(), True),
    StructField("segment_id", StringType(), True),
    StructField("elements", ArrayType(StringType()), True),
    StructField("raw_segment", StringType(), True),
    StructField("seg_order", LongType(), True),
])
```

`elements` is kept as an **array of strings**, not individually typed columns — this is what lets a single Bronze schema represent every segment type without a different column set per segment, at the cost of pushing positional extraction (`elements[8]` for a member ID, etc.) into Silver.

The relationship code map

```
RELATIONSHIP_MAP = {
    "18": "self",
    "01": "spouse",
    "03": "child",
    "19": "domestic_partner",
}
```

Built into a Spark `create_map()` expression once at construction time (`self.rel_map`) so relationship-code lookups in downstream transforms are a vectorized Spark SQL expression rather than a Python UDF.

Also supports in-memory DataFrames

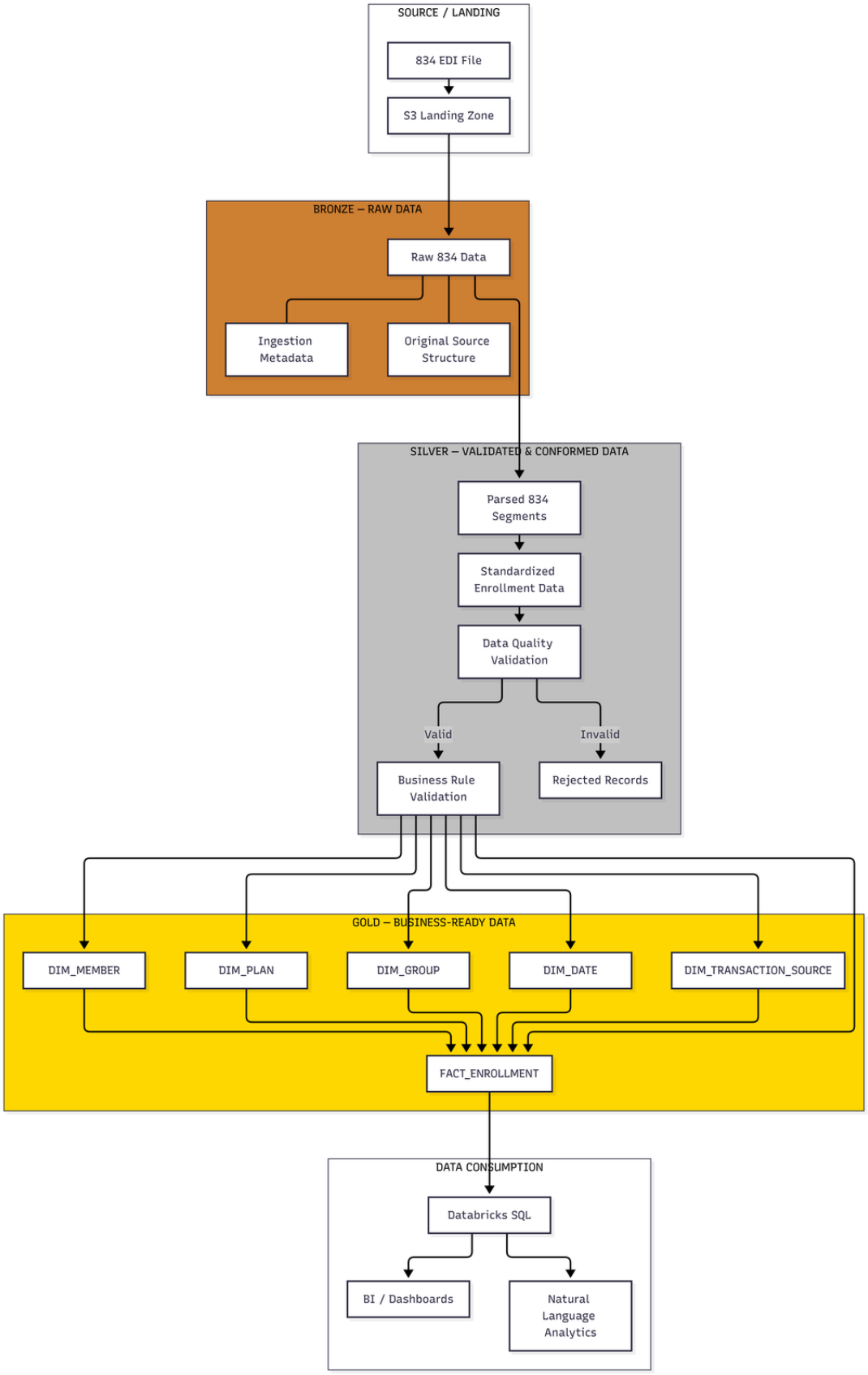
```
def parse_dataframe(self, df: DataFrame, file_name_col: str = "raw") -> DataFrame:
    rows = []
    for row in df.collect():
        content = row[file_name_col]
        ...
```

Used when 834 content is already loaded into a Spark DataFrame (e.g. read via `spark.read.text()` from a Unity Catalog Volume) rather than present as local files — this is the shape the production Databricks notebook job uses (see Chapter 21).

13. Bronze Layer

Source: `src/medenroll834/ingestion/bronze.py`

The Bronze layer is the **raw, untransformed** landing zone: one row per X12 segment, schema fixed, content untouched beyond splitting into elements. Its job is to preserve the source data exactly as received, so every downstream transformation can be replayed or audited against it.


SOURCE / LANDING
834 EDI File
S3 Landing Zone
BRONZE — RAW DATA
Raw 834 Data
Ingestion Metadata
Original Source Structure
SILVER — VALIDATED & CONFORMED DATA
Parsed 834 Segments
Standardized Enrollment Data
Data Quality Validation
Valid
Invalid
Business Rule Validation
Rejected Records
GOLD — BUSINESS-READY DATA
DIM_MEMBER
DIM_PLAN
DIM_GROUP
DIM_DATE
DIM_TRANSACTION_SOURCE
FACT_ENROLLMENT
DATA CONSUMPTION
Databricks SQL
BI / Dashboards
Natural Language Analytics

`BronzeLayer.ingest()`

```
class BronzeLayer:
    def __init__(self, spark: SparkSession, config: Dict):
        self.spark = spark
        self.config = config

    def ingest(self, landing_path: str) -> DataFrame:
        self.logger = setup_logging("bronze")
        start_time = log_pipeline_start(self.logger, "BRONZE", f"landing_path={landing
_path}")

        path = Path(landing_path)
        files = sorted(path.glob("*.834"))
        rows = []
        try:
            for file in files:
                content = file.read_text(encoding="utf-8")
                file_name = file.name
                for seg_order, line in enumerate(split_segments(content)):
                    elements = line.split("*")
                    rows.append({
                        "file_name": file_name,
                        "segment_id": elements[0],
                        "elements": elements,
                        "raw_segment": line,
                        "seg_order": seg_order,
                    })

            parsed = self.spark.createDataFrame(rows, schema=BRONZE_SCHEMA)
            parsed = parsed.withColumn("ingested_at", F.current_timestamp().cast("stri
ng"))

            table_path = self.config["layers"]["bronze"]["path"]
            (
                parsed.write.format("delta")
                .mode("overwrite")
                .option("mergeSchema", "true")
                .option("delta.autoOptimize.optimizeWrite", "true")
                .save(table_path)
            )
        except Exception:
            log_pipeline_error(self.logger, "BRONZE", f"landing_path={landing_path}",
start_time)
            raise

        result_count = parsed.count()
        log_pipeline_end(self.logger, "BRONZE", f"segments_loaded={result_count}", sta
rt_time)
        return parsed
```

Write semantics

Option	Value	Why
`mode`	`overwrite`	Each pipeline run ingests the current landing directory's files as a fresh Bronze snapshot
`mergeSchema`	`true`	Tolerates schema drift (e.g. an added column) without a manual `ALTER TABLE`
`delta.autoOptimize.optimizeWrite`	`true`	Compacts small files at write time — important because 834 ingestion produces many small per-file row batches

Output schema

```
BRONZE_SCHEMA = StructType([
    StructField("file_name", StringType(), True),
    StructField("segment_id", StringType(), True),
    StructField("elements", ArrayType(StringType()), True),
    StructField("raw_segment", StringType(), True),
    StructField("seg_order", LongType(), True),
    StructField("ingested_at", StringType(), True),
])
```

Stored at the Delta path configured in `03_config/pipeline_config.yaml`:

```
layers:
  bronze:
    path: 10_data/bronze/edi834_segments
    table: bronze_edi_segments
    format: delta
    options:
      mergeSchema: "true"
      delta.autoOptimize.optimizeWrite: "true"
```

On Databricks, the equivalent table is `med834enrl.edi.bronze_edi_segments` (see `04_infrastructure/02_sql/01_delta/bronze_edi_segments.sql`).

Databricks notebook equivalent

The production Databricks job (`05_orchestration/databricks/med834_medallion_job.py`) implements the same Bronze logic using `read_files()` SQL + `explode()` instead of a Python loop over local files, because it reads from a Unity Catalog Volume rather than the local filesystem:

```
raw = spark.sql(
    f"SELECT value, element_at(split(_metadata.file_path, '/'), -1) AS file_name "
    f"FROM read_files('{LANDING}', format => 'text', wholeText => true)"
)
bronze = (
    raw.withColumn("segment", F.explode(F.split(F.regexp_replace("value", r"[\r\n]+",
"~"), "~")))
    .filter(F.trim("segment") != "")
    .withColumn("elements", F.split(F.col("segment"), r"\*"))
    .withColumn("segment_id", F.element_at("elements", 1))
    .withColumn("seg_order", F.monotonically_increasing_id())
    .withColumn("raw_segment", F.col("segment"))
    .withColumn("ingested_at", F.current_timestamp())
    .select("file_name", "segment_id", "elements", "raw_segment", "seg_order", "ingest
ed_at")
)
bronze.write.format("delta").mode("overwrite").option("overwriteSchema", "true") \
    .saveAsTable(f"{EDI}.bronze_edi_segments")
```

Same segment-normalization idea (`~`/newline → `~`), same per-segment `elements` array shape — just expressed in distributed SQL/DataFrame operations instead of a Python-side file loop, because `read_files()` and `explode()` are what scale across a Databricks cluster reading many files from a Volume at once.

Pipeline summary contribution

`BronzeLayer.ingest()`'s row count feeds directly into the `DeltaMedallionPipeline.run()` summary dict (see Chapter 18):

```
summary = {
    "bronze_segments": bronze_df.count(),
    ...
}
```

14. Silver Layer

> Source: `src/medenroll834/ingestion/silver.py`

The Silver layer turns raw Bronze segments into one normalized row **per member**, resolving the X12 `HL` (hierarchical level) structure, extracting every relevant segment type, formatting dates, and applying validation — all as Spark DataFrame/window operations, with **no row-by-row Python loops** on the member data itself.

Why HL resolution is hard (and how it's solved)

X12 834 files model subscriber/dependent relationships through `HL` segments: `HL*<id>*<parent_id>*<level>*<child_count>`. The catch, documented directly in the code, is that **HL ids are only unique within a transaction set** (each `ST*834.../SE` pair restarts numbering at 1) — not within the whole file:

```
# HL ids are only unique *within a transaction set* (each ST*834.../SE
# pair restarts numbering at 1), not within the file. A file with 100
# enrolments contains 100 HL rows with hl_id=1, so resolving dependents
# against subscribers on hl_id alone matches every subscriber in the
# file and fans out 100x.
```

The fix is a transaction-scoped key, derived by carrying the most recent `ST` row forward with a window function:

```
file_order = Window.partitionBy("file_name").orderBy(F.col("seg_row"))
ordered = ordered.withColumn(
    "txn_row",
    F.last(
        F.when(F.col("segment_id") == "ST", F.col("seg_row")),
        ignorenulls=True,
    ).over(file_order.rowsBetween(Window.unboundedPreceding, Window.currentRow)),
)
ordered = ordered.withColumn(
    "hl_key",
    F.concat_ws("-", F.col("file_name"), F.col("txn_row").cast("string")),
)
```

An earlier version range-joined against every `ST` row instead, which matched a segment to all preceding transaction sets and multiplied member rows ~5x. `last()` over a cumulative window yields exactly one key per segment.

Deriving member row ranges from HL boundaries

Each `HL` segment marks the start of one member's data block; the next `HL` segment (within the same transaction) marks the end:

```
 hl_window = Window.partitionBy("hl_key").orderBy(F.col("seg_row"))
hl_with_next = hl_events.withColumn(
    "next_hl_row", F.lead("seg_row").over(hl_window)
).fillna(999999999, subset=["next_hl_row"])
## Segment extraction (NM1, INS, DTP, HD, EB, N3/N4, REF)

Each segment type is extracted with its own `groupBy(...).agg(F.max(F.when(...)))`
helper, keyed on `(file_name, seg_start)` — the member block identifier:

```python
def _extract_nm1_data(self, members_df):
    nm1_seg = members_df.filter(F.col("segment_id") == "NM1")
    return nm1_seg.groupBy("file_name", "seg_start", "hl_level").agg(
        F.max(F.when(F.col("elements")[1] == "IL", F.col("elements")[2])).alias("last_
name"),
        F.max(F.when(F.col("elements")[1] == "IL", F.col("elements")[3])).alias("first
_name"),
        F.max(F.when(F.col("elements")[1] == "IL", F.col("elements")[8])).alias("membe
r_id"),
    )
```

Helper	Segment	Extracts
`_extract_nm1_data`	`NM1`	last name, first name, member id (entity code `IL`)
`_extract_ins_data`	`INS`	relationship (via `RELATIONSHIP_MAP`; subscriber is hardcoded `"self"`)
`_extract_dtp_data`	`DTP`	DOB (`291`), coverage start (`472` / `348`), coverage end (`349`)
`_extract_hd_data`	`HD`	plan id
`_extract_eb_data`	`EB`	fallback coverage start/end (coalesced if `DTP` is absent)
`_extract_address_data`	`N3` / `N4` / `REF*2L`	street/city/state/zip, resolved to the member via the nearest preceding `REF*2L`
```

Subscriber resolution without a fan-out join

HL level `20` = subscriber, level `21` = dependent. A dependent must be tied to its subscriber's `member_id`. The naive approach — a self-join on `hl_parent_id == hl_id` — duplicated every dependent against every candidate subscriber and **blew the executor heap**. The fix is a broadcast map built from a small `collect()`:

```
 # Implemented with a broadcast map rather than a self-join. A join here
# duplicated every dependent against every candidate subscriber and
# blew the executor heap; a broadcast lookup is O(1) per row and cannot
# fan out because the key set is collapsed with first().
subscriber_map = {
    f"{r['hl_key']}|{r['hl_id']}": r["member_id"]
    for r in base.filter(F.col("is_subscriber"))
    .select("hl_key", "hl_id", "member_id")
    .dropDuplicates(["hl_key", "hl_id"])
    .collect()
}
subscriber_lookup = F.create_map(*[item for kv in subscriber_map.items() for item in (
F.lit(kv[0]), F.lit(kv[1]))])
base = base.withColumn(
    "subscriber_id",
    F.when(F.col("is_subscriber"), F.col("member_id"))
     .otherwise(F.element_at(subscriber_lookup, F.concat_ws("|", F.col("hl_key"), F.co
l("hl_parent_id")))),
)
```

This only `collect()`s the (small) set of distinct subscribers, not the full member population, so it scales with group count rather than member

Date formatting

X12 dates arrive as `YYYYMMDD` strings; Silver reformats them to `YYYY-MM-DD`:

```
def _format_date(self, col_name: str):
    return F.when(
        (F.length(F.col(col_name)) == 8) & F.col(col_name).rlike(r"^\d{8}$"),
        F.concat(F.substring(F.col(col_name), 1, 4), F.lit("-"),
                 F.substring(F.col(col_name), 5, 2), F.lit("-"),
                 F.substring(F.col(col_name), 7, 2)),
    ).otherwise(F.col(col_name))
```

Output schema

`SILVER_SCHEMA` adds `hl_id`, `hl_parent_id`, `subscriber_id`, `address_member_id`, `validation_errors`, and `is_valid` on top of the flattened member attributes — see `04_infrastructure/02_sql/01_delta/silver_members.sql` for the Delta table DDL. Written with `delta.deletedFileRetentionDuration: "interval 7 days"` per `pipeline_config.yaml`, giving a 7-day time-travel window for debugging.

See Chapter 15 for how Gold consumes `is_subscriber` / `subscriber_id`, and Chapter 16 for `_apply_validation()`.

count.

```

A range join (`seg_row >= seg_start AND seg_row < seg_end`) then attaches every segment between consecutive `HL`s to the right member block.
```

15. Business Rules / Plan Assignment

This chapter covers how a member is assigned to a benefit plan and rolled up under the correct subscriber/group — the pipeline's core "enrollment business rule" logic, spread across Silver (`HD` segment extraction, HL resolution) and Gold (`subscriber_id` finalization).

Where the plan code comes from

The `HD` (Health Coverage) segment carries the plan identifier. Silver's `_extract_hd_data()` (see Chapter 14) pulls element index 3 directly as the plan code:

```
def _extract_hd_data(self, members_df):
    hd_seg = members_df.filter(F.col("segment_id") == "HD")
    return hd_seg.groupBy("file_name", "seg_start").agg(
        F.max(F.col("elements")[3]).alias("plan_code"),
    ).withColumn("plan_id", F.col("plan_code")).select("file_name", "seg_start", "plan
_id")
```

Valid plan codes (the business rule)

The complete set of accepted plan products is defined once and shared between the generator, the validator, and `pipeline_config.yaml` :

```
VALID_PLANS = {"100", "101", "102", "103", "104", "105", "106", "107"}
```

```
# 03_config/pipeline_config.yaml
validation:
  require_plan_code: true
  valid_plan_codes:
    - "100"
    - "101"
    - "102"
    - "103"
    - "104"
    - "105"
    - "106"
    - "107"
```

Any `plan_id` outside this set produces a validation error (`"Unknown plan: <code>"`) and the row is quarantined — see Chapter 16.

Relationship-driven subscriber assignment

Plan assignment is not just "attach a plan code" — it must correctly determine **whose** enrollment record a plan applies to, by resolving the X12 relationship hierarchy:

```
RELATIONSHIP_MAP = {
    "18": "self",               # subscriber (primary policyholder)
    "01": "spouse",              # dependent
    "03": "child",               # dependent
    "19": "domestic_partner",    # dependent
}
```

HL level `20` always means subscriber; level `21` always means dependent. Silver materializes `is_subscriber` directly from this, because Gold's plan/group roll-up depends on it:

```
# HL level 20 = subscriber, 21 = dependent. Gold depends on this flag to
# roll dependents up to their subscriber, so it must be materialised
# here; leaving it null silently produces null group_id in Gold.
base = base.withColumn(
    "is_subscriber",
    F.when(F.col("hl_level") == "20", F.lit(True)).otherwise(F.lit(False)),
)
```

Gold finalizes the subscriber assignment

> Source: `src/medenroll834/ingestion/gold.py`

```
gold_df = (
    silver_df.withColumn(
        "subscriber_id",
        F.when(F.col("is_subscriber"), F.col("member_id")).otherwise(
            F.col("subscriber_id")
        ),
    )
    .filter(F.col("is_valid") == True)
    .dropDuplicates(["file_name", "member_id"])
    ...
)
```

This is a correction over an earlier version that joined on `group_id` — which Silver had actually populated from the `REF*2L` *address-association* key (a member id, not a group id). That join both failed to resolve correctly and fanned out rows. Because `subscriber_id` is already resolved in Silver via the HL hierarchy (see Chapter 14), Gold does no re-derivation

and no positional guesswork — it only finalizes the subscriber's own row to point at itself.

Enrollment maintenance type (OLTP load)

When loading into the OLTP CORE schema, every enrollment event is written with a fixed maintenance type code `"021"` (addition) and an `enrl_stat` of `"ACTIVE"`:

```
cur.execute(
    "INSERT INTO core.core_meen_enrollment (meme_id, plpl_id, txn_id, maint_typ_cd, "
    "eff_dt, term_dt, enrl_stat, crtd_ts, updt_ts) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
",
    (meme_id, plpl_id, txn_id, "021", _as_date(r.get("coverage_start_date")) or now.da
te(),
     _as_date_open_ended(r.get("coverage_end_date")),
     "ACTIVE" if _as_date(r.get("coverage_end_date")) is None else "TERMED", now, now)
,
)
```

`_as_date_open_ended()` replaces a missing/`NULL` coverage end date with the fixed `9999-12-31` sentinel (`OPEN_ENDED_DATE`) instead of `NULL`, so `term_dt`/`END_DATE` is always a real, queryable date across both OLTP and OLAP (see Chapter 19).

The OLAP `FACT_ENROLLMENT` builder in the Databricks notebook derives `ENROLLMENT_STATUS` dynamically instead, based on whether a coverage end date is present:

```
.withColumn("ENROLLMENT_STATUS",
    F.when(F.col("coverage_end_date").isNull() | (F.col("coverage_end_date") == ""),
           F.lit("ACTIVE")).otherwise(F.lit("TERMED")))
```

See Chapter 17 and Chapter 19 for the full downstream loads.

16. Data Quality & Reject Handling

MedEnroll834 validates every member record with Spark-SQL expressions (no row-by-row Python), tags each row with its specific failure reasons, and routes invalid rows to a dedicated Delta quarantine table instead of dropping or silently failing them.

Validation rules

> Source: `src/medenroll834/quality/validator.py` (standalone) and `SilverLayer._apply_validation()` (inline, identical logic)

```
class DataFrameValidator:
    VALID_PLANS = {"100", "101", "102", "103", "104", "105", "106", "107"}

    def validate(self, df: DataFrame) -> DataFrame:
        errors = F.filter(
            F.array(
                F.when(F.col("member_id") == "", F.lit("Member ID is required")),
                F.when(F.col("first_name") == "", F.lit("First name is required")),
                F.when(F.col("last_name") == "", F.lit("Last name is required")),
                F.when(F.col("plan_id") == "", F.lit("Plan ID is required")),
                F.when(
                    (F.col("coverage_start_date").isNotNull()) & (F.col("coverage_end_
date").isNotNull()),
                    F.when(F.try_to_date(F.col("coverage_start_date")) > F.try_to_date
(F.col("coverage_end_date")),
                           F.lit("Coverage start after end")),
                ),
                F.when(
                    (F.col("date_of_birth").isNotNull()) & (F.col("coverage_start_date
").isNotNull()),
                    F.when(F.try_to_date(F.col("coverage_start_date")) < F.try_to_date
(F.col("date_of_birth")),
                           F.lit("Coverage before DOB")),
                ),
                F.when(~F.col("plan_id").isin(list(self.VALID_PLANS)),
                       F.concat(F.lit("Unknown plan: "), F.col("plan_id"))),
            ),
            lambda x: x.isNotNull(),
        )
        return df.withColumn("validation_errors", errors).withColumn(
            "is_valid", F.size(F.col("validation_errors")) == 0
        )
```

The six validation rules

#	Rule	Error message
1	`member_id` required	`Member ID is required`
2	`first_name` required	`First name is required`
3	`last_name` required	`Last name is required`
4	`plan_id` required	`Plan ID is required`
5	coverage start ≤ coverage end (when both present)	`Coverage start after end`
6	coverage start ≥ date of birth (when both present)	`Coverage before DOB`
7	`plan_id` is one of the 8 valid plan codes	`Unknown plan: <code>`

Each row can accumulate **multiple** errors at once — `validation_errors` is an array, not a single flag, so a reject-remediation team sees every problem with a record in one place rather than fixing issues one at a time across repeated pipeline runs. `F.try_to_date()` is used (not `F.to_date()`) so a malformed date string produces `null` and is excluded from the comparison rather than crashing the whole DataFrame evaluation.

`is_valid` flag

```
.withColumn("is_valid", F.size(F.col("validation_errors")) == 0)
```

A row is valid if and only if its error array is empty — this single boolean is what the pipeline orchestrator (`DeltaMedallionPipeline.run()`) and the Gold layer filter on.

Quarantine routing

Source: `src/medenroll834/quality/quarantine.py`

```
class DeltaQuarantineManager:
    def write_invalid(self, invalid_df: DataFrame) -> None:
        quarantine_df = invalid_df.withColumn(
            "quarantined_at", F.current_timestamp().cast("string")
        ).withColumn("quarantine_id", F.expr("uuid()"))

        table_path = self.config["quarantine"]["path"]
        (
            quarantine_df.write.format("delta")
            .mode("append")
            .option("mergeSchema", "true")
            .save(table_path)
        )
```

Quarantine writes are **appended**, not overwritten — every run's rejects accumulate historically, each tagged with a unique `quarantine_id` and a `quarantined_at` timestamp, so a remediation team can audit when and how often a given record kept failing.

Where quarantine fits in the pipeline

```
# src/medenroll834/ingestion/pipeline.py
bronze_df = self.bronze.ingest(landing_path)
silver_df = self.silver.transform(bronze_df)
gold_df = self.gold.transform(silver_df)

valid_df = silver_df.filter(F.col("is_valid") == True)
invalid_df = silver_df.filter(F.col("is_valid") == False)
self.quarantine.write_invalid(invalid_df)
```

Quarantine operates on the **Silver** output (after HL resolution and field extraction, before Gold deduplication/enrichment) — so a quarantined record retains all the context Silver resolved about it (relationship, subscriber hierarchy, address) even though it failed validation, which makes remediation easier than quarantining raw Bronze segments.

OLTP-side rejects (`ENRL_RJRW_REJECT`)

The OLTP schema has a parallel reject concept at the operational layer:

```
CREATE TABLE IF NOT EXISTS ENRL.ENRL_RJRW_REJECT (
  RJRW_ID BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  TXN_ID BIGINT NOT NULL,
```

```
  SRC_ROW TEXT NOT NULL,
  RJCT_CD VARCHAR(20) NOT NULL,
  RJCT_RSN VARCHAR(255) NOT NULL,
  RJCT_TS TIMESTAMPTZ NOT NULL
);
```

FK'd to `ENRL_EDXN_TRANSACTION`, so every reject traces back to the specific inbound file transaction it came from — the OLTP equivalent of the Delta quarantine table's `file_name` / `quarantine_id` lineage.

Pipeline run summary

Quarantine counts are surfaced directly in the pipeline's return value, so reject volume is visible without a separate query:

```
summary = {
    "bronze_segments": bronze_df.count(),
    "silver_total": silver_df.count(),
    "silver_valid": valid_df.count(),
    "silver_invalid": invalid_df.count(),
    "gold_records": gold_df.count(),
}
```

17. PostgreSQL Bulk Load / Upsert

Source: `src/medenroll834/oltp/oltp_loader.py`

`load_gold_to_oltp()` reads the Databricks Gold table via a SQL warehouse and writes it into the PostgreSQL CORE/ENRL schemas, using natural-key upserts so re-running the load does not duplicate reference data (groups, plans, subscribers).

Reading Gold from Databricks

```
GOLD_TABLE = "med834enrl.edi.gold_enrollments"

def _read_gold_from_databricks(warehouse_id=None, profile=None) -> List[Dict]:
    from databricks.sdk import WorkspaceClient
    from databricks.sdk.service.sql import StatementState

    w = WorkspaceClient(**({"profile": profile} if profile else {}))
    if not warehouse_id:
        whs = list(w.warehouses.list())
        warehouse_id = whs[0].id

    sql = (
        "SELECT member_id, first_name, last_name, date_of_birth, gender, "
        "group_id, group_name, plan_id, coverage_start_date, coverage_end_date, "
        "street_1, street_2, city, state, zip_code, relationship, is_subscriber, "
        f"file_name FROM {GOLD_TABLE}"
    )
    resp = w.statement_execution.execute_statement(
        warehouse_id=warehouse_id, statement=sql, wait_timeout="50s"
    )
    while resp.status.state in (StatementState.PENDING, StatementState.RUNNING):
        time.sleep(2)
        resp = w.statement_execution.get_statement(resp.statement_id)
    if resp.status.state != StatementState.SUCCEEDED:
        raise RuntimeError(f"Gold read failed: {resp.status.error.message}")

    cols = [c.name for c in resp.manifest.schema.columns]
    return [dict(zip(cols, r)) for r in (resp.result.data_array or [])]
```

Uses the `databricks-sdk`'s SQL Statement Execution API — no JDBC driver, no cluster to keep warm; the serverless SQL warehouse spins up on demand.

The load: one transaction, natural-key upserts

```
def load_gold_to_oltp(oltp_db_url=None, warehouse_id=None, databricks_profile=None, r
ows=None) -> Dict:
    oltp_db_url = oltp_db_url or os.environ["OLTP_DATABASE_URL"]
    if rows is None:
        rows = _read_gold_from_databricks(warehouse_id, databricks_profile)

    conn = psycopg2.connect(oltp_db_url)
    conn.autocommit = False
    cur = conn.cursor()
    loaded = {"groups": 0, "plans": 0, "subscribers": 0, "members": 0,
              "addresses": 0, "transactions": 0, "enrollments": 0}
    try:
        # ... per-row processing (below) ...
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()
        conn.close()
    return loaded
```

The **entire load runs in one transaction** — any failure partway through rolls back everything, so the OLTP database never ends up with a half-loaded file's data.

Upsert pattern (`ON CONFLICT ... DO UPDATE`)

Reference entities (group, plan, subscriber) are upserted on their natural key, with an in-memory cache (`group_cache`, `plan_cache`) to avoid redundant round-trips within the same load:

```
if grp_num not in group_cache:
    cur.execute(
        "INSERT INTO core.core_grgr_group (grgr_num, st_cd, eff_dt, crtd_ts, updt_ts)
"
        "VALUES (%s,%s,%s,%s,%s) ON CONFLICT (grgr_num) DO UPDATE SET updt_ts=EXCLUDED
.updt_ts "
        "RETURNING grgr_id",
        (grp_num, (r.get("state") or "NA")[:2], _as_date(r.get("coverage_start_date"))
 or now.date(), now, now),
    )
    group_cache[grp_num] = cur.fetchone()[0]
```

This is why the loader is **idempotent**: re-running it against the same Gold data updates `updt_ts` on existing groups/plans/subscribers instead of inserting duplicates — see Chapter

22.

Member and enrollment inserts (not upserted)

`core_meme_member` and `core_meen_enrollment` are plain inserts — a member row is created fresh for each processed Gold record (member identity is owned by the subscriber's natural key + relationship code, not a separately upserted key in this simplified schema):

```
cur.execute(
    "INSERT INTO core.core_meme_member (mesb_id, rltn_cd, fst_nm, lst_nm, dob, gndr_cd
, "
    "is_act, crtd_ts, updt_ts) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s) RETURNING meme_id",
    (mesb_id, _rel_code(r.get("relationship")), ..., True, now, now),
)
meme_id = cur.fetchone()[0]
```

One transaction row created per file

```
for file_nm in {r["file_name"] for r in rows if r.get("file_name")}:
    cur.execute(
        "INSERT INTO enrl.enrl_edxn_transaction "
        "(file_nm, isa_ctrl_num, gs_ctrl_num, st_ctrl_num, sndr_id, rcvr_id, "
        "rcvd_ts, proc_stat, rec_cnt) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s) RETURNING tx
n_id",
        (file_nm, "N/A", "N/A", "N/A", "SENDERID", "RECEIVERID", now, "PROCESSED", 0),
    )
```

`file_txn` maps each file name to its transaction id, so every `core_meen_enrollment` row can be FK'd back to the specific 834 file it came from — this is the OLTP-side equivalent of Gold's `file_name` lineage column.

Helper conversions

```
def _as_date(value):
    """Try YYYY-MM-DD then YYYYMMDD; returns None if neither parses."""
    ...

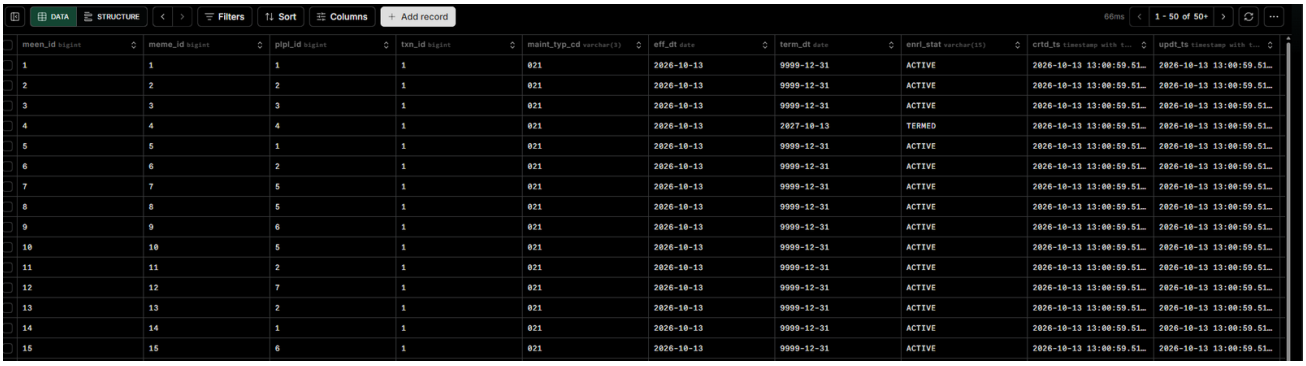
def _rel_code(relationship):
    """Reverse-maps Silver's relationship string back to the X12 code (e.g. 'spouse' -
> '01')."""
    mapping = {"self": "18", "spouse": "01", "child": "03", "domestic_partner": "19"}
    return mapping.get((relationship or "self").lower(), "18")
```

Invoked from the Airflow DAG

```
 # med834_e2e_pipeline.py
@task
def load_oltp_database(_run_id: int) -> dict:
    from src.medenroll834.oltp.oltp_loader import load_gold_to_oltp
    summary = load_gold_to_oltp(oltp_db_url=_cfg("OLTP_DATABASE_URL"),
                                warehouse_id=_cfg("DATABRICKS_WAREHOUSE_ID"))
    return summary
```

See Chapter 21 for the full DAG task graph this step sits in.

Verified load evidence



18. Gold Layer

Source: `src/medenroll834/ingestion/gold.py`

The Gold layer is the final, **analytics-ready** output: deduplicated, valid-only enrollment records with derived convenience fields, written with Change Data Feed enabled so downstream consumers (OLAP star schema builder, OLTP loader) can read incrementally in the future.

Full transform

```
class GoldLayer:
    def transform(self, silver_df: DataFrame) -> DataFrame:
        self.logger = setup_logging("gold")
        start_time = log_pipeline_start(self.logger, "GOLD", "transforming silver to g
old")

        try:
            gold_df = (
                silver_df.withColumn(
                    "subscriber_id",
                    F.when(F.col("is_subscriber"), F.col("member_id")).otherwise(
                        F.col("subscriber_id")
                    ),
                )
                .filter(F.col("is_valid") == True)
                .dropDuplicates(["file_name", "member_id"])
                .withColumn("full_name", F.concat_ws(" ", F.col("first_name"), F.col("
last_name")))
                .withColumn(
                    "age_years",
                    F.floor(F.datediff(F.current_date(), F.to_date(F.col("date_of_birt
h"), "yyyy-MM-dd")) / 365.25),
                )
                .withColumn("ingested_at", F.current_timestamp().cast("string"))
            )

            table_path = self.config["layers"]["gold"]["path"]
            (
                gold_df.write.format("delta")
                .mode("overwrite")
                .option("mergeSchema", "true")
                .option("delta.enableChangeDataFeed", "true")
                .save(table_path)
            )
        except Exception:
            log_pipeline_error(self.logger, "GOLD", "transforming silver to gold", sta
rt_time)
            raise

        log_pipeline_end(self.logger, "GOLD", f"records={gold_df.count()}", start_time
)
        return gold_df
```

What Gold does, step by step

1. **Finalize** `subscriber_id` — see Chapter 15 for why this does **no** re-derivation from `group_id` (a past bug).
2. **Filter to** `is_valid == True` — invalid rows never reach Gold; they were already routed to quarantine from Silver (see Chapter 16).
3. **Deduplicate on** `(file_name, member_id)` — guards against the same member appearing twice within one file (e.g. duplicate `HL` blocks from a malformed or re-sent 834).
4. **Derive** `full_name` — `concat_ws(" ", first_name, last_name)`, a display convenience not present in Silver.
5. **Derive** `age_years` — `floor(datediff(today, dob) / 365.25)`, accounting for leap years via the `365.25` divisor rather than a flat `365`.
6. **Write with Change Data Feed enabled** — `delta.enableChangeDataFeed` lets a future incremental consumer read only the rows that changed since a given version, instead of re-scanning the whole Gold table.

Write semantics

Option	Value	Why
`mode`	`overwrite`	Gold reflects the current full state of valid enrollments per run
`mergeSchema`	`true`	Tolerates schema drift from upstream Silver changes
`delta.enableChangeDataFeed`	`true`	Enables future incremental/CDC-style consumption

Configured path (`03_config/pipeline_config.yaml`):

```
layers:
  gold:
    path: 10_data/gold/enrollments
    table: gold_enrollments
    format: delta
    options:
      mergeSchema: "true"
      delta.enableChangeDataFeed: "true"
```

On Databricks, this is `med834enrl.edi.gold_enrollments` — the table read by both the OLTP loader (`_read_gold_from_databricks`) and the report generator's `REPORT_QUERIES` (see Chapter 27).

Gold's row count in the pipeline summary

```
summary = {
    "bronze_segments": bronze_df.count(),
    "silver_total": silver_df.count(),
    "silver_valid": valid_df.count(),
    "silver_invalid": invalid_df.count(),
    "gold_records": gold_df.count(),
}
```

From the verified smoke-test run in Chapter 1: `gold_records = 6` from 6 valid Silver rows (3 subscribers + 3 dependents), matching 1:1 because none were duplicates.

Gold feeds two independent downstream paths — see Chapter 17 for the OLTP load and Chapter 19 for the OLAP star schema build.

19. OLAP Star Schema

> Source: `05_orchestration/databricks/med834_medallion_job.py` (section `# ---------- OLAP star-schema dimensions ----------`)

The OLAP star schema is built directly in the Databricks notebook job, immediately after the medallion Bronze/Silver/Gold run, from the same `gold_enrollments` table. `FACT_ENROLLMENT` sits at the center, surrounded by five dimensions — see Chapter 9 for the full DDL and ERD.

Dimension builds (all `SCD Type 2`-shaped via `CURRENT_FLAG`)

```
g = spark.table(f"{EDI}.gold_enrollments")

# DIM_PLAN - PLAN_TIER_NAME maps the numeric plan code (e.g. "100") to a
# readable tier label ("GOLD", "SILVER", ...) for PLAN_NAME synthesis.
(g.select(F.col("plan_id").alias("PLPL_ID"), F.col("plan_id").alias("PLAN_CODE"),
          F.concat(F.coalesce(PLAN_TIER_NAME[F.col("plan_id")], F.col("plan_id")), F.l
it(" Plan")).alias("PLAN_NAME"))
  .withColumn("LINE_OF_BUSINESS_CODE", F.lit("COMMERCIAL")).dropDuplicates(["PLPL_ID"]
)
  .withColumn("EFFECTIVE_DATE", F.current_date()).withColumn("END_DATE", F.to_date(F.l
it("9999-12-31")))
  .withColumn("CURRENT_FLAG", F.lit(True))
  .write.format("delta").mode("overwrite").option("mergeSchema", "true").saveAsTable(f
"{GOLD}.dim_plan"))

# DIM_GROUP - GRGR_ID/GROUP_NUMBER come from the REF*1L group number captured
# in Silver, not a hardcoded default.
(g.select(F.col("group_id").alias("GRGR_ID"), F.col("group_id").alias("GROUP_NUMBER"),
          F.col("group_name").alias("GROUP_NAME"), F.col("state").alias("STATE_CODE"))
.dropDuplicates(["GRGR_ID"])
  .withColumn("EFFECTIVE_DATE", F.current_date()).withColumn("END_DATE", F.to_date(F.l
it("9999-12-31")))
  .withColumn("CURRENT_FLAG", F.lit(True))
  .write.format("delta").mode("overwrite").option("mergeSchema", "true").saveAsTable(f
"{GOLD}.dim_group"))

# DIM_MEMBER
(g.select(F.col("member_id").alias("MEME_ID"), F.col("member_id").alias("MESB_ID"),
          F.col("first_name").alias("FIRST_NAME"), F.col("last_name").alias("LAST_NAME
"),
          F.to_date("date_of_birth", "yyyyMMdd").alias("DATE_OF_BIRTH"), F.col("gender
").alias("GENDER_CODE"),
          F.lit("18").alias("RELATIONSHIP_CODE")).dropDuplicates(["MEME_ID"])
  .withColumn("EFFECTIVE_DATE", F.current_date()).withColumn("END_DATE", F.to_date(F.l
it("9999-12-31")))
  .withColumn("CURRENT_FLAG", F.lit(True))
  .write.format("delta").mode("overwrite").option("mergeSchema", "true").saveAsTable(f
"{GOLD}.dim_member"))
```

`END_DATE` is never `NULL` — an active/open-ended dimension row stores the fixed `9999-12-31` sentinel instead, so "no end date yet" is a real, queryable date rather than `NULL` (see Chapter 9).

`DIM_DATE` — generated date spine

Rather than hardcoding a date range, `DIM_DATE` is generated to span the actual min/max coverage dates seen in Gold, padded by a year on each side:

```
date_bounds = g.select(
    F.to_date("coverage_start_date", "yyyyMMdd").alias("d1"),
    F.to_date("coverage_end_date", "yyyyMMdd").alias("d2"),
).agg(F.min("d1").alias("min_d"), F.max(F.greatest("d1", "d2")).alias("max_d")).first(
)
min_d = (date_bounds["min_d"] or today) - timedelta(days=365)
max_d = (date_bounds["max_d"] or today) + timedelta(days=365)

dim_date = (
    spark.sql(f"SELECT explode(sequence(to_date('{min_d}'), to_date('{max_d}'), interv
al 1 day)) AS FULL_DATE")
    .withColumn("DATE_KEY", F.date_format("FULL_DATE", "yyyyMMdd").cast("int"))
    .withColumn("YEAR_NUMBER", F.year("FULL_DATE"))
    .withColumn("QUARTER_NUMBER", F.quarter("FULL_DATE"))
    .withColumn("MONTH_NUMBER", F.month("FULL_DATE"))
    .withColumn("DAY_NUMBER", F.dayofmonth("FULL_DATE"))
    .withColumn("DAY_OF_WEEK_NAME", F.date_format("FULL_DATE", "EEEE"))
)
```

`DATE_KEY` is an `INT` in `YYYYMMDD` form (e.g. `20260827`), matching the surrogate-key convention documented in the table comment: `'DATE_KEY uses YYYYMMDD format (e.g., 20260827 = 27-Aug-2026)'`.

A fixed sentinel row is appended with `DATE_KEY = 99991231` / `FULL_DATE = 9999-12-31` / `DAY_OF_WEEK_NAME = "Open Ended"`, so `FACT_ENROLLMENT.COVERAGE_END_DATE_KEY` always has a valid `DIM_DATE` row to join against even for open-ended coverage — see below.

`DIM_TRANSACTION_SOURCE` — hashed file identity

```
dim_txn_src = (
    g.select("file_name").dropDuplicates()
    .withColumn("TXN_ID", F.abs(F.xxhash64("file_name")))
    .withColumn("FILE_NAME", F.col("file_name"))
    .withColumn("SENDER_ID", F.lit("SENDERID"))
    .withColumn("RECEIVED_TIMESTAMP", F.current_timestamp())
)
dim_txn_src.write.format("delta").mode("overwrite").option("mergeSchema", "true") \
    .saveAsTable(f"{GOLD}.dim_transaction_source")
```

`xxhash64` deterministically derives a numeric `TXN_ID` from the file name, so the same file always maps to the same transaction-source key across runs without needing a separate sequence/lookup table.

`FACT_ENROLLMENT` — resolving surrogate keys

The fact table is built by joining Gold against the **current** row of each dimension (`CURRENT_FLAG = true`) on natural keys, then selecting only the surrogate keys:

```
dim_member_cur = spark.table(f"{GOLD}.dim_member").filter("CURRENT_FLAG = true") \
    .select(F.col("MEME_KEY"), F.col("MEME_ID").alias("jk_meme_id"))
dim_plan_cur = spark.table(f"{GOLD}.dim_plan").filter("CURRENT_FLAG = true") \
    .select(F.col("PLPL_KEY"), F.col("PLPL_ID").alias("jk_plpl_id"))
# ... dim_group_cur, dim_txn_tbl similarly ...

fact = (
    g.withColumn("jk_cov_start_key", F.date_format(F.to_date("coverage_start_date", "y
yyyMMdd"), "yyyyMMdd").cast("int"))
     # Open-ended coverage resolves to the 9999-12-31 sentinel key instead of NULL
     .withColumn("jk_cov_end_key", F.coalesce(
         F.date_format(F.to_date("coverage_end_date", "yyyyMMdd"), "yyyyMMdd").cast("i
nt"),
         F.lit(99991231),
     ))
     .join(dim_member_cur, F.col("member_id") == F.col("jk_meme_id"), "left").drop("jk
_meme_id")
     .join(dim_plan_cur, F.col("plan_id") == F.col("jk_plpl_id"), "left").drop("jk_plp
l_id")
     .join(dim_group_cur, F.col("group_id") == F.col("jk_grgr_id"), "left").drop("jk_g
rgr_id")
     .join(dim_txn_tbl, F.col("file_name") == F.col("jk_file_name"), "left").drop("jk_
file_name")
     .join(dim_date_start, F.col("jk_cov_start_key") == F.col("start_date_key"), "left
")
     .join(dim_date_end, F.col("jk_cov_end_key") == F.col("end_date_key"), "left")
     .withColumn("MAINTENANCE_TYPE_CODE", F.lit("021"))
     .withColumn("ENROLLMENT_STATUS",
         F.when(F.col("coverage_end_date").isNull() | (F.col("coverage_end_date") == "
"),
                F.lit("ACTIVE")).otherwise(F.lit("TERMED")))
     .withColumn("ENROLLMENT_COUNT", F.lit(1))
     .withColumn("LOAD_TIMESTAMP", F.current_timestamp())
     .filter(F.col("MEME_KEY").isNotNull() & F.col("PLPL_KEY").isNotNull()
             & F.col("GRGR_KEY").isNotNull() & F.col("TXN_SRC_KEY").isNotNull()
             & F.col("COVERAGE_START_DATE_KEY").isNotNull())
)
fact.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsT
able(f"{GOLD}.fact_enrollment")
```

The final `.filter(...)` enforces **referential integrity at write time** — a Gold row that fails to resolve against any required dimension (e.g. a plan code dimension row wasn't built yet) is excluded from the fact table rather than being written with a null foreign key.

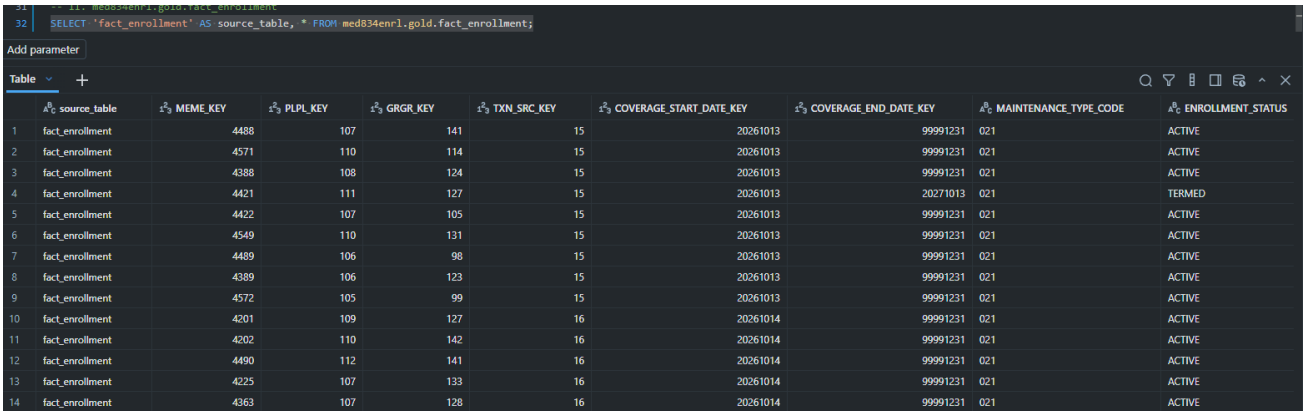
Verifying the star schema

Source: `07_scripts/03_utils/verify_star_schema.py`

```
print("Sample FACT_ENROLLMENT joined to all dims:")
print(q("""
    SELECT p.PLAN_CODE, gr.GROUP_NUMBER, m.FIRST_NAME, m.LAST_NAME,
           d1.FULL_DATE AS coverage_start, f.ENROLLMENT_STATUS
    FROM med834enrl.gold.fact_enrollment f
    JOIN med834enrl.gold.dim_plan p ON f.PLPL_KEY = p.PLPL_KEY
    JOIN med834enrl.gold.dim_group gr ON f.GRGR_KEY = gr.GRGR_KEY
    JOIN med834enrl.gold.dim_member m ON f.MEME_KEY = m.MEME_KEY
    JOIN med834enrl.gold.dim_date d1 ON f.COVERAGE_START_DATE_KEY = d1.DATE_KEY
    LIMIT 5
"""))
```

See Chapter 27 for how this star schema is queried by the HTML report and Genie.

Verified fact table evidence



```
SELECT 'fact_enrollment' AS source_table, * FROM med834enrl.gold.fact_enrollment;
```

Add parameter

Table

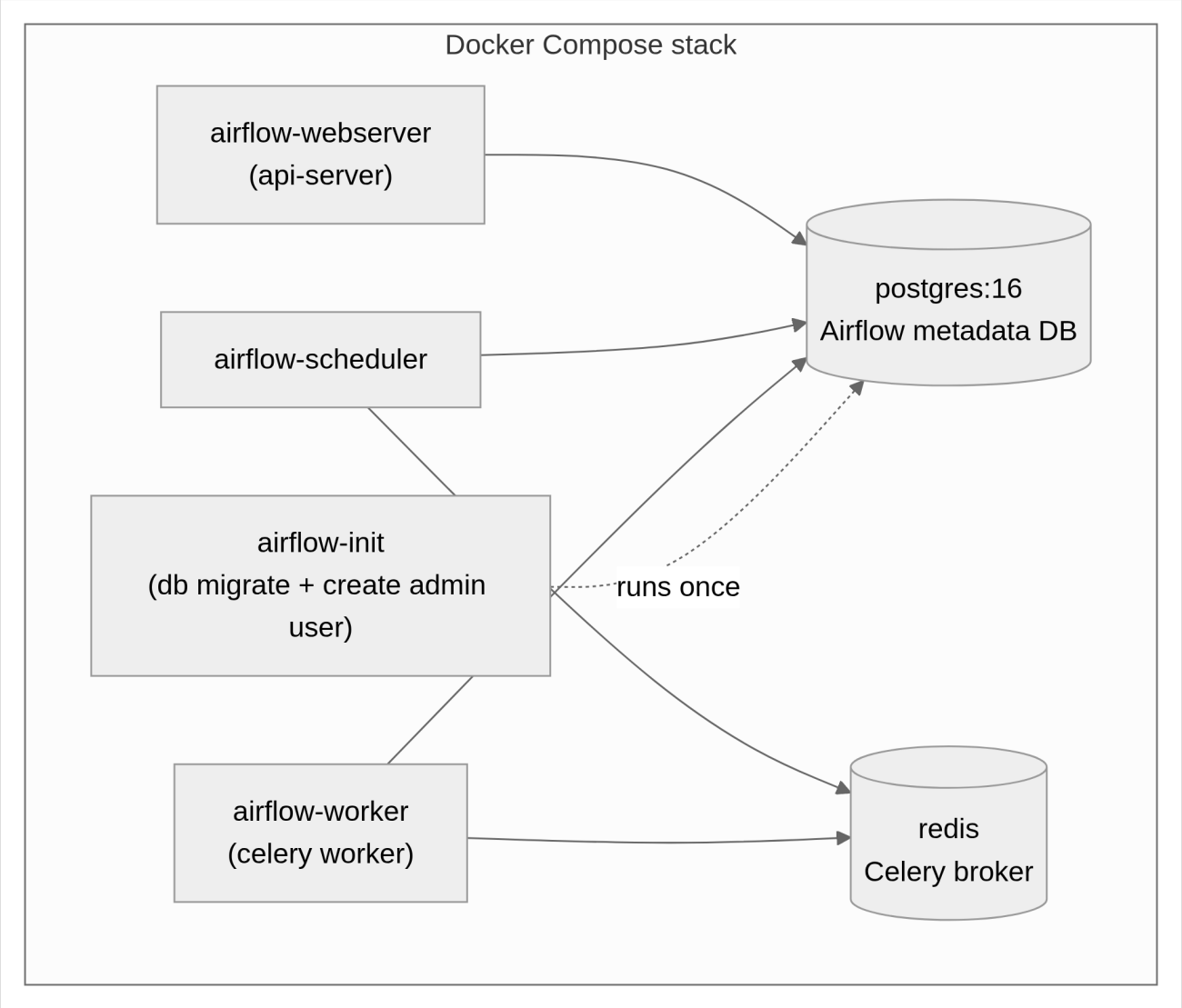
	source_table	MEME_KEY	PLPL_KEY	GRGR_KEY	TXN_SRC_KEY	COVERAGE_START_DATE_KEY	COVERAGE_END_DATE_KEY	MAINTENANCE_TYPE_CODE	ENROLLMENT_STATUS
1	fact_enrollment	4488	107	141	15	20261013	99991231	021	ACTIVE
2	fact_enrollment	4571	110	114	15	20261013	99991231	021	ACTIVE
3	fact_enrollment	4388	108	124	15	20261013	99991231	021	ACTIVE
4	fact_enrollment	4421	111	127	15	20261013	20271013	021	TERMED
5	fact_enrollment	4422	107	105	15	20261013	99991231	021	ACTIVE
6	fact_enrollment	4549	110	131	15	20261013	99991231	021	ACTIVE
7	fact_enrollment	4489	106	98	15	20261013	99991231	021	ACTIVE
8	fact_enrollment	4389	106	123	15	20261013	99991231	021	ACTIVE
9	fact_enrollment	4572	105	99	15	20261013	99991231	021	ACTIVE
10	fact_enrollment	4201	109	127	16	20261014	99991231	021	ACTIVE
11	fact_enrollment	4202	110	142	16	20261014	99991231	021	ACTIVE
12	fact_enrollment	4490	112	141	16	20261014	99991231	021	ACTIVE
13	fact_enrollment	4225	107	133	16	20261014	99991231	021	ACTIVE
14	fact_enrollment	4363	107	128	16	20261014	99991231	021	ACTIVE

20. Airflow Architecture

MedEnroll834 orchestrates its end-to-end workflow with **Apache Airflow 3.x**, running in Docker Compose with the `CeleryExecutor`. Airflow sits **above** the pipeline as pure orchestration — it owns scheduling, retries, task dependencies, and failure handling, but no transformation logic itself lives in a DAG task beyond thin calls into `src/medenroll834/`.

Stack topology

> Source: `docker-compose.yaml`



Service	Role
`postgres`	Airflow's own metadata database (separate from the pipeline's OLTP database)
`redis`	Celery broker for task distribution to workers
`airflow-webserver`	`api-server` command — serves the UI at `localhost:8080`
`airflow-scheduler`	Parses DAGs, schedules task instances
`airflow-worker`	`celery worker` — executes task instances
`airflow-init`	One-shot: `airflow db migrate` + creates the `admin` / `admin` user

Project mount and dependency injection

```
x-airflow-common: &airflow-common
  image: apache/airflow:3.3.0
  environment: &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: CeleryExecutor
    _PIP_ADDITIONAL_REQUIREMENTS: "databricks-sdk>=0.20.0 boto3>=1.34.0 psycopg2-binar
y>=2.9.9"
    BLOB_STORAGE_TOKEN: ${BLOB_STORAGE_TOKEN}
    OLTP_DATABASE_URL: ${OLTP_DATABASE_URL}
    DATABRICKS_HOST: ${DATABRICKS_HOST}
    DATABRICKS_TOKEN: ${DATABRICKS_TOKEN}
    DATABRICKS_JOB_ID: ${DATABRICKS_JOB_ID}
    DATABRICKS_WAREHOUSE_ID: ${DATABRICKS_WAREHOUSE_ID}
    GENIE_SPACE_ID: ${GENIE_SPACE_ID}
    PYTHONPATH: /opt/airflow/project
  volumes:
    - ./05_orchestration/airflow/01_dags:/opt/airflow/dags
    - airflow-logs:/opt/airflow/logs
    - .:/opt/airflow/project
```

Two decisions make the DAG able to import the actual pipeline code without packaging it as a wheel:

1. **The whole repo is bind-mounted** at `/opt/airflow/project`, so `src/medenroll834/` and `generator/` are importable paths.
2. `PYTHONPATH=/opt/airflow/project` makes those imports resolve inside the Airflow container without a separate `pip install -e .` step at container build time.
3. `_PIP_ADDITIONAL_REQUIREMENTS` installs `databricks-sdk`, `boto3`, and `psycopg2-binary` at container startup (the official Airflow image doesn't ship these by default).

Configuration precedence

Every pipeline secret/setting can be overridden by an **Airflow Variable**, falling back to the environment variable if unset:

```
def _cfg(key: str, default: str | None = None) -> str | None:
    try:
        from airflow.models import Variable
        val = Variable.get(key, default_var=None)
        if val:
            return val
    except Exception:
        pass
    return os.environ.get(key, default)
```

This lets an operator override `DATABRICKS_WAREHOUSE_ID` or `GENIE_SPACE_ID` from the Airflow UI (Admin → Variables) without touching `.env` or restarting containers — useful for pointing at a different SQL warehouse during a one-off investigation.

Why CeleryExecutor (not LocalExecutor)

CeleryExecutor was chosen over the simpler `LocalExecutor` so the `airflow-worker` service can scale independently of the scheduler — the stack is structured the same way a production Airflow deployment would be, even though this POC runs a single worker replica.

See Chapter 21 for the actual DAG, and Chapter 32 for standing the stack up locally.

21. Airflow DAG Implementation

> Source: `05_orchestration/airflow/01_dags/med834_e2e_pipeline.py`

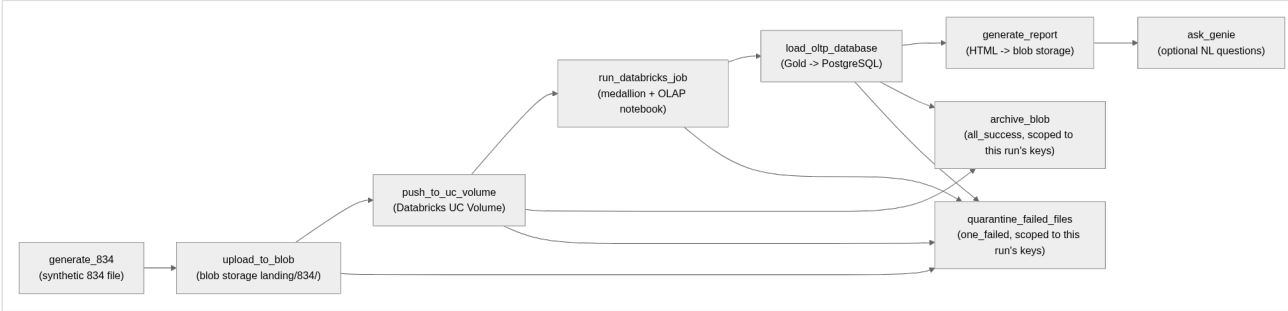
`med834_e2e_pipeline` is written with Airflow's **TaskFlow API** (`@dag` / `@task` decorators), which keeps each stage as a typed Python function with explicit inputs/outputs instead of operator boilerplate.

DAG declaration

```
@dag(
    dag_id="med834_e2e_pipeline",
    description="MedEnroll834 e2e: 834 -> blob storage -> UC Volume -> Databricks -> O
LTP DB -> Report/Genie",
    default_args={"owner": "med834enrl", "retries": 1, "retry_delay": timedelta(minute
s=3)},
    schedule="@daily",
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    catchup=False,
    tags=["med834enrl", "e2e", "medallion", "blob-storage"],
)
def med834_e2e():
    ...
```

`catchup=False` means no backfill runs are scheduled for the gap between `start_date` and "now" — only the daily run going forward. Every task inherits `retries=1` with a 3-minute delay from `default_args`.

Task graph




```
local = generate_834()
uploaded = upload_to_blob(local)          # returns exact blob keys, e.g.
                                           # ["landing/834/enrollment_...834"]
pushed = push_to_uc_volume(uploaded)      # pushes + returns the same keys
run_id = run_databricks_job(pushed)
```

```
summary = load_oltp_database(run_id)
report = generate_report(summary)
genie = ask_genie(report)

# archive_blob depends on the core chain's own outputs (pushed, summary),
# not on genie -- default all_success trigger rule means Airflow skips it
# automatically if push/Databricks/OLTP failed.
archive_blob(pushed) << summary

# quarantine_failed_files (trigger_rule="one_failed") is wired to the same
# core chain and fires if any of those tasks failed or were UPSTREAM_FAILED,
# moving this run's exact files to failed/834/ instead of landing/834/.
quarantine_failed_files(uploaded) << [pushed, run_id, summary]
```

`uploaded` / `pushed` carry the **exact blob keys this run touched** end to end via XCom -- `push_to_uc_volume`, `archive_blob`, and `quarantine_failed_files` all operate on that list directly rather than re-listing `landing/834/`, so a concurrent or straggling file from another run can never be mistakenly archived or quarantined alongside this run's files. See Chapter 23 for why this replaced an earlier `trigger_rule="all_done"` + re-list design that could archive a file Databricks had actually failed to process.

Why files are staged into a Unity Catalog Volume

```
UC_VOLUME_PATH = "/Volumes/med834enrl/edi/landing/834"
```

Serverless Databricks Free Edition **cannot read external buckets directly**, so `push_to_uc_volume` downloads each file from blob storage and re-uploads it into the UC Volume via `WorkspaceClient().files.upload()` before the Databricks job runs:

```
@task
def push_to_uc_volume(keys: list[str]) -> list[str]:
    """Pushes only the exact keys this run uploaded (received as an
    argument, never a fresh blob_storage.list_landing_files() call) and
    passes that same key list through unchanged."""
    from databricks.sdk import WorkspaceClient
    from src.medenroll834.storage import blob_storage
    w = WorkspaceClient()
    w.files.create_directory(UC_VOLUME_PATH)
    for key in keys:
        name = key.split("/")[-1]
        local = f"/tmp/{name}"
        blob_storage.download_file(key, local)
        with open(local, "rb") as fh:
            w.files.upload(f"{UC_VOLUME_PATH}/{name}", fh, overwrite=True)
    return keys
```

This is a POC-specific hop — in the enterprise target architecture where the landing layer is AWS S3 directly, a Databricks job with an instance profile can read S3 natively and this staging task would be unnecessary.

Running and waiting on the Databricks job

```
@task
def run_databricks_job(_pushed_keys: list[str]) -> int:
    from databricks.sdk import WorkspaceClient
    job_id = int(_cfg("DATABRICKS_JOB_ID"))
    run = WorkspaceClient().jobs.run_now_and_wait(job_id=job_id)
    state = run.state.result_state.value if run.state and run.state.result_state else
"UNKNOWN"
    if state != "SUCCESS":
        raise RuntimeError(f"Databricks job failed: {state}")
    return run.run_id
```

`run_now_and_wait()` blocks the Airflow task until the Databricks job finishes, so Airflow's own retry mechanism covers both transient Databricks API failures and job-level failures — a non-`SUCCESS` result state is raised as a Python exception, which Airflow treats as a task failure subject to the DAG's `retries=1`.

Archiving is scoped and success-gated; failures are quarantined, not archived

```
@task
def archive_blob(keys: list[str]) -> int:
    """No trigger_rule override -- default all_success means Airflow skips
    this task automatically if push_to_uc_volume, run_databricks_job, or
    load_oltp_database failed. Wired directly off those tasks' outputs
    (pushed, summary), not off the optional generate_report/ask_genie
    branch."""
    from src.medenroll834.storage import blob_storage
    for key in keys:
        blob_storage.archive_834(key)
    return len(keys)


@task(trigger_rule="one_failed")
def quarantine_failed_files(keys: list[str] | None) -> int:
    """Fires if any task in the core chain (push_to_uc_volume,
    run_databricks_job, load_oltp_database, or upload_to_blob itself)
    failed or was UPSTREAM_FAILED. Moves this run's exact files to
    failed/834/ instead of leaving them ambiguous in landing/834/ or
    archiving them as if they had succeeded."""
    keys = keys or []
    if not keys:
        return 0
    from src.medenroll834.storage import blob_storage
    for key in keys:
        blob_storage.archive_834(key, archive_prefix="failed/834/")
    return len(keys)
```

An earlier version used `trigger_rule="all_done"` on a single `archive_blob` task that re-listed `landing/834/` from scratch — that let archiving run (and silently move files) even when `run_databricks_job` / `load_oltp_database` had failed, and it was not scoped to the files this run actually uploaded. The current design fixes both problems: `archive_blob` only runs on full core-chain success and only touches this run's exact keys (passed through via XCom from `upload_to_blob`), while `quarantine_failed_files` handles the failure path explicitly by relocating those same keys to `failed/834/` rather than silently archiving or leaving them in `landing/834/`. See Chapter 23 for the full failure-handling rationale.

Genie is skippable

```
@task
def ask_genie(_report_uri: str) -> dict:
    space_id = _cfg("GENIE_SPACE_ID")
    if not space_id:
        print("GENIE_SPACE_ID not set - skipping Genie step")
        return {"skipped": True}
    ...
```

Genie is strictly optional — the DAG runs end to end with just the HTML report path if `GENIE_SPACE_ID` is unset. See Chapter 28.

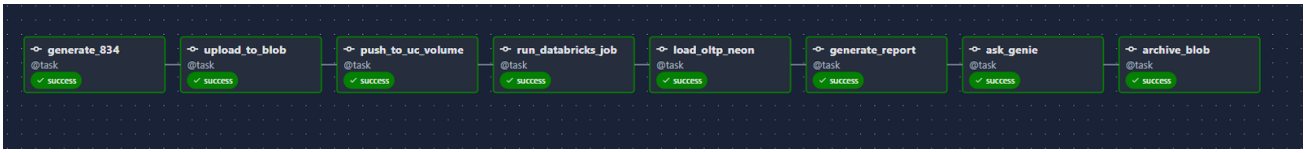
Validated DAG import

The DAG has been verified to load with **zero import errors** in the live Airflow scheduler:

```
docker exec medenroll834-airflow-scheduler-1 airflow dags list-import-errors
# -> No data found
```

See Chapter 33 for the full step-by-step runbook to deploy and trigger this DAG.

Verified run evidence





22. Idempotency & Incremental Processing

Every layer in MedEnroll834 is designed so **re-running it with the same input produces the same result** — a core requirement for a pipeline that Airflow may retry automatically.

Delta layer idempotency: `overwrite` + natural grain

Bronze, Silver, and Gold all write with `mode("overwrite")`:

```
# Bronze
parsed.write.format("delta").mode("overwrite").option("mergeSchema", "true")...

# Gold
gold_df.write.format("delta").mode("overwrite").option("mergeSchema", "true")...
```

Because each run re-derives its output **entirely** from the current landing directory's files, re-running the pipeline against the same input files produces byte-for-byte the same Bronze/Silver/Gold content — there is no incremental merge logic to get out of sync. The one exception is **quarantine**, which appends (see below), because quarantine history is meant to accumulate across runs, not reflect only the latest run.

Gold-level deduplication

```
.dropDuplicates(["file_name", "member_id"])
```

Guards against a malformed or re-sent 834 file producing duplicate `HL` blocks for the same member within one file — Gold's grain is `(file_name, member_id)`, so duplicates collapse to one row regardless of how many times a Silver row for that member appeared.

OLTP idempotency: natural-key upserts

The PostgreSQL loader's reference-entity inserts use `ON CONFLICT ... DO UPDATE`, keyed on each table's natural unique key:

```
cur.execute(
    "INSERT INTO core.core_grgr_group (grgr_num, st_cd, eff_dt, crtd_ts, updt_ts) "
    "VALUES (%s,%s,%s,%s,%s) ON CONFLICT (grgr_num) DO UPDATE SET updt_ts=EXCLUDED.upd
t_ts "
```

```
    "RETURNING grgr_id",
    ...
)
```

Table	Natural key	Upsert behavior
`core_grgr_group`	`grgr_num`	Update `updt_ts` on conflict
`core_plpl_plan`	`plpl_cd`	Update `updt_ts` on conflict
`core_mesb_subscriber`	`sbsb_num`	Update `updt_ts` on conflict

Re-running `load_gold_to_oltp()` against the same Gold data therefore does **not** create duplicate groups, plans, or subscribers — only their `updt_ts` advances. (Member and enrollment rows are plain inserts in this simplified demo schema, not idempotent upserts — see Chapter 17 and Chapter 37 for the natural extension.)

OLAP idempotency: deterministic hashed keys

`DIM_TRANSACTION_SOURCE.TXN_ID` is derived with `F.abs(F.xxhash64("file_name"))` rather than an auto-incrementing counter — the same file name always produces the same `TXN_ID` across repeated notebook runs, so re-running the medallion job does not fork transaction-source identity for the same file.

Transactional atomicity

Both the OLTP loader and the OLTP schema deployment script wrap all work in a **single transaction**:

```
conn.autocommit = False
try:
    ...  # all inserts/upserts
    conn.commit()
except Exception:
    conn.rollback()
    raise
```

A failure partway through either operation leaves the database exactly as it was before the attempt — no partial writes to reconcile manually.

Incremental processing: Change Data Feed

Gold is written with `delta.enableChangeDataFeed: "true"`:

```
layers:
  gold:
    options:
      delta.enableChangeDataFeed: "true"
```

This enables a future consumer to read only rows that changed since a given Delta table version (`spark.read.format("delta").option("readChangeData", "true").option("startingVersion", N)`) instead of re-scanning the whole table — the current OLTP loader and report generator both still do a full `SELECT *` against Gold, so CDF is provisioned but not yet exploited; see Chapter 37 for making the OLTP load CDF-incremental.

File-level idempotency boundary

The Airflow DAG's `archive_blob` task moves this run's exact processed files out of `landing/834/` into `archive/834/` only after the core processing chain (push -> Databricks -> OLTP) succeeds (default `all_success` trigger rule, wired directly to that chain's XCom outputs — see Chapter 23). This means a given 834 file is only ever re-processed if it is manually moved back into `landing/` — the pipeline itself does not re-ingest archived files on a subsequent scheduled run, which is what keeps daily runs idempotent at the file level even though Bronze/Silver/Gold are always full `overwrite`s of "whatever is currently in landing." If the core chain fails instead, `quarantine_failed_files` moves the same files to `failed/834/` rather than archiving or re-listing `landing/834/` from scratch.

23. Error Handling & Retry

Airflow-level retries

Every task in `med834_e2e_pipeline` inherits a blanket retry policy from `default_args`:

```
DEFAULT_ARGS = {"owner": "med834enrl", "retries": 1, "retry_delay": timedelta(minutes
=3)}
```

A single transient failure (a Databricks API hiccup, a momentary network blip to blob storage) is automatically retried once, 3 minutes later, before the task — and the whole DAG run — is marked failed.

Explicit failure propagation from Databricks

`run_databricks_job` treats a non-`SUCCESS` Databricks job result as a hard Python exception, so Airflow's retry/alerting machinery applies uniformly to both "the API call failed" and "the job ran but produced a bad result":

```
@task
def run_databricks_job(_count: int) -> int:
    run = WorkspaceClient().jobs.run_now_and_wait(job_id=job_id)
    state = run.state.result_state.value if run.state and run.state.result_state else
"UNKNOWN"
    if state != "SUCCESS":
        raise RuntimeError(f"Databricks job failed: {state}")
    return run.run_id
```

Transactional rollback (OLTP)

The OLTP loader and schema deployment script both catch broadly and roll back on **any** exception, re-raising afterward so Airflow still sees the task as failed:

```
try:
    ...  # all inserts/upserts in one transaction
    conn.commit()
except Exception:
    conn.rollback()
    raise
finally:
    cur.close()
    conn.close()
```

This guarantees the OLTP database never ends up partially loaded — a retry of `load_oltp_database` after a rollback starts from the same pre-load state, re-running the (idempotent, see Chapter 22) upsert logic cleanly.

Per-query error isolation in reporting

The HTML report generator does **not** let one failing analytics query take down the whole report — each `REPORT_QUERIES` entry is wrapped individually:

```
for title, sql in REPORT_QUERIES:
    try:
        cols, rows = _run_query(w, wid, sql)
        sections.append(_table_html(title, cols, rows))
    except Exception as exc:
        sections.append(f"<h2>{title}</h2><p style='color:red'>Error: {exc}</p>")
```

A broken query renders as a visible red error message inline in the generated HTML report instead of crashing the `generate_report` task and blocking the rest of the report's sections — a deliberate degrade-gracefully choice for an analytics/reporting path (as opposed to the OLTP load, which intentionally fails hard and rolls back, because operational data integrity matters more than reporting completeness).

Operational failure handling: archive vs. quarantine

```
@task
def archive_blob(keys: list[str]) -> int:
    ...  # default all_success trigger rule

@task(trigger_rule="one_failed")
def quarantine_failed_files(keys: list[str] | None) -> int:
    ...
```

Unlike an earlier `trigger_rule="all_done"` design (which re-listed `landing/834/` from scratch and could archive a file Databricks had actually failed to process), the archive/quarantine split is now both **run-scoped** and **failure-aware**:

- `archive_blob` has the **default** `all_success` trigger rule and is wired directly to `push_to_uc_volume` / `load_oltp_database`'s own XCom outputs (not to the optional `generate_report` / `ask_genie` branch), so Airflow automatically skips it if any task in the core processing chain failed — it is structurally impossible for it to archive a file that Databricks or the OLTP load never actually succeeded on.

- `quarantine_failed_files` uses `trigger_rule="one_failed"` and is wired to every task in that same core chain (`upload_to_blob`, `push_to_uc_volume`, `run_databricks_job`, `load_oltp_database`). Airflow's `ONE_FAILED` rule fires whenever a direct upstream is `FAILED` **or** `UPSTREAM_FAILED` — confirmed against the Airflow 3.3.0 `TriggerRuleDep` source — so a failure anywhere in the chain reaches this task even if it only manifests several hops downstream as `UPSTREAM_FAILED`. It moves this run's exact files to `failed/834/` instead of leaving them ambiguous in `landing/834/`.
- Both tasks operate on the **exact blob keys this run uploaded** (`upload_to_blob` returns them, threaded unchanged through `push_to_uc_volume` via XCom) rather than re-listing the bucket, so a concurrent run's files are never touched by either path.
- A missing/`None` `keys` XCom (e.g. `upload_to_blob` itself is what failed, so it never returned anything) is handled as a no-op in `quarantine_failed_files`, not an error — there is nothing to move.

The DAG run as a whole is still reported as failed if any upstream task failed; archive/quarantine only decide what happens to the landing files, not the DAG run's final state.

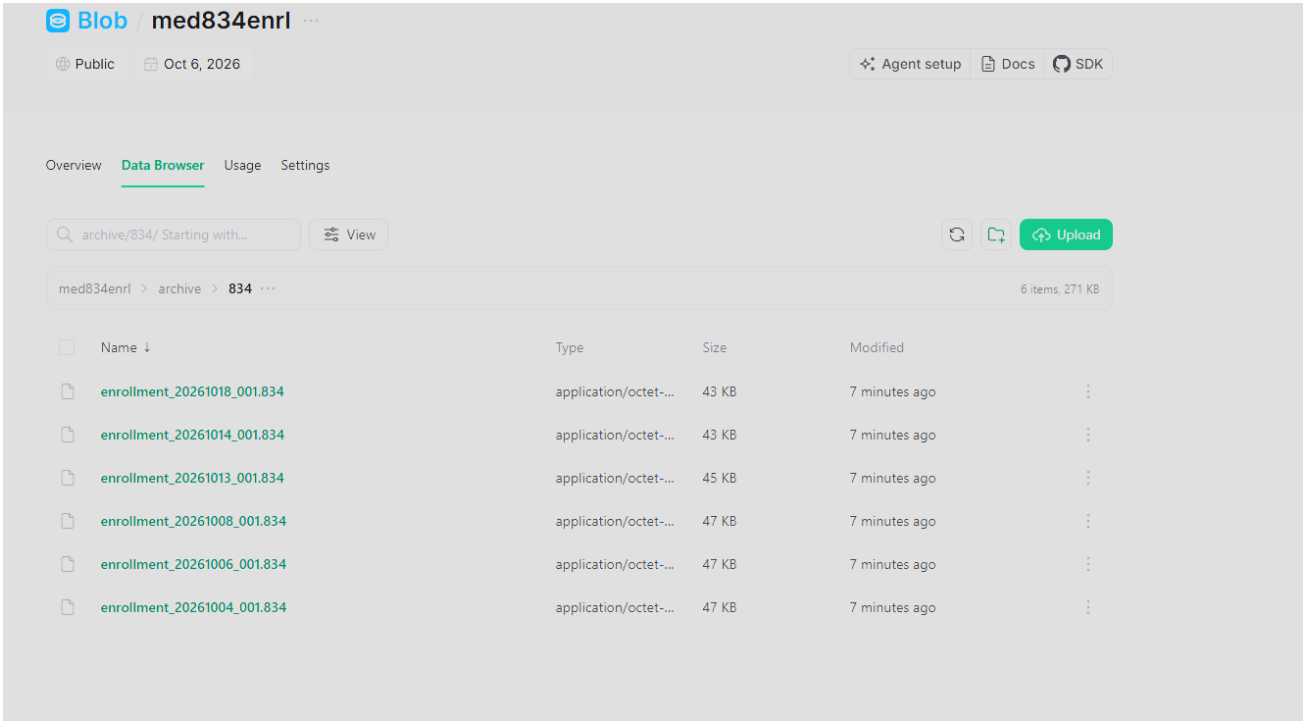
Validation errors are not failures

A critical distinction in this pipeline: a member record failing **data-quality validation** is not a pipeline failure — it is expected, handled output. `is_valid=False` rows are quarantined (see Chapter 16), not raised as exceptions; only infrastructure/connectivity/job-level problems trigger Airflow's retry and failure-reporting path.

Known error classes and their handling

Failure	Where caught	Behavior
Databricks job non-`SUCCESS`	`run_databricks_job` task	`RuntimeError` → Airflow retry (1x) → DAG failure
OLTP connection/constraint error	`load_gold_to_oltp()`	Transaction rollback → re-raise → Airflow retry
Blob storage token expired/invalid	any `blob_storage.*` call	`requests` / `boto3` exception → Airflow retry
Single bad analytics SQL query	`generate_report()`	Caught per-query, rendered as an error row, report still completes
Member record fails validation	`SilverLayer._apply_validation()`	Not an exception — routed to quarantine (see Ch. 16)
Databricks SQL warehouse read failure	`_run_query()` / `_read_gold_from_databricks()`	`RuntimeError` with the Databricks error message, propagated up
Core chain (push/Databricks/OLTP) failure	`quarantine_failed_files` (`trigger_rule="one_failed"`)	This run's exact files moved to `failed/834/`; never archived

Verified archive evidence



24. Observability

MedEnroll834's observability is deliberately lightweight for a POC — structured, configurable logging with run-id correlation, pipeline-level run summaries, and Airflow's own task UI — rather than a separate metrics/tracing stack, but every signal that exists is real and wired into actual code paths.

Standardized, configurable logging

Source: `src/medenroll834/common/logging.py`

```
def setup_logging(name: str = "med834enrl") -> logging.Logger:
    """Console handler always on; env-driven level + optional rotating file
    handler. Every record also gets run_id/file_name injected via a
    logging.Filter backed by a contextvar (see run_context() below)."""
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger
    level = _resolve_level()  # MED834_LOG_LEVEL, default INFO
    logger.setLevel(level)
    formatter = logging.Formatter(LOG_FORMAT, datefmt=DATE_FORMAT)

    stream_handler = logging.StreamHandler(sys.stdout)
    stream_handler.setFormatter(formatter)
    stream_handler.addFilter(_CONTEXT_FILTER)
    logger.addHandler(stream_handler)

    if os.environ.get("MED834_LOG_FILE"):   # optional rotating file handler
        ...
    return logger

def log_pipeline_start(logger, layer: str, details: str) -> float:
    logger.info("START %s | %s | %s", layer, details, datetime.now(timezone.utc).isofo
rmat())
    return time.monotonic()  # feed into log_pipeline_end/_error for duration

def log_pipeline_end(logger, layer: str, details: str, start_time=None) -> None:
    if start_time is not None:
        details = f"{details} | duration_s={time.monotonic() - start_time:.2f}"
    logger.info("END %s | %s | %s", layer, details, datetime.now(timezone.utc).isoform
at())

def log_pipeline_error(logger, layer: str, details: str, start_time=None) -> None:
    """Call from an except block -- logs full traceback via logger.exception()."""
    ...
```

Env vars: `MED834_LOG_LEVEL` (default `INFO`), `MED834_LOG_FILE` (optional, adds a `RotatingFileHandler` alongside stdout), `MED834_LOG_MAX_BYTES`, `MED834_LOG_BACKUP_COUNT`. Wired into `docker-compose.yaml` for the Airflow containers and documented in `.env.example`.

Every layer (`BronzeLayer`, `SilverLayer`, `GoldLayer`, `DeltaQuarantineManager`, the OLTP loader, the report generator) calls `log_pipeline_start` / `log_pipeline_end` with a layer name and a details string that includes the actual row count and elapsed duration, e.g.:

```
2026-10-04 10:40:23 [INFO] bronze | run=3f9a21 file=- - START BRONZE | landing_path=1
0_data/raw/ | 2026-10-04T10:40:23+00:00
2026-10-04 10:40:25 [INFO] bronze | run=3f9a21 file=- - END BRONZE | files=2 segments_
loaded=83 | duration_s=1.84 | 2026-10-04T10:40:25+00:00
2026-10-04 10:40:26 [INFO] silver | run=3f9a21 file=- - END SILVER | total=6 valid=6 i
nvalid=0 | duration_s=0.92 | 2026-10-04T10:40:26+00:00
2026-10-04 10:40:27 [INFO] gold | run=3f9a21 file=- - END GOLD | records=6 | duration_
s=0.41 | 2026-10-04T10:40:27+00:00
```

Logger names (`bronze`, `silver`, `gold`, `quarantine`, `pipeline`, `validator`, `oltp_loader`, `report_generator`, `s3_utils`, `blob_storage`, `edi834_parser`, `airflow_dag`) make it trivial to additionally `grep`/filter logs by component, and the `logger.handlers` guard in `setup_logging()` prevents duplicate log lines if it is called more than once for the same name within a process.

Pipeline run-context correlation

> `run_context()` / `set_run_context()` in `common/logging.py`

Every log line above carries `run=3f9a21` — a `contextvars.ContextVar` bound once per pipeline invocation and injected into every `LogRecord` via a `logging.Filter`, so Bronze/Silver/Gold/Quarantine (each of which calls `setup_logging()` independently, since they are separate classes instantiated by the orchestrator) share one correlation id without threading `run_id` through every method signature:

```
# src/medenroll834/ingestion/pipeline.py
def run(self, landing_path: str, run_id: Optional[str] = None) -> Dict:
    run_id = run_id or uuid.uuid4().hex[:12]
    with run_context(run_id=run_id):
        ...  # every log line emitted by bronze/silver/gold/quarantine
             # during this `with` block carries run=<run_id>
```

The Airflow DAG does the same per-task, binding the Airflow `dag_run.run_id` (via `get_current_context()["run_id"]`) so a human can `grep` the Airflow task log and the pipeline module logs together for one DAG run, and forwards that same id to the Databricks

job as a `job_parameters={"airflow_run_id": ...}` value — the notebook reads it back via `dbutils.widgets.get("airflow_run_id")` and includes it in its own `print()` lines, closing the loop from Airflow UI -> task log -> Databricks job run log.

Error logging

`log_pipeline_error()` is called from every layer's top-level `except` block before re-raising, so failures are never silently swallowed: it logs via `logger.exception()`, which captures the full traceback plus the elapsed duration up to the point of failure. Modules with no "layer" concept (`s3_utils`, `blob_storage`, `oltp_loader`'s Databricks read, `report_generator`'s Genie call) use `logger.exception()` / `logger.error()` directly at each external I/O call site (S3/blob upload-download-archive, SQL warehouse statement execution, Genie conversation) with the operation's key identifiers (bucket/key, warehouse_id, space_id) in the message -- credentials/tokens are never included.

Pipeline-level run summaries

`DeltaMedallionPipeline.run()` returns a structured summary dict rather than just logging — this makes the pipeline's outcome programmatically consumable (by a test, by a future Airflow XCom, by a monitoring script):

```
summary = {
    "run_id": run_id,
    "bronze_segments": bronze_df.count(),
    "silver_total": silver_df.count(),
    "silver_valid": valid_df.count(),
    "silver_invalid": invalid_df.count(),
    "gold_records": gold_df.count(),
}
```

The Databricks notebook job produces an equivalent structured result and exits with it explicitly:

```
result["airflow_run_id"] = RUN_ID
print("MEDALLION RESULT:", json.dumps(result, indent=2))
dbutils.notebook.exit(json.dumps(result))
```

`dbutils.notebook.exit()` makes this result retrievable from the **Databricks Jobs API** by anything that calls the job (including the Airflow task), without needing to parse log output.

Airflow-native observability

- **Task graph UI** (`localhost:8080`) shows per-task state (queued/running/success/failed/retried) for every DAG run.
- **Per-task logs** are captured automatically by Airflow and viewable per task instance; the DAG's own tasks use `logging.getLogger`-based loggers (not bare `print()`) so their output is level-tagged and run-id-tagged consistently with the pipeline's own `setup_logging()` output, which still appears within the same task log.
- **DAG import error detection**: `airflow dags list-import-errors` — used as a CI-style sanity check (`07_scripts/03_utils/validate_dag.py`) before deploying a DAG change.

OLTP and OLAP lineage columns

Both stores carry lineage back to the originating 834 file:

- OLTP: every `core_meen_enrollment` row FKs to an `enrl_edxn_transaction.txn_id`, which stores `file_nm`, `rcvd_ts`, and `proc_stat`.
- OLAP: `DIM_TRANSACTION_SOURCE.FILE_NAME` + `FACT_ENROLLMENT.TXN_SRC_KEY` trace every fact row back to its source file.
- Delta: `file_name` is carried as a column through Bronze → Silver → Gold → quarantine, so any row can be traced to its source file without a separate lineage table.

What's deliberately out of scope for this POC

No Prometheus/Grafana metrics stack, no distributed tracing (OpenTelemetry), and no alerting integration are wired up for this POC. See Chapter 37 for the monitoring roadmap.

25. Spark Performance Optimization

Every Spark tuning decision documented here is an **actual configuration or code change in this repo**, most captured with an inline comment explaining the real failure it fixed — not generic Spark tuning advice.

Adaptive Query Execution (AQE)

```
 .config("spark.sql.adaptive.enabled", "true")
.config("spark.sql.adaptive.coalescePartitions.enabled", "true")
.config("spark.sql.shuffle.partitions", "8")
```

AQE lets Spark re-plan shuffle partition counts at runtime based on actual data size rather than a fixed guess, and `coalescePartitions` merges small post-shuffle partitions automatically — important for this pipeline's small-to-medium local/POC data volumes where a fixed large partition count would create excessive task overhead. `shuffle.partitions=8` is tuned down from Spark's default of 200, appropriate for local/`local[*]` execution rather than a large cluster.

Broadcast map instead of a self-join (biggest algorithmic win)

The single largest performance/correctness fix in the codebase: resolving each dependent's subscriber via `hl_parent_id` / `hl_id` was originally a **self-join**, which fanned out catastrophically because HL ids repeat across transaction sets (see Chapter 14):

```
 # Implemented with a broadcast map rather than a self-join. A join here
# duplicated every dependent against every candidate subscriber and
# blew the executor heap; a broadcast lookup is O(1) per row and cannot
# fan out because the key set is collapsed with first().
subscriber_map = {
    f"{r['hl_key']}|{r['hl_id']}": r["member_id"]
    for r in base.filter(F.col("is_subscriber"))
    .select("hl_key", "hl_id", "member_id")
    .dropDuplicates(["hl_key", "hl_id"])
    .collect()
}
```

Collecting only the (small) distinct-subscriber set and broadcasting it as a Spark `create_map()` expression turns an O(n²)-shaped join into an O(1) per-row map lookup — this scales with the number of **subscribers**, not the number of **members**, which is the correct scaling dimension for this workload.

Window-based "carry forward latest value" instead of a loop

```
file_order = Window.partitionBy("file_name").orderBy(F.col("seg_row"))
ordered = ordered.withColumn(
    "txn_row",
    F.last(F.when(F.col("segment_id") == "ST", F.col("seg_row")), ignorenulls=True)
     .over(file_order.rowsBetween(Window.unboundedPreceding, Window.currentRow)),
)
```

A previous version range-joined every segment against every `ST` row to find "the most recent transaction start," which matched a segment against **all preceding** transaction sets and multiplied member rows roughly 5x. A cumulative-window `last(..., ignorenulls=True)` carries the most recent value forward in a single pass with no join at all.

Delta write-time optimization

```
# Bronze
.option("delta.autoOptimize.optimizeWrite", "true")
```

Compacts small files at write time — relevant because 834 ingestion produces many small per-segment row batches per file; without `optimizeWrite`, Bronze would accumulate a large number of tiny Parquet files under the Delta table over many pipeline runs, degrading future read performance.

Delta retention tuning

```
silver:
  options:
    delta.deletedFileRetentionDuration: "interval 7 days"
```

Keeps a 7-day time-travel window on Silver for debugging/auditing without retaining deleted files indefinitely (the Delta default is 30 days via `VACUUM`'s retention check) — a deliberate storage-cost vs. debuggability tradeoff for this layer specifically.

Change Data Feed for future incremental reads

```
gold:
  options:
    delta.enableChangeDataFeed: "true"
```

Enabled proactively so a future consumer can read only changed rows since a given version instead of a full re-scan — see Chapter 22 for the current state (not yet exploited by the OLTP loader) and Chapter 37 for the roadmap.

Databricks-side: `read_files()` + `explode()` over a Python loop

The Databricks notebook job reads landing files with a single distributed-SQL statement rather than iterating files in Python, because it must scale across a cluster reading many files from a Unity Catalog Volume at once (see Chapter 13):

```
raw = spark.sql(
    f"SELECT value, element_at(split(_metadata.file_path, '/'), -1) AS file_name "
    f"FROM read_files('{LANDING}', format => 'text', wholeText => true)"
)
bronze = raw.withColumn("segment", F.explode(F.split(F.regexp_replace("value", r"[\r\n
]+", "~"), "~"))) ...
```

`monotonically_increasing_id()` is used for `seg_order` in this distributed path (rather than a Python `enumerate()`), since it can be computed without collecting data back to the driver.

26. Benchmark Framework

> **What exists vs. what is proposed**
>
> This repository does **not** currently ship a dedicated, automated benchmark harness (e.g. a script that runs N file sizes and plots throughput). What exists is the **instrumentation a benchmark framework would consume** — pipeline run summaries, per-layer logging, and a validated smoke-test baseline. This chapter documents that real instrumentation, then proposes a concrete framework design that is not yet implemented. See Chapter 34 for the actual measured numbers vs. the legacy design targets.

What already exists to build a benchmark on

1. Structured per-run summaries

`DeltaMedallionPipeline.run()` returns row counts for every layer on every run:

```
summary = {
    "bronze_segments": bronze_df.count(),
    "silver_total": silver_df.count(),
    "silver_valid": valid_df.count(),
    "silver_invalid": invalid_df.count(),
    "gold_records": gold_df.count(),
}
```

This is already a timed, structured artifact — wrapping the call in `time.perf_counter()` before/after gives per-run wall-clock duration for free, with no change to pipeline code.

2. Timestamped layer-boundary logs

`log_pipeline_start` / `log_pipeline_end` emit ISO-8601 timestamps at the start and end of every layer (Bronze, Silver, Gold, Quarantine):

```
START BRONZE | landing_path=10_data/raw/ | 2026-10-04T10:40:23+00:00
END BRONZE   | segments_loaded=83        | 2026-10-04T10:40:25+00:00
```

Parsing these gives **per-layer** duration breakdowns (Bronze vs. Silver vs. Gold time), which is more actionable than a single end-to-end number when deciding where to optimize.

3. A known, reproducible smoke-test baseline

```
BRONZE     : segments_loaded = 83
SILVER     : total = 6, valid = 6, invalid = 0
GOLD       : records = 6
QUARANTINE : records = 0
```

(1 synthetic file, 3 subscribers + 3 dependents — see `PIPELINE_STATUS.md`). This is a real, run result, suitable as a regression baseline: any future change that alters these counts for the same input is a signal worth investigating.

Proposed framework design (not yet implemented)

A natural next step, using only what's already in the repo:

```
# proposed: 07_scripts/03_utils/benchmark_pipeline.py
import time
from generator.synthetic_834_generator import generate_batch
from src.medenroll834.ingestion.pipeline import DeltaMedallionPipeline
from src.medenroll834.common.spark_session import get_local_spark

SIZES = [(1, 10), (5, 100), (10, 1000), (20, 5000)]  # (files, subscribers_per_file)

def run_benchmark():
    spark = get_local_spark("benchmark")
    results = []
    for num_files, subs in SIZES:
        generate_batch("10_data/bench_raw", num_files=num_files, subscribers_per_file=
subs)
        pipeline = DeltaMedallionPipeline(spark)
        start = time.perf_counter()
        summary = pipeline.run("10_data/bench_raw")
        elapsed = time.perf_counter() - start
        results.append({"files": num_files, "subs": subs, "elapsed_s": elapsed, **summ
ary})
    return results
```

This would reuse `generate_batch()` (Chapter 10) and `DeltaMedallionPipeline` (Chapter 18) exactly as-is — no pipeline code changes needed, only a new orchestrating script that scales input size and records elapsed time per run.

What a benchmark should measure

Dimension	Metric	Source
Volume	Records/sec at increasing file sizes	`summary["gold_records"] / elapsed_s`
Velocity	End-to-end wall-clock time per run	`time.perf_counter()` around `pipeline.run()`
Variety	Validation pass rate as plan-code/field variety increases	`silver_valid / silver_total`
Reliability	Quarantine rate under `--messy` synthetic data	`silver_invalid / silver_total`

See Chapter 34 for how these dimensions map back to the legacy system's documented constraints.

27. Databricks SQL Dashboard

Source: `src/medenroll834/reporting/report_generator.py`

Rather than a separate BI tool, MedEnroll834's analytics layer runs a fixed set of SQL queries against Databricks SQL warehouses and renders them as a single self-contained HTML report — a deliberately simple "traditional analytics" consumption path that pairs with Genie's "AI analytics" path (see Chapter 4).

The seven report queries

```
REPORT_QUERIES: List[Tuple[str, str]] = [
    ("Total enrollments", "SELECT COUNT(*) AS total FROM med834enrl.edi.gold_enrollmen
ts"),
    ("Members by plan",
     "SELECT plan_id, COUNT(*) AS members FROM med834enrl.edi.gold_enrollments "
     "GROUP BY plan_id ORDER BY members DESC"),
    ("Members by state",
     "SELECT state, COUNT(*) AS members FROM med834enrl.edi.gold_enrollments "
     "WHERE state IS NOT NULL GROUP BY state ORDER BY members DESC LIMIT 15"),
    ("Subscriber vs dependent",
     "SELECT is_subscriber, COUNT(*) AS members FROM med834enrl.edi.gold_enrollments "
     "GROUP BY is_subscriber"),
    ("Average age by plan",
     "SELECT plan_id, ROUND(AVG(age_years),1) AS avg_age FROM med834enrl.edi.gold_enro
llments "
     "GROUP BY plan_id ORDER BY avg_age DESC"),
    ("Enrollments by plan (FACT_ENROLLMENT star schema)",
     "SELECT p.PLAN_CODE, COUNT(*) AS enrollments "
     "FROM med834enrl.gold.FACT_ENROLLMENT f "
     "JOIN med834enrl.gold.DIM_PLAN p ON f.PLPL_KEY = p.PLPL_KEY "
     "GROUP BY p.PLAN_CODE ORDER BY enrollments DESC"),
    ("Enrollments by group and plan (FACT_ENROLLMENT star schema)",
     "SELECT gr.GROUP_NUMBER, p.PLAN_CODE, COUNT(*) AS enrollments "
     "FROM med834enrl.gold.FACT_ENROLLMENT f "
     "JOIN med834enrl.gold.DIM_GROUP gr ON f.GRGR_KEY = gr.GRGR_KEY "
     "JOIN med834enrl.gold.DIM_PLAN p ON f.PLPL_KEY = p.PLPL_KEY "
     "GROUP BY gr.GROUP_NUMBER, p.PLAN_CODE ORDER BY enrollments DESC LIMIT 15"),
]
```

Notice the first five queries hit the **flat** `gold_enrollments` table directly, while the last two join through the **OLAP star schema** (`FACT_ENROLLMENT` + `DIM_PLAN` / `DIM_GROUP`) — demonstrating both consumption shapes from the same analytics entry point.

Executing against a SQL warehouse

```
def _run_query(w, warehouse_id, sql):
    from databricks.sdk.service.sql import StatementState
    resp = w.statement_execution.execute_statement(
        warehouse_id=warehouse_id, statement=sql, wait_timeout="50s"
    )
    while resp.status.state in (StatementState.PENDING, StatementState.RUNNING):
        time.sleep(2)
        resp = w.statement_execution.get_statement(resp.statement_id)
    if resp.status.state != StatementState.SUCCEEDED:
        raise RuntimeError(f"Query failed: {resp.status.error.message}")
    return [c.name for c in resp.manifest.schema.columns], resp.result.data_array or [
]
```

Uses the Databricks SQL Statement Execution API — the serverless warehouse spins up on demand if not already running, with a 2-second poll loop while `PENDING` / `RUNNING`.

Rendering self-contained HTML

```
def generate_report(profile=None, warehouse_id=None) -> str:
    w = _client(profile)
    wid = _warehouse_id(w, warehouse_id)
    sections = []
    for title, sql in REPORT_QUERIES:
        try:
            cols, rows = _run_query(w, wid, sql)
            sections.append(_table_html(title, cols, rows))
        except Exception as exc:
            sections.append(f"<h2>{title}</h2><p style='color:red'>Error: {exc}</p>")
    return (
        "<!DOCTYPE html><html>...<title>MedEnroll834 Enrollment Report</title>..."
        f"<p>Generated: {generated} | Source: med834enrl.edi.gold_enrollments</p>"
        + "".join(sections) + "</body></html>"
    )
```

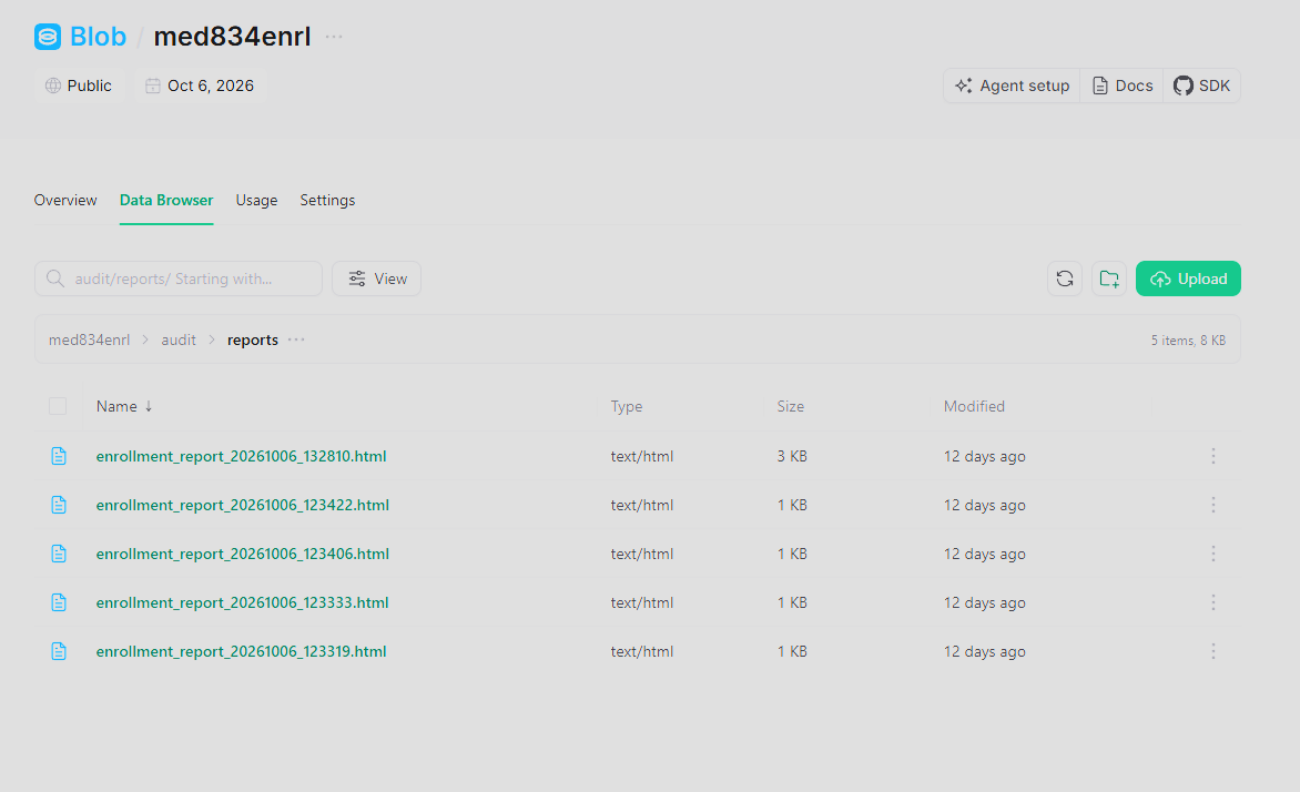
The report is a single HTML string with inline CSS — no external dependencies, no JavaScript — so it can be written straight to blob storage (`audit/reports/enrollment_report_<timestamp>.html`) and opened in any browser with no server required. Each query's failure is isolated (see Chapter 23), so one broken query renders a red error message inline rather than failing the whole report.

Invoked from the Airflow DAG

```
@task
def generate_report(_summary: dict) -> str:
    from src.medenroll834.reporting.report_generator import generate_report as gen
    from src.medenroll834.storage import blob_storage
    html = gen(warehouse_id=_cfg("DATABRICKS_WAREHOUSE_ID"))
    stamp = datetime.utcnow().strftime("%Y%m%d_%H%M%S")
    uri = blob_storage.put_text(f"audit/reports/enrollment_report_{stamp}.html",
                                html, content_type="text/html")
    return uri
```

Each run produces a uniquely timestamped report file — historical reports accumulate under `audit/reports/` rather than overwriting each other.

Verified report evidence



28. Genie / GenAI

Source: `src/medenroll834/reporting/report_generator.py` (`ask_genie()`)

Databricks Genie is the pipeline's **natural-language analytics** consumer — the second of the two analytical consumption modes described in Chapter 4. It queries the exact same governed Gold/OLAP tables as the HTML report; it is not a different data source and not a pipeline transformation step.

Implementation

```
def ask_genie(space_id: str, question: str, profile=None) -> Dict:
    """Ask a Databricks Genie space a natural-language question.

    Requires a Genie space (space_id) in a workspace with Genie enabled.
    Returns the final message payload. Best-effort: raises if Genie API or
    space is unavailable.
    """
    w = _client(profile)
    conv = w.genie.start_conversation_and_wait(space_id=space_id, content=question)
    out = {"question": question, "attachments": []}
    for att in (conv.attachments or []):
        if att.text:
            out["attachments"].append({"type": "text", "content": att.text.content})
        if att.query:
            out["attachments"].append({"type": "query", "query": att.query.query,
                                       "description": att.query.description})
    return out
```

`start_conversation_and_wait()` blocks until Genie has processed the question and returns, synchronously, a list of **attachments** — each either a natural-language text answer or a generated SQL query (with description) that Genie ran to produce that answer. Surfacing the generated SQL alongside the text answer is valuable for auditability: a reviewer can see exactly what query Genie executed against the governed Gold/OLAP data, not just trust an opaque answer.

Invoked from the Airflow DAG (optional)

```
@task
def ask_genie(_report_uri: str) -> dict:
    space_id = _cfg("GENIE_SPACE_ID")
    if not space_id:
        print("GENIE_SPACE_ID not set - skipping Genie step")
        return {"skipped": True}
    from src.medenroll834.reporting.report_generator import ask_genie as genie
    qs = ["How many members are enrolled on each plan?",
          "What is the subscriber to dependent ratio?"]
    answers = [genie(space_id, q) for q in qs]
    return {"answers": answers}
```

Two representative questions are asked on every DAG run when `GENIE_SPACE_ID` is configured — chosen because they mirror two of the HTML report's own queries ("Members by plan", "Subscriber vs dependent"), demonstrating that the NL path and the fixed-SQL path answer the same underlying business questions through different interfaces.

Enabling Genie

From the E2E Pipeline Runbook:

1. In Databricks: **Genie → New space**, add `med834enrl.edi.gold_enrollments` and `med834enrl.gold.*` tables as the space's data.
2. Copy the space id from the URL and set `GENIE_SPACE_ID` in `.env`.
3. Re-run the DAG; `ask_genie` now queries the same governed Gold/OLAP data as the HTML report.

Why Genie is optional, not required

`GENIE_SPACE_ID` is intentionally unset by default — the pipeline runs fully end-to-end (generation → ingestion → medallion → OLTP → HTML report → archive) with Genie entirely skipped. This reflects its position as an **additional** analytical consumption mode layered on top of a complete, independently-functioning pipeline, not a required dependency of it.

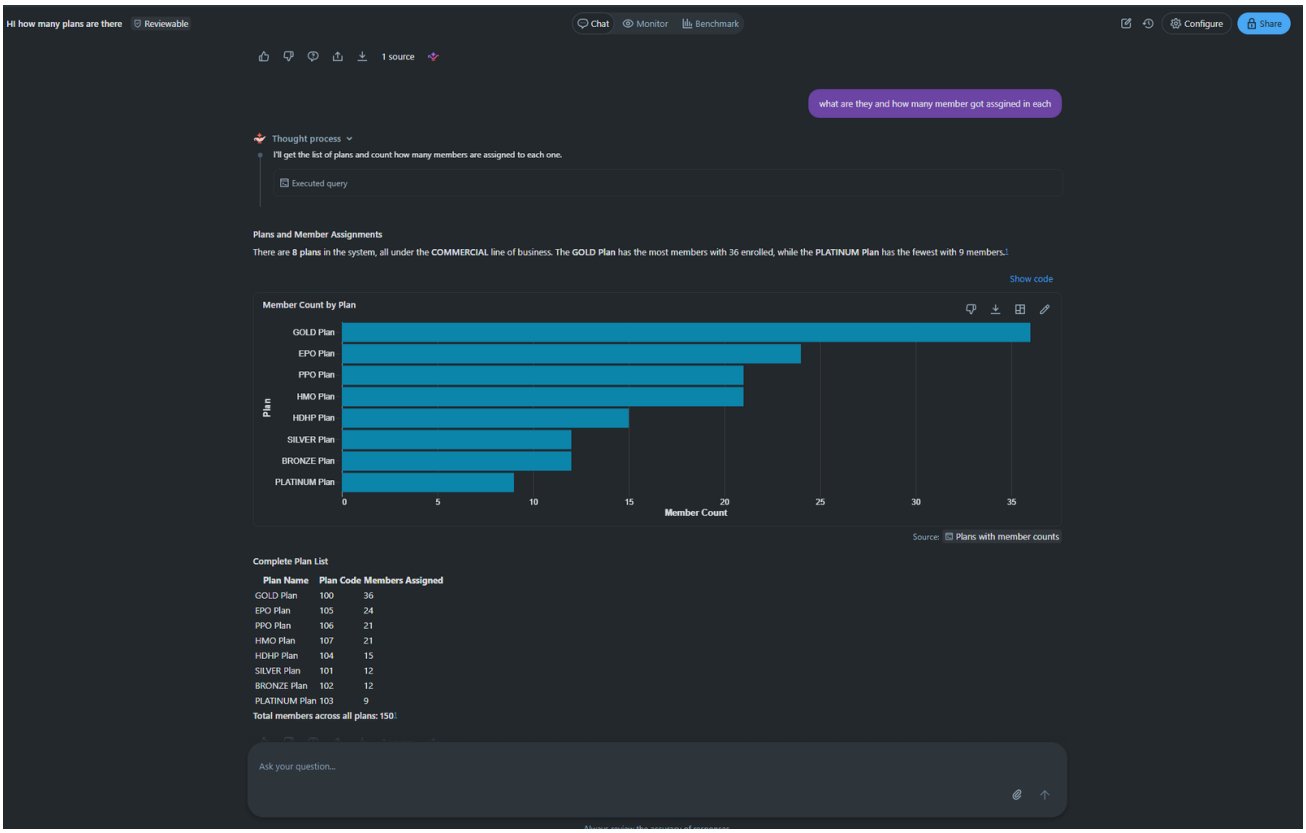
Positioning note (why this matters architecturally)

A common modernization-story mistake is to treat an AI/NL feature as part of the data pipeline's transformation logic. This repo deliberately keeps Genie downstream of Gold/OLAP, alongside — not inside — the HTML reporting path, so:

- Pipeline reliability (Bronze/Silver/Gold/OLTP) never depends on Genie's availability.

- Both consumption modes are governed by the same data quality rules applied upstream (see Chapter 16) — Genie cannot "see" data that never passed validation.
- Swapping or removing Genie has zero blast radius on the rest of the system — see Chapter 4 for the architectural framing this reflects.

Verified Genie evidence



29. Security

Credential handling

Blob storage credentials

Source: `src/medenroll834/storage/blob_storage.py`

```
def _client_and_bucket():
    """Mint fresh short-lived creds and return (s3_client, bucket_name)."""
    token = os.environ["BLOB_STORAGE_TOKEN"]
    creds = requests.post(
        CREDENTIALS_URL, headers={"Authorization": f"Bearer {token}"}, timeout=30
    ).json()
    s3 = boto3.client("s3", endpoint_url=creds["endpoint"], region_name="auto",
        aws_access_key_id=creds["accessKeyId"],
        aws_secret_access_key=creds["secretAccessKey"],
        aws_session_token=creds["sessionToken"])
    return s3, creds["bucket"]
```

A single long-lived `BLOB_STORAGE_TOKEN` is used only to mint short-lived, scoped credentials on each call — see Chapter 5 for why a third-party blob provider was used in place of AWS S3 for this proof of concept.

AWS enterprise-target credentials

```
def _client():
    return boto3.client("s3", region_name=os.environ.get("AWS_REGION", "us-east-1"))
```

Resolved through boto3's default credential chain — environment variables, shared config, or (in a real deployment) an IAM role — never hardcoded.

Secrets live only in `.env` (git-ignored)

```
# .gitignore
.env
```

All secrets (`BLOB_STORAGE_TOKEN` , `OLTP_DATABASE_URL` , `DATABRICKS_TOKEN` , AWS keys) live in `.env` , which is git-ignored; `.env.example` ships as the template with **no real values**. `docker-compose.yaml` interpolates these as `${VAR}` references rather than inlining them.

IAM least privilege (S3)

> Source: `04_infrastructure/01_terraform/s3_pipeline_least_privilege_policy.json`

```
{
  "Statement": [
    { "Sid": "ListTheBucket", "Effect": "Allow", "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::med834enrl-s3" },
    { "Sid": "ReadWriteObjects", "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::med834enrl-s3/*" }
  ]
}
```

No `s3:*` , no cross-bucket access, no bucket-policy modification rights. This IAM policy and its Terraform-managed bucket (encryption, versioning, public-access blocking — see Chapter 7) apply once a deployment moves to the enterprise AWS S3 target; a reader on a standard paid AWS account does not need to configure anything beyond attaching this policy to the pipeline's IAM role.

Database connection security

`OLTP_DATABASE_URL` requires `?sslmode=require` for the managed Postgres provider used in this POC — enforced by convention/documentation (see the PostgreSQL Setup chapter) rather than programmatically, since `psycopg2.connect()` accepts the sslmode as part of the connection string itself.

Transactional integrity as a security property

Both the OLTP schema deployment and the OLTP data load wrap all statements in a single transaction with rollback on any exception (see Chapter 23) — this prevents a failed deploy or load from leaving the database in a state where constraints/FKs are partially applied, which could otherwise be an integrity/availability risk.

PHI/PII posture

This project processes **zero production member data** — every 834 file is synthetically generated (see Chapter 10). That said, the schema and pipeline are structured as if the data were real PHI/PII, per `SECURITY.md` :

- Compliance with HIPAA regulations is a stated design goal for any production deployment of this architecture.
- Data encryption at rest (S3 SSE, Delta/Parquet on encrypted storage) and in transit (`sslmode=require`, HTTPS-only S3 policy) are both already enforced at the infrastructure level documented above.
- Access logging/auditing is intentionally left as a future improvement — out of scope for this POC's synthetic-data threat model.

CI security scanning

`.github/workflows/ci.yml` runs a dedicated `security` job on every push/PR:

```
security:
  steps:
    - run: pip install bandit safety
    - name: Run Bandit
      run: bandit -r src/ -f json -o bandit-report.json
    - name: Run Safety
      run: safety check --json > safety-report.json
```

`bandit` statically scans `src/` for common Python security anti-patterns (e.g. `eval`, hardcoded secrets, insecure deserialization); `safety` checks installed dependencies against the PyUp/OSV known-vulnerability database. See Chapter 31 for the full CI pipeline.

30. Testing

Source: `02_tests/conftest.py`, `02_tests/test_pipeline.py`

MedEnroll834 tests against real PySpark + Delta and real synthetic 834 files (via `generate_batch()`, see Chapter 10) — every test exercises the actual transformation logic end to end against generated data rather than mocked DataFrames.

Test coverage by layer

Test class	What it verifies
`TestEDI834Parser`	`parse_files()` returns a non-empty Bronze DataFrame with `segment_id` / `elements` columns
`TestSilverLayer`	Silver produces `member_id` / `is_valid` columns; synthetic (non- `--messy`) data is always valid
`TestDataFrameValidator`	Missing `member_id` and unknown `plan_id` are each correctly flagged invalid
`TestGoldLayer`	Gold deduplicates and derives `full_name` / `age_years`

Running the suite

```
pytest 02_tests/ -v --cov=src --cov-report=term-missing
# or
make test
```

DAG validation (outside pytest)

```
# 07_scripts/03_utils/validate_dag.py
from airflow.models.dagbag import DagBag
db = DagBag("/opt/airflow/dags")
# writes IMPORT_ERRORS and DAGS to /tmp/dagresult.txt
```

Run inside the live Airflow scheduler container to catch DAG-definition syntax/import errors before relying on the UI — this is how the "0 import errors" claim in Chapter 21 is verified.

What is and isn't covered

Covered	Not covered (yet)
Parser, Silver validation/transform, Gold dedup/derive	OLTP loader (`oltp_loader.py`) — requires a live Postgres + Databricks warehouse
DataFrameValidator rules	Report generator / Genie (`report_generator.py`) — requires a live Databricks workspace
DAG import/parse correctness	End-to-end Airflow task execution (covered manually via the runbook, not CI)

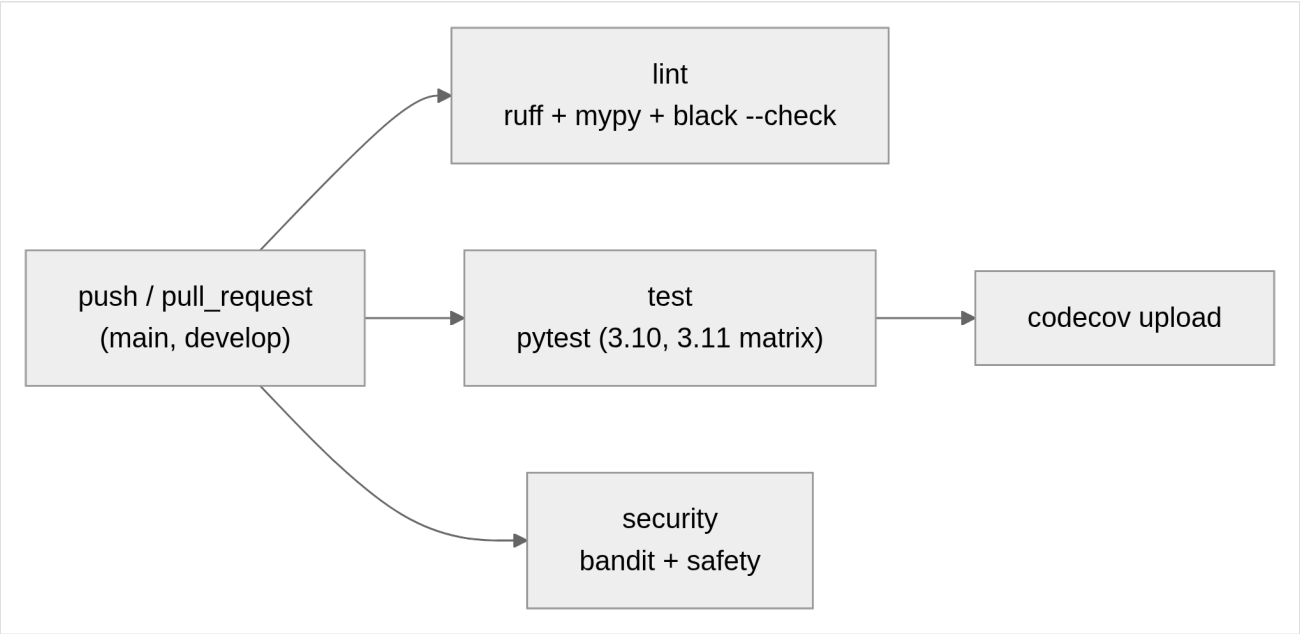
See Chapter 37 for extending coverage to the OLTP/reporting modules with a mocked `databricks-sdk` / `psycopg2` .

31. CI/CD

Source: `.github/workflows/ci.yml`, `.github/workflows/cd.yml`

CI pipeline (`ci.yml`)

Triggered on every push/PR to `main` or `develop`, with three parallel jobs:




```yaml
jobs:
  lint:
    steps:
      - run: pip install -r requirements.txt
      - run: ruff check src/ 02_tests/
      - run: mypy src/
      - run: black --check src/ 02_tests/

  test:
    strategy:
      matrix:
        python-version: ['3.10', '3.11']
    steps:
      - run: pytest 02_tests/ -v --cov=src --cov-report=xml --cov-report=term-missing
      - uses: codecov/codecov-action@v3

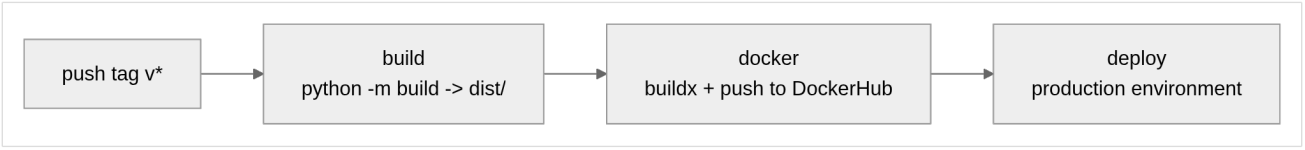
  security:
    steps:
      - run: pip install bandit safety
      - run: bandit -r src/ -f json -o bandit-report.json
      - run: safety check --json > safety-report.json
```

- **`lint`** — `ruff` (fast linter), `mypy` (static typing), `black --check` (formatting, no auto-fix in CI).
- **`test`** — runs across a **Python 3.10 / 3.11 matrix**, uploads coverage to Codecov.
- **`security`** — `bandit` (static security scan of `src/`) and `safety` (dependency vulnerability check) — see Chapter 29.

`lint` and `test` both reference `src/` and `02_tests/`, matching `pyproject.toml`'s `testpaths = ["02_tests"]` and the `Makefile`'s own `pytest 02_tests/ -v` target exactly — an earlier version of this workflow referenced a non-existent `tests/` directory, which meant CI would not actually find or run the test suite; that mismatch has been fixed.

CD pipeline (`cd.yml`)

Triggered only on version tags (`v*`):




```
jobs:
  build:
    steps:
      - run: pip install build twine
      - run: python -m build
      - uses: actions/upload-artifact@v3
        with: { name: dist, path: dist/ }

  docker:
    needs: build
    steps:
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        with: { context: ., push: true, tags: ..., labels: ... }

  deploy:
    needs: [build, docker]
    environment: production
    steps:
      - run: echo "Deployment steps would go here"
```

The `deploy` job is a **documented placeholder** — it runs in a GitHub Environment named `production` (so it can be gated with required reviewers/approval rules in repo settings) but

the actual deployment command (`kubectl apply` / `helm upgrade`) is intentionally commented out, since this POC has no Kubernetes/Helm target yet.

Local equivalents (Makefile)

```
install: pip install -r requirements.txt && pip install -e .
test:    pytest 02_tests/ -v --cov=src --cov-report=term-missing
lint:    ruff check src/ 02_tests/ && mypy src/
format:  black src/ 02_tests/ && ruff check --fix src/ 02_tests/
```

Running `make lint test` locally before pushing exercises the same ruff/mypy/pytest checks CI runs, catching most CI failures before they happen.

32. Docker / Local Development

Starting the stack

```
docker compose up -d
# Airflow UI: http://localhost:8080 (admin/admin)
```

`docker-compose.yaml` brings up `postgres` (Airflow metadata), `redis` (Celery broker), `airflow-webserver`, `airflow-scheduler`, `airflow-worker`, and a one-shot `airflow-init` — see Chapter 20 for the full topology.

Wait ~2 minutes after first start for `_PIP_ADDITIONAL_REQUIREMENTS` (`databricks-sdk`, `boto3`, `psycopg2-binary`) to install into the containers, then check:

```
docker compose ps
```

Verifying the stack is healthy

```
docker exec medenroll834-airflow-scheduler-1 airflow dags list-import-errors
# -> No data found   (0 import errors)

docker exec medenroll834-airflow-scheduler-1 airflow dags list | Select-String med834
# -> med834_e2e_pipeline | /opt/airflow/dags/med834_e2e_pipeline.py | med834enrl | ...
```

Dockerfile (standalone package image)

The repo also ships a standalone `Dockerfile` for building a distributable image of the package itself (used by the CD pipeline's `docker` job — see Chapter 31), separate from the `docker-compose.yaml` Airflow development stack.

Backing out the local stack

> Source: `07_scripts/02_deployment/backout_docker_stack.ps1`

```
# Stop and remove containers, keep volumes (metadata/logs preserved)
.\07_scripts\02_deployment\backout_docker_stack.ps1

# Also wipe the airflow-logs and postgres-db-volume named volumes
.\07_scripts\02_deployment\backout_docker_stack.ps1 -Full
```

```
if ($Full) {
    docker compose down -v
} else {
    docker compose down
}
```

Redeploy afterward with `docker compose up -d`.

Full environment teardown (all deployed components)

> Source: `08_docs/04_operations_deployment/BACKOUT_RUNBOOK.md`

Every deployed component — Docker stack, Databricks (medallion + OLAP), OLTP database, and blob storage content — has a matching, idempotent backout script:

```
.\07_scripts\02_deployment\backout_docker_stack.ps1 -Full
python 07_scripts/02_deployment/backout_databricks.py --profile <profile>
python 07_scripts/02_deployment/backout_oltp_database.py "$env:OLTP_DATABASE_URL"
python 07_scripts/02_deployment/backout_blob_storage.py
```

All backout SQL uses `IF EXISTS` / `DROP ... CASCADE`, so re-running a backout that already ran is a safe no-op — components are independent and can be backed out in any order or individually.

Suggested teardown order (if backing out everything)

1. **Docker stack** — stop the orchestrator first so nothing re-triggers jobs while you're tearing down its dependencies.
2. **Databricks** — medallion tables, OLAP star schema, job, notebook.
3. **OLTP database** — CORE/ENRL/UTIL schemas.
4. **Blob storage** — landing/archive/report objects.

Verifying a backout

```
# Databricks - should error "table/schema not found"
python 07_scripts/03_utils/verify_e2e_data.py

# Docker - no medenroll834-* containers
docker ps -a --filter "name=medenroll834"
```

Local Python development (without Docker)

```
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
pytest 02_tests/ -v
```

This path exercises the PySpark/Delta pipeline directly against local files under `10_data/`, without Airflow or Databricks — useful for fast iteration on parser/Silver/Gold logic (see Chapter 11).

33. End-to-End Execution

> Source: `08_docs/04_operations_deployment/E2E_PIPELINE_RUNBOOK.md`

This is the complete, real runbook for running `med834_e2e_pipeline` against live Databricks/OLTP/blob storage infrastructure — it implements both the **Distributed Processing** and **Analytical Platform** capabilities from Chapter 4.

Flow

```
Generate 834 -> blob storage landing -> Databricks UC Volume
  -> Databricks medallion (bronze/silver/gold) -> OLAP star schema
  -> OLTP database load -> HTML report + Genie (analytical consumers)
  -> archive in blob storage
```

Already-provisioned prerequisites

- Databricks catalog `med834enrl` with `edi` Delta tables + `gold` OLAP star schema (`DIM_MEMBER`, `DIM_PLAN`, `DIM_GROUP`, `DIM_DATE`, `DIM_TRANSACTION_SOURCE`, `FACT_ENROLLMENT`)
- Databricks notebook: `/Shared/med834enrl/med834_medallion_job`
- Databricks Job id and a SQL warehouse id
- OLTP database schema (CORE/ENRL) deployed (see Chapter 8)
- Blob storage bucket (token-based, mints short-lived S3-compatible creds)
- Airflow DAG `med834_e2e_pipeline` (validated, 0 import errors)

Step 1 — Fill in secrets in `.env`

```
BLOB_STORAGE_TOKEN=...      # POC landing/archive/report object store
DATABRICKS_TOKEN=...        # Personal Access Token (containers can't use browser OAut
h)
OLTP_DATABASE_URL=...       # already set to the OLTP med834enrl database
GENIE_SPACE_ID=             # optional
```

Generate a Databricks PAT via CLI:

```
$db = "$env:LOCALAPPDATA\Microsoft\WinGet\Packages\Databricks.DatabricksCLI_Microsoft
.Winget.Source_8wekyb3d8bbwe\databricks.exe"
& $db tokens create --comment airflow --lifetime-seconds 7776000
```

Step 2 — Recreate the Airflow stack

```
docker compose down
docker compose up -d
```

See Chapter 32 for what this stack contains and how to verify it's healthy.

Step 3 — Open Airflow and run

- UI: `http://localhost:8080` (admin/admin)
- Unpause and trigger DAG **`med834_e2e_pipeline`**
- Watch the task graph: `generate → upload_to_blob → push_to_uc_volume → databricks → oltp → report → genie → archive` (see Chapter 21 for what each task does)

Step 4 — Verify outputs

Store	What to check
Blob storage	`archive/834/` (processed files), `audit/reports/enrollment_report_*.html`
Databricks (OLAP)	`med834enrl.edi.gold_enrollments`, `med834enrl.gold.dim_*`, `med834enrl.gold.fact_enrollment`
OLTP database	`core.core_meme_member`, `core.core_meen_enrollment`, `enrl.enrl_edxn_transaction`

Re-deploying the Databricks notebook after a code change

```
$db = "...\databricks.exe"
& $db workspace import --language PYTHON --format SOURCE --overwrite `
  --file 05_orchestration/databricks/med834_medallion_job.py `
  /Shared/med834enrl/med834_medallion_job
```

Troubleshooting

Symptom	Cause / fix
Blob storage 401/expired	`BLOB_STORAGE_TOKEN` credentials expire ~10 min; re-minted automatically per call — a persistent 401 means the token itself is invalid/revoked
Databricks 403	PAT expired/invalid — regenerate and update `.env`
OLTP database connection failure	ensure `?sslmode=require` is in `OLTP_DATABASE_URL`
DAG not visible in UI	`docker compose logs airflow-scheduler | Select-String error`
pip deps missing in container	wait for `_PIP_ADDITIONAL_REQUIREMENTS` startup install, or `docker compose restart`

Enterprise migration note

To move from the POC landing layer to the enterprise target, replace the `upload_to_blob` / `archive_blob` tasks with the already-implemented AWS S3 equivalents in `src/medenroll834/s3/s3_utils.py`, and update the Databricks job's landing path accordingly. The Bronze-Silver-Gold, OLTP, OLAP, BI, and AI layers require **no changes** — see Chapter 7.

Local (non-Airflow) smoke test

For fast iteration without the full Airflow stack:

```
python -c "import sys; sys.path.insert(0,'.'); from generator.synthetic_834_generator
 import generate_batch; generate_batch('10_data/raw', num_files=3, subscribers_per_fil
e=10)"
python 07_scripts/03_utils/run_e2e_smoke.py
```

This exercises `DeltaMedallionPipeline` directly against local Delta tables — the same validated baseline referenced throughout this documentation:

```
BRONZE     : segments_loaded = 83
SILVER     : total = 6, valid = 6, invalid = 0
GOLD       : records = 6
QUARANTINE : records = 0
```

34. Legacy vs Modern Benchmark

> **Measured vs. design target**
>
> This chapter clearly separates **numbers actually measured** in this repository from the legacy system's **documented constraint**, which was never independently re-measured against the new pipeline at equivalent scale. Presenting one as the other would be misleading — see Chapter 26 for how to close this gap with real throughput testing.

Legacy system (documented constraint)

> Source: Chapter 2, business-problem framing.

Enrollment processing jobs in the legacy system are observed to take approximately **one hour** end to end for a full daily batch file, across the full EDI → ECA → Legacy Programs → SCP → CORE Programs chain. This number describes the *existing production workflow's* behavior as reported by the business, not a benchmark run against this repository's code.

Modern pipeline (measured, smoke-test scale)

> Source: `PIPELINE_STATUS.md`, "VERIFIED: End-to-End Run" section.

A real, reproducible run of this repository's `DeltaMedallionPipeline` against 1 synthetic file (3 subscribers + 3 dependents) produced:

```
BRONZE     : segments_loaded = 83
SILVER     : total = 6, valid = 6, invalid = 0
GOLD       : records = 6
QUARANTINE : records = 0
```

All four Delta tables were written with a valid `_delta_log`. This confirms **functional correctness** end to end (parsing, HL resolution, validation, dedup, Delta writes) — it is not yet a throughput/scale measurement at anything near the legacy system's full daily batch volume.

Qualitative comparison (architecturally grounded)

Dimension	Legacy	Modern (this repo)
Processing model	Serial, single-machine, 6+ distinct tools (EDI/ECA/Legacy Programs/SCP/Bash/CORE Programs)	Distributed PySpark/Delta, 3-layer medallion (see Ch. 13–18)
Parallelism	None — one file processed at a time, one machine	Spark partitions data across cores/executors automatically (AQE, see Ch. 25)
Failure isolation	A failure in any of 6 tools (EDI/ECA/Legacy Programs/SCP/Bash/CORE Programs) stops the chain with separate logs per tool	Per-layer logging with structured `log_pipeline_start/end` (see Ch. 24); invalid *records* (not whole files) are quarantined, not fatal (see Ch. 16)
Recoverability	Manual investigation across 6 tool logs	Idempotent re-runs (see Ch. 22); Airflow automatic retry (see Ch. 23)
Analytical access	None — CORE System keyword-file load only	OLAP star schema + Databricks SQL + Genie (see Ch. 19, Ch. 27–28)
Data lineage	Proprietary intermediate format, no shared trace	`file_name` carried through every layer; `ENRL_EDXN_TRANSACTION` / `DIM_TRANSACTION_SOURCE` FK every record to its source file
Observability	6 separate tool-specific logs	Unified structured logging + per-run summary dicts (see Ch. 24)
Infrastructure as Code	Manually configured Tidal/SSIS/DataStage jobs	Terraform (S3), SQL DDL scripts, Docker Compose, all version-controlled

What a rigorous head-to-head benchmark would require

1. Generate synthetic 834 data at a representative daily-batch scale using `generate_batch()` with a large `subscribers_per_file`.
2. Run `DeltaMedallionPipeline.run()` against it on a representative Databricks cluster size, timed with the methodology proposed in Chapter 26.
3. Compare wall-clock time directly against the legacy system's ~1 hour figure, at the same record count.

This has not yet been executed in this repository — the honest claim this documentation makes is about **architecture** (distributed vs. serial, quarantine vs. hard-fail, governed analytics vs. none), not a measured speedup multiplier.

35. Business Impact

Velocity

The legacy chain's six-tool handoff (EDI → ECA → Legacy Programs → SCP → Bash → CORE Programs, see Chapter 3) means enrollment changes are only as fast as the slowest, most fragile link in that chain. The modern pipeline collapses this into a single orchestrated DAG (`med834_e2e_pipeline`) with Airflow-managed retries, reducing the number of places a delay or failure can silently accumulate. Faster, more predictable enrollment processing means coverage is active sooner — which matters directly to members submitting claims against a plan that hasn't finished loading yet.

Volume

Distributed PySpark processing (see Chapter 25) scales horizontally with file/record count, rather than being bound to a single machine's throughput ceiling the way the legacy chain is. The business impact is a pipeline whose capacity grows with the Databricks cluster/warehouse size provisioned, not with the age and specs of a specific on-prem server.

Variety

A single validation ruleset (`DataFrameValidator`, Chapter 16) applied consistently across every record — rather than variety handling being spread implicitly across 6 different tools' individual quirks — means a new plan code, relationship type, or malformed field is caught and explained (`validation_errors`) in one place, by one team, instead of surfacing as an opaque downstream CORE System load failure.

Operational cost reduction

- **Fewer tools to license, patch, and staff** — Tidal, SSIS, DataStage, Legacy Programs, SCP, and Bash scripts are five-to-six separate operational surfaces; the modern stack is Airflow + Databricks + PostgreSQL.
- **Unified observability** reduces mean-time-to-diagnosis: a structured log line (`START SILVER | ...`) and a per-run summary dict (`silver_invalid=12`) replace grepping six different tool-specific log formats (see Chapter 24).
- **Idempotent re-runs** (see Chapter 22) mean a failed run can simply be retried rather than requiring manual reconciliation of partial state.

New analytical capability (not present in the legacy system at all)

The legacy flow terminates at a CORE System keyword-file load — there is no historical reporting, BI, or ad hoc analytics path. The modern platform adds, at no cost to the operational path:

- An **OLAP star schema** (`FACT_ENROLLMENT` + 5 dimensions, Chapter 19) purpose-built for enrollment analytics.
- A **traditional analytics** path: scheduled HTML reports over fixed KPIs (Chapter 27).
- An **AI analytics** path: natural-language ad hoc exploration via Genie (Chapter 28), answering questions like "How many members are enrolled on each plan?" without writing SQL.

This is a genuinely new business capability, not a modernization of an existing one — the legacy architecture has no equivalent to show stakeholders a trend in enrollment-by-plan over time, for example.

Compliance and audit posture

Every enrollment record and every rejected record traces back to its source file via a transaction identifier (`ENRL_EDXN_TRANSACTION` / `DIM_TRANSACTION_SOURCE`, see Chapter 24) — a stronger audit trail than the legacy proprietary intermediate format allows, which matters directly for HIPAA-adjacent compliance reviews (see Chapter 29).

36. Portfolio README

A condensed, portfolio/resume-ready version of this project's README — use this chapter as the elevator-pitch text for a GitHub profile, LinkedIn project entry, or portfolio site.

MedEnroll834 — Modern Distributed Data Platform for Growing Data

A proof-of-concept healthcare data platform that demonstrates how distributed processing modernizes HIPAA X12 834 enrollment transaction processing — built to show the same architectural pattern (distributed processing + OLTP/OLAP separation + BI/AI consumption) generalizes beyond this one business module.

Highlights

- **Distributed medallion pipeline** (Bronze → Silver → Gold) in PySpark 4.0+ / Delta Lake, parsing real X12 834 (5010) EDI structure including the `HL` hierarchy, segment extraction, and date normalization.
- **Dual persistence**: PostgreSQL OLTP for operational reads/writes, a Delta Lake star schema (`FACT_ENROLLMENT` + 5 dimensions) for analytics — deliberately kept architecturally separate.
- **Dual analytics consumption**: a traditional SQL/HTML reporting path and a Databricks Genie natural-language AI path, both querying the same governed Gold/OLAP data.
- **Apache Airflow 3.x** orchestration (TaskFlow API, CeleryExecutor) with retries, run-scoped archive-on-success / quarantine-on-failure handling, and Airflow Variable config overrides.
- **Data quality by design**: Spark-SQL validation rules producing multi-reason error arrays, with invalid records quarantined (not dropped) to a dedicated Delta table.
- **Idempotent throughout**: Delta overwrite semantics, PostgreSQL natural-key upserts, deterministic hashed surrogate keys.
- **Infrastructure as Code**: Terraform-managed S3 bucket + least-privilege IAM policy, versioned SQL DDL for both OLTP and OLAP schemas, full Docker Compose local dev stack.
- **CI/CD**: GitHub Actions — lint (ruff/mypy/black), test (pytest matrix, Codecov), security scan (bandit/safety), build/Docker publish on tag.
- **Zero production data**: all 834 files are synthetically generated with a custom X12-compliant generator (realistic name/address/plan pools, valid ISA checksums).

Tech stack

`PySpark` `Delta Lake` `Apache Airflow` `Databricks SQL` `Databricks Genie` `PostgreSQL` `Docker` `Terraform` `GitHub Actions` `Python 3.10+`

What this demonstrates

Skill area	Evidence
Distributed data engineering	Medallion architecture, window-function-based hierarchy resolution, broadcast-map optimization over a fan-out join
Data modeling	Normalized OLTP schema + dimensional OLAP star schema, both from first principles
Orchestration	TaskFlow DAG with explicit retry/failure semantics
Data quality engineering	Multi-rule validation with structured, actionable rejection reasons
Cloud/infra	Terraform IaC, least-privilege IAM, Docker Compose
AI integration	Genie natural-language analytics as a governed consumer of curated data
Software engineering practice	CI/CD, typed config, structured logging, idempotent design, documented runbooks

Links

- Full documentation (this site) — chapters covering every layer of the system in depth, generated from and kept consistent with the actual source code.
- Modern Architecture
- End-to-End Execution Runbook

> **Using this chapter**
>
> Copy the sections above directly into a project's top-level `README.md`, a portfolio site project card, or a resume project bullet list — they're written to stand alone outside this documentation's context.

37. Future Improvements

A concrete, prioritized roadmap grounded in gaps and TODOs actually identified while building this documentation — not a generic wishlist.

Testing

- **Extend coverage to `oltp_loader.py` and `report_generator.py`** — both currently require a live Databricks workspace and PostgreSQL instance to exercise (see Chapter 30). Mocking `databricks.sdk.WorkspaceClient` and `psycopg2.connect` would let these modules run in CI without live infrastructure.
- **Add a true end-to-end Airflow task-execution test** — current DAG validation (`validate_dag.py`) only checks import/parse correctness, not that tasks actually execute correctly against mocked dependencies.
- **Add `--messy` synthetic data test cases** so the quarantine path (Chapter 16) has automated test coverage, not just manual verification.

Performance & benchmarking

- **Implement the benchmark framework proposed in Chapter 26** — scale synthetic data generation across a range of file/record counts and measure actual wall-clock throughput, to get a real number against the legacy system's documented ~1 hour per batch constraint (see Chapter 34).
- **Exploit Delta Change Data Feed for incremental OLTP loads** — `gold` is already written with `delta.enableChangeDataFeed: "true"`, but `load_gold_to_oltp()` still does a full `SELECT *` against Gold on every run. Switching to `readChangeData` / `startingVersion` would make repeated OLTP loads proportional to what changed, not the full table size.

Data model

- **Make `core_meme_member` / `core_meen_enrollment` upsertable** — these are currently plain inserts (see Chapter 17), unlike the natural-key-upserted group/plan/subscriber tables. A natural key (e.g. `mesb_id + rltn_cd + eff_dt`) would let member/enrollment loads be fully idempotent rather than relying on upstream Gold deduplication alone.
- **Add SCD Type 2 history tracking to the OLTP schema**, mirroring the `CURRENT_FLAG` pattern already used in the OLAP dimensions (Chapter 9), so an OLTP member's historical plan changes are queryable, not just the current state.

Observability

- **Add a `09_monitoring/` metrics/alerting stack** (e.g. Prometheus + Grafana, or a lightweight Airflow-native alerting integration) — none currently exists (see Chapter 24).
- **Add structured JSON logging** (vs. the current human-readable-formatted log lines) so logs can be ingested by a log aggregation platform without custom parsing.
- **Add access/audit logging** for PHI/PII-adjacent operations, per the compliance posture discussed in Chapter 29 — not yet implemented since this POC processes only synthetic data.

Architecture

- **Complete the AWS S3 migration** described in Chapter 7 and Chapter 33 — swap `upload_to_blob` / `archive_blob` DAG tasks for the already-built `src/medenroll834/s3/s3_utils.py` equivalents, eliminating the POC's blob-storage-to-UC-Volume staging hop entirely (a Databricks job with an instance profile can read S3 directly).
- **Generalize the parser** to other X12 transaction types (837 claims, 270/271 eligibility) to validate the "reusable platform" claim made in Chapter 1 beyond the 834 module alone.
- **CD pipeline's `deploy` job** is currently a documented placeholder (see Chapter 31) — wiring an actual Kubernetes/Helm (or Databricks Asset Bundle) deployment target would complete the CI/CD story end to end.

Documentation

- **Add a benchmark results page** once Chapter 26's framework is implemented, with real throughput numbers and charts.
- **Keep this documentation in sync with code changes** going forward — treat `docs-site/docs/*.md` updates as a required part of any PR that changes the behavior a chapter documents (this is the core discipline of "docs-as-code").